

DevLab_ICM20948

1.0.0

Generated on Fri Jun 12 2026 11:58:24 for DevLab_ICM20948 by Doxygen 1.17.0

Fri Jun 12 2026 11:58:24

Chapter 1

Directory Hierarchy

1.1 Directories

examples	??
I2C_Accel	??
I2C_Accel.ino	??
I2C_Basic	??
I2C_Basic.ino	??
I2C_Magneto	??
I2C_Magneto.ino	??
SPI_Basic	??
SPI_Basic.ino	??
I2C_Accel	??
I2C_Accel.ino	??
I2C_Basic	??
I2C_Basic.ino	??
I2C_Magneto	??
I2C_Magneto.ino	??
SPI_Basic	??
SPI_Basic.ino	??
src	??
7Semi_I2C_Interface.h	??
7Semi_Interface.h	??
7Semi_SPI_Interface.h	??
BusIO_7Semi.h	??
DevLab_ICM20948.cpp	??
DevLab_ICM20948.h	??
ICM20948_platform.h	??
ICM20948_regs.h	??

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

BusIO_7Semi< Bus >	??
DevLab_ICM20948	??
ICM20948_IntEnableConfig	??
ICM20948_IntPinConfig	??
ICM20948_IntStatus	??
icm_plat_t	??
Interface_7Semi	??
I2C_Interface	??
SPI_Interface	??

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BusIO_7Semi< Bus >	??
DevLab_ICM20948	??
I2C_Interface	??
ICM20948_IntEnableConfig	??
ICM20948_IntPinConfig	??
ICM20948_IntStatus	??
icm_plat_t	??
Interface_7Semi	??
SPI_Interface	??

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

examples/I2C_Accel/I2C_Accel.ino	??
examples/I2C_Basic/I2C_Basic.ino	??
examples/I2C_Magneto/I2C_Magneto.ino	??
examples/SPI_Basic/SPI_Basic.ino	??
src/7Semi_I2C_Interface.h	??
src/7Semi_Interface.h	??
src/7Semi_SPI_Interface.h	??
src/BusIO_7Semi.h	??
src/DevLab_ICM20948.cpp	??
src/DevLab_ICM20948.h	??
src/ICM20948_platform.h	??
src/ICM20948_regs.h	??

Chapter 5

Directory Documentation

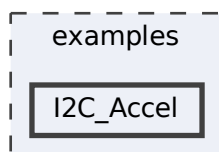
5.1 examples Directory Reference

Directories

- directory [I2C_Accel](#)
- directory [I2C_Basic](#)
- directory [I2C_Magneto](#)
- directory [SPI_Basic](#)

5.2 examples/I2C_Accel Directory Reference

Directory dependency graph for I2C_Accel:

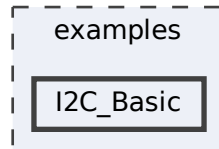


Files

- file [I2C_Accel.ino](#)

5.3 examples/I2C_Basic Directory Reference

Directory dependency graph for I2C_Basic:

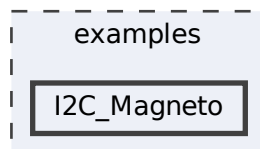


Files

- file [I2C_Basic.ino](#)

5.4 examples/I2C_Magneto Directory Reference

Directory dependency graph for I2C_Magneto:

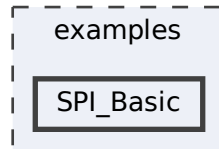


Files

- file [I2C_Magneto.ino](#)

5.5 examples/SPI_Basic Directory Reference

Directory dependency graph for SPI_Basic:



Files

- file [SPI_Basic.ino](#)

5.6 src Directory Reference

Files

- file [7Semi_I2C_Interface.h](#)
- file [7Semi_Interface.h](#)
- file [7Semi_SPI_Interface.h](#)
- file [BusIO_7Semi.h](#)
- file [DevLab_ICM20948.cpp](#)
- file [DevLab_ICM20948.h](#)
- file [ICM20948_platform.h](#)
- file [ICM20948_regs.h](#)

Chapter 6

Class Documentation

6.1 BusIO_7Semi< Bus > Class Template Reference

```
#include <BusIO_7Semi.h>
```

Public Member Functions

- [BusIO_7Semi](#) (Bus &busRef)
- bool [read](#) (uint8_t reg, uint8_t &value)
- bool [read](#) (uint8_t reg, uint16_t &value)
- bool [read](#) (uint8_t reg, uint8_t *data, uint32_t len)
- bool [write](#) (uint8_t reg, uint8_t value)
- bool [write](#) (uint8_t reg, uint16_t value)
- bool [write](#) (uint8_t reg, const uint8_t *data, uint32_t len)
- bool [readBits](#) (uint8_t reg, uint8_t pos, uint8_t len, uint8_t &value)
- bool [readBits](#) (uint8_t reg, uint8_t pos, uint8_t len, uint16_t &value)
- bool [writeBits](#) (uint8_t reg, uint8_t pos, uint8_t len, uint8_t value)
- bool [writeBits](#) (uint8_t reg, uint8_t pos, uint8_t len, uint16_t value)
- bool [readBit](#) (uint8_t reg, uint8_t pos, uint8_t &value)
- bool [readBit](#) (uint8_t reg, uint8_t pos, uint16_t &value)
- bool [writeBit](#) (uint8_t reg, uint8_t pos, uint8_t value)
- bool [writeBit](#) (uint8_t reg, uint8_t pos, uint16_t value)

6.1.1 Detailed Description

```
template<typename Bus>  
class BusIO_7Semi< Bus >
```

Definition at line 5 of file [BusIO_7Semi.h](#).

6.1.2 Constructor & Destructor Documentation

6.1.2.1 BusIO_7Semi()

```
template<typename Bus>
BusIO_7Semi< Bus >::BusIO_7Semi (
    Bus & busRef) [inline]
```

Constructor

- Stores interface reference

Definition at line 14 of file [BusIO_7Semi.h](#).

6.1.3 Member Function Documentation

6.1.3.1 read() [1/3]

```
template<typename Bus>
bool BusIO_7Semi< Bus >::read (
    uint8_t reg,
    uint16_t & value) [inline]
```

read 16-bit

Definition at line 23 of file [BusIO_7Semi.h](#).

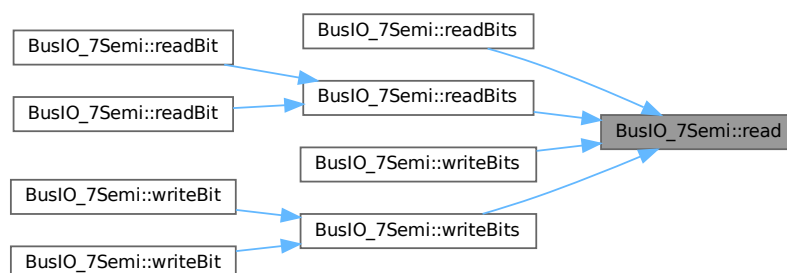
6.1.3.2 read() [2/3]

```
template<typename Bus>
bool BusIO_7Semi< Bus >::read (
    uint8_t reg,
    uint8_t & value) [inline]
```

read 8-bit

Definition at line 17 of file [BusIO_7Semi.h](#).

Here is the caller graph for this function:



6.1.3.3 read() [3/3]

```
template<typename Bus>
bool BusIO_7Semi< Bus >::read (
    uint8_t reg,
    uint8_t * data,
    uint32_t len) [inline]
```

read burst

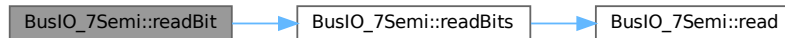
Definition at line 33 of file [BusIO_7Semi.h](#).

6.1.3.4 readBit() [1/2]

```
template<typename Bus>
bool BusIO_7Semi< Bus >::readBit (
    uint8_t reg,
    uint8_t pos,
    uint16_t & value) [inline]
```

Definition at line 142 of file [BusIO_7Semi.h](#).

Here is the call graph for this function:

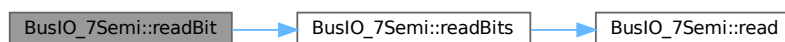


6.1.3.5 readBit() [2/2]

```
template<typename Bus>
bool BusIO_7Semi< Bus >::readBit (
    uint8_t reg,
    uint8_t pos,
    uint8_t & value) [inline]
```

Definition at line 137 of file [BusIO_7Semi.h](#).

Here is the call graph for this function:



6.1.3.6 readBits() [1/2]

```
template<typename Bus>
bool BusIO_7Semi< Bus >::readBits (
    uint8_t reg,
    uint8_t pos,
    uint8_t len,
    uint16_t & value) [inline]
```

readBits (16-bit register)

Definition at line 78 of file [BusIO_7Semi.h](#).

Here is the call graph for this function:



6.1.3.7 readBits() [2/2]

```
template<typename Bus>
bool BusIO_7Semi< Bus >::readBits (
    uint8_t reg,
    uint8_t pos,
    uint8_t len,
    uint8_t & value) [inline]
```

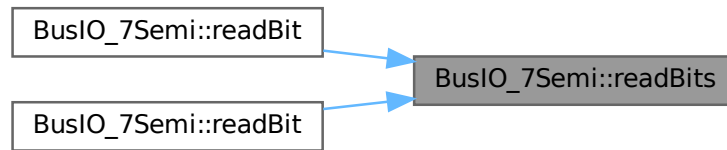
readBits (8-bit register)

Definition at line 60 of file [BusIO_7Semi.h](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.3.8 write() [1/3]

```

template<typename Bus>
bool BusIO_7Semi< Bus >::write (
    uint8_t reg,
    const uint8_t * data,
    uint32_t len) [inline]
  
```

write burst

Definition at line 52 of file [BusIO_7Semi.h](#).

6.1.3.9 write() [2/3]

```

template<typename Bus>
bool BusIO_7Semi< Bus >::write (
    uint8_t reg,
    uint16_t value) [inline]
  
```

write 16-bit

Definition at line 45 of file [BusIO_7Semi.h](#).

6.1.3.10 write() [3/3]

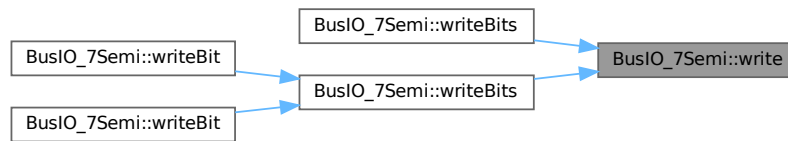
```

template<typename Bus>
bool BusIO_7Semi< Bus >::write (
    uint8_t reg,
    uint8_t value) [inline]
  
```

write 8-bit

Definition at line 39 of file [BusIO_7Semi.h](#).

Here is the caller graph for this function:



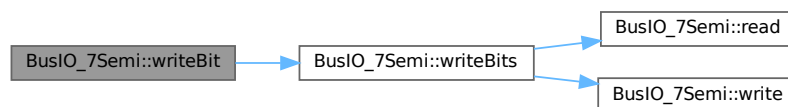
6.1.3.11 `writeBit()` [1/2]

```

template<typename Bus>
bool BusIO_7Semi< Bus >::writeBit (
    uint8_t reg,
    uint8_t pos,
    uint16_t value) [inline]
  
```

Definition at line 151 of file [BusIO_7Semi.h](#).

Here is the call graph for this function:



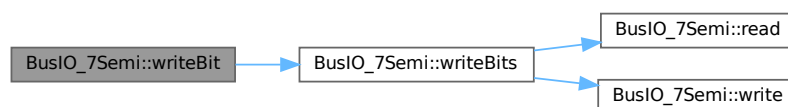
6.1.3.12 `writeBit()` [2/2]

```

template<typename Bus>
bool BusIO_7Semi< Bus >::writeBit (
    uint8_t reg,
    uint8_t pos,
    uint8_t value) [inline]
  
```

Definition at line 147 of file [BusIO_7Semi.h](#).

Here is the call graph for this function:



6.1.3.13 writeBits() [1/2]

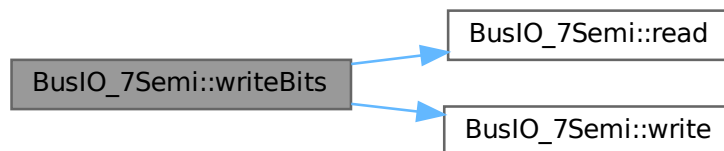
```
template<typename Bus>
bool BusIO_7Semi< Bus >::writeBits (
    uint8_t reg,
    uint8_t pos,
    uint8_t len,
    uint16_t value) [inline]
```

writeBits (16-bit register)

- Modifies specific bits in 16-bit register

Definition at line 120 of file [BusIO_7Semi.h](#).

Here is the call graph for this function:



6.1.3.14 writeBits() [2/2]

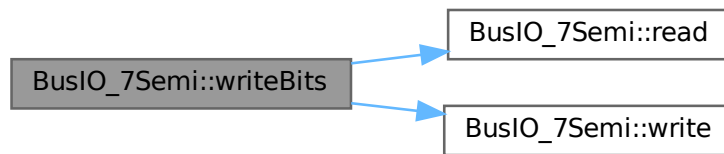
```
template<typename Bus>
bool BusIO_7Semi< Bus >::writeBits (
    uint8_t reg,
    uint8_t pos,
    uint8_t len,
    uint8_t value) [inline]
```

writeBits (8-bit register)

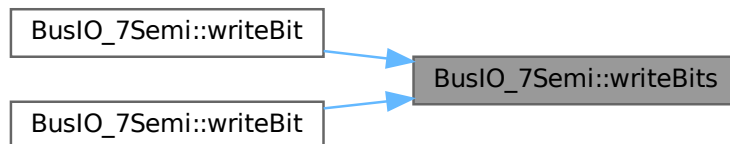
- Modifies specific bits in 8-bit register

Definition at line 98 of file [BusIO_7Semi.h](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following file:

- [src/BusIO_7Semi.h](#)

6.2 DevLab_ICM20948 Class Reference

```
#include <DevLab_ICM20948.h>
```

Public Member Functions

- [DevLab_ICM20948](#) ()
- bool [beginI2C](#) (uint8_t address=0x69, TwoWire &i2cPort=Wire, uint32_t i2cSpeed=400000)
- bool [beginSPI](#) (uint8_t csPin, SPIClass &spiPort=SPI, uint32_t spiSpeed=1000000)
- bool [readWhoAmI](#) (uint8_t &whoAmI)
- bool [selectBank](#) (uint8_t bank)
- bool [softReset](#) ()
- bool [sleep](#) (bool en)
- bool [applyBasicDefaults](#) ()
- bool [readAccel](#) (float &x, float &y, float &z)
- bool [readGyro](#) (float &x, float &y, float &z)
- bool [readTemperature](#) (float &temperature)

- bool [readMag](#) (float &x, float &y, float &z)
- bool [initMag](#) ()
- bool [setMagOpMode](#) (ICM20948_Op_Mode opMode)
- bool [setLowPower](#) (bool enable)
- bool [getLowPower](#) (bool &enable)
- bool [setClock](#) (ICM20948_Clock_Source clock)
- bool [getClock](#) (uint8_t &clock)
- bool [setGyroSampleRate](#) (float sampleRate)
- bool [getGyroSampleRate](#) (float &sampleRate)
- bool [setGyroDivRate](#) (uint8_t divisor)
- bool [setDLPF](#) (ICM20948_Gyro_DLPF dlpf, bool bypass)
- bool [getDLPF](#) (uint8_t &dlpf, bool &bypass)
- bool [setGyroScale](#) (ICM20948_Gyro_FullScale fullScale)
- bool [setGyroAveraging](#) (ICM20948_Gyro_Average avg)
- bool [getGyroScale](#) (uint8_t &fullScale)
- bool [selfTestGyro](#) (bool x, bool y, bool z)
- bool [setAccelDivRate](#) (uint16_t divisor)
- bool [setAccelSampleRate](#) (uint16_t sampleRate)
- bool [getAccelSampleRate](#) (float &sampleRate)
- bool [selfTestAccel](#) (bool x, bool y, bool z)
- bool [setAccelScale](#) (ICM20948_Accel_FullScale fullScale)
- bool [getAccelScale](#) (uint8_t &fullScale)
- bool [setAccelDLPF](#) (uint8_t dlpf, bool bypass)
- bool [getAccelDLPF](#) (uint8_t &dlpf, bool &bypass)
- bool [setAccelAveraging](#) (ICM20948_Accel_Average avg)
- bool [getAccelAveraging](#) (uint8_t &avg)
- bool [setSensors](#) (bool accel_on, bool gyro_on, bool temp_on)
- bool [getSensors](#) (bool &accel_on, bool &gyro_on, bool &temp_on)
- bool [setGyroOffset](#) (uint16_t offsetX, uint16_t offsetY, uint16_t offsetZ)
- bool [getGyroOffset](#) (int16_t &offsetX, int16_t &offsetY, int16_t &offsetZ)
- bool [setAccelOffset](#) (int16_t offsetX, int16_t offsetY, int16_t offsetZ)
- bool [getAccelOffset](#) (int16_t &offsetX, int16_t &offsetY, int16_t &offsetZ)
- bool [intInit](#) ()
- bool [checkIntStatus](#) (ICM20948_IntStatus &status)

6.2.1 Detailed Description

Class

- Manages ICM-20948 over I2C (SPI path disabled in this cut)
- Caches scale factors for LSB->physical conversions

Definition at line 149 of file [DevLab_ICM20948.h](#).

6.2.2 Constructor & Destructor Documentation

6.2.2.1 DevLab_ICM20948()

```
DevLab_ICM20948::DevLab_ICM20948 ()
```

- Construct with default I2C address; call begin() to attach bus

Definition at line 4 of file [DevLab_ICM20948.cpp](#).

6.2.3 Member Function Documentation

6.2.3.1 applyBasicDefaults()

```
bool DevLab_ICM20948::applyBasicDefaults ()
```

applyBasicDefaults

- Apply basic sensor configuration
- Configure power, filters, ranges, and sampling rates

Returns:

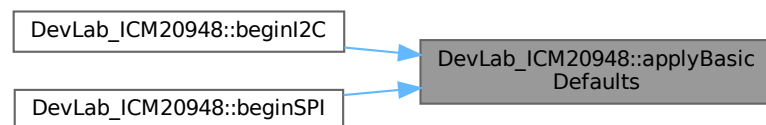
- true → Configuration successful
- false → Any register write failed

Definition at line 181 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.2.3.2 beginI2C()

```
bool DevLab_ICM20948::beginI2C (
    uint8_t address = 0x69,
    TwoWire & i2cPort = Wire,
    uint32_t i2cSpeed = 400000)
```

beginI2C

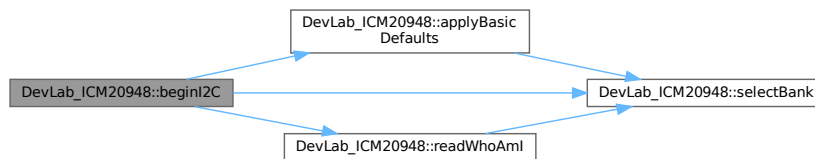
- Initialize ICM20948 using I2C interface
- Sets up communication layer (Interface + BusIO)
- Verifies device identity (WHO_AM_I)
- Configures basic power and clock settings

Returns:

- true → Device initialized successfully
- false → Communication or configuration failed

Definition at line 6 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:

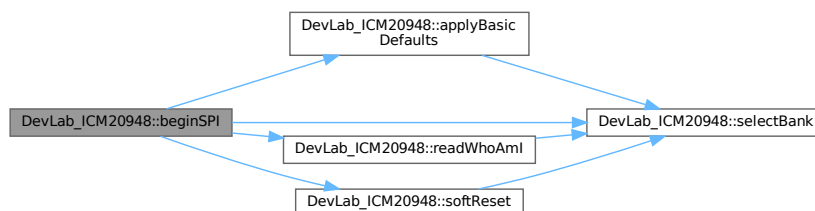


6.2.3.3 beginSPI()

```
bool DevLab_ICM20948::beginSPI (
    uint8_t csPin,
    SPIClass & spiPort = SPI,
    uint32_t spiSpeed = 1000000)
```

Definition at line 60 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.4 checkIntStatus()

```
bool DevLab_ICM20948::checkIntStatus (  
    ICM20948_IntStatus & status)
```

Definition at line 1231 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.5 getAccelAveraging()

```
bool DevLab_ICM20948::getAccelAveraging (  
    uint8_t & avg)
```

`getAccelAveraging`

- Read accelerometer averaging / decimation setting (DEC3)

Flow:

- Validate bus pointer
- Select USER BANK 2
- Read DEC3 bits [1:0]

Returns:

- true → Read successful
- false → Operation failed

Definition at line 988 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.6 getAccelDLPF()

```
bool DevLab_ICM20948::getAccelDLPF (
    uint8_t & dlpf,
    bool & bypass)
```

getAccelDLPF

- Read accelerometer DLPF configuration
- Returns filter setting and bypass state

Flow:

- Validate bus pointer
- Select USER BANK 2
- Read ACCEL_CONFIG register
- Extract bypass and DLPF bits

Returns:

- true → Read successful
- false → Operation failed

Definition at line 926 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.7 getAccelOffset()

```
bool DevLab_ICM20948::getAccelOffset (
    int16_t & offsetX,
    int16_t & offsetY,
    int16_t & offsetZ)
```

getAccelOffset

- Read accelerometer offset values for X, Y, Z axes
- Reads offset registers from USER BANK 1

Flow:

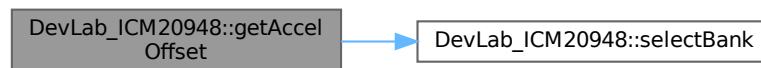
- Validate bus pointer
- Select USER BANK 1
- Read offset registers
- Convert to signed values

Returns:

- true → Read successful
- false → Operation failed

Definition at line 1151 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.8 getAccelSampleRate()

```
bool DevLab_ICM20948::getAccelSampleRate (  
    float & sampleRate)
```

getAccelSampleRate

- Get accelerometer output data rate (ODR)
- Computes value from divider registers

Returns:

- true → Read successful
- false → Operation failed

Definition at line 796 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.9 getAccelScale()

```
bool DevLab_ICM20948::getAccelScale (
    uint8_t & fullScale)
```

getAccelScale

- Get current accelerometer full-scale range (FS_SEL)

Returns:

- true → Read successful
- false → Operation failed

Definition at line 878 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.10 getClock()

```
bool DevLab_ICM20948::getClock (
    uint8_t & clock)
```

getClock

- Read current clock source (CLKSEL [2:0])

Returns:

- true → Read successful
- false → Operation failed

Definition at line 534 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.11 getDLPF()

```
bool DevLab_ICM20948::getDLPF (
    uint8_t & dlpf,
    bool & bypass)
```

getDLPF

- Read gyroscope DLPF configuration
- Returns filter setting and bypass state

Returns:

- true → Read successful
- false → Operation failed

Definition at line 619 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.12 getGyroOffset()

```
bool DevLab_ICM20948::getGyroOffset (
    int16_t & offsetX,
    int16_t & offsetY,
    int16_t & offsetZ)
```

getGyroOffset

- Read gyroscope offset values for X, Y, Z axes
- Reads offset registers from USER BANK 2

Flow:

- Validate bus pointer
- Select USER BANK 2
- Read offset registers
- Convert to signed values

Returns:

- true → Read successful
- false → Operation failed

Definition at line 1099 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.13 getGyroSampleRate()

```
bool DevLab_ICM20948::getGyroSampleRate (
    float & sampleRate)
```

getGyroSampleRate

- Get current gyroscope sample rate
- Computes value from divider register

Returns:

- true → Read successful
- false → Operation failed

Definition at line 571 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.14 getGyroScale()

```
bool DevLab_ICM20948::getGyroScale (
    uint8_t & fullScale)
```

getGyroScale

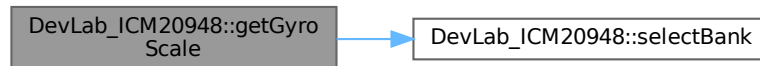
- Get current gyroscope full-scale range (FS_SEL

Returns:

- true → Read successful
- false → Operation failed

Definition at line 665 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.15 getLowPower()

```
bool DevLab_ICM20948::getLowPower (
    bool & enable)
```

getLowPower

- Read low power mode status
- Returns state of LP_EN bit

Returns:

- true → Read successful
- false → Operation failed

Definition at line 502 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.16 getSensors()

```
bool DevLab_ICM20948::getSensors (
    bool & accel_on,
    bool & gyro_on,
    bool & temp_on)
```

getSensors

- Read accel, gyro, and temperature enable state

Flow:

- Validate bus pointer
- Select USER BANK 0
- Read PWR_MGMT_2 register
- Read TEMP_DIS bit
- Decode enable states

Returns:

- true → Read successful
- false → Operation failed

Definition at line 1039 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.17 initMag()

```
bool DevLab_ICM20948::initMag ()
```

initMag

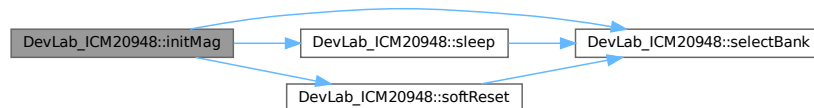
- Initialize AK09916 magnetometer using internal I2C master
- Verifies WHO_AM_I and enables continuous mode
- Configures SLV0 for automatic data read

Returns:

- true → Initialization successful
- false → Operation failed

Definition at line 334 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.18 intInit()

```
bool DevLab_ICM20948::intInit ()
```

Definition at line 1176 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.19 readAccel()

```
bool DevLab_ICM20948::readAccel (
    float & x,
    float & y,
    float & z)
```

readAccel

- Read accelerometer data (X, Y, Z)
- Convert raw values to g using LSB scale

Returns:

- true → Read successful
- false → Read failed

Definition at line 238 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.20 readGyro()

```
bool DevLab_ICM20948::readGyro (
    float & x,
    float & y,
    float & z)
```

readGyro

- Read gyroscope data (X, Y, Z)
- Convert raw values to dps using LSB scale

Returns:

- true → Read successful
- false → Read failed

Definition at line 258 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.21 readMag()

```
bool DevLab_ICM20948::readMag (
    float & x,
    float & y,
    float & z)
```

readMag

- Read magnetometer data via internal I2C master
- Data comes from EXT_SLV_SENS_DATA registers
- Convert raw values to μT

Returns:

- true → Read successful
- false → Read failed

Definition at line 310 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.22 readTemperature()

```
bool DevLab_ICM20948::readTemperature (
    float & temperature)
```

readTemperature

- Read temperature sensor
- Convert raw values to $^{\circ}\text{C}$

Returns:

- true → Read successful
- false → Read failed

Definition at line 278 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.23 readWhoAml()

```
bool DevLab_ICM20948::readWhoAmI (
    uint8_t & whoAmI)
```

readWhoAml

- Read WHO_AM_I register (device ID)
- Used during initialization to verify ICM20948
- Expected value: 0xEA

Returns:

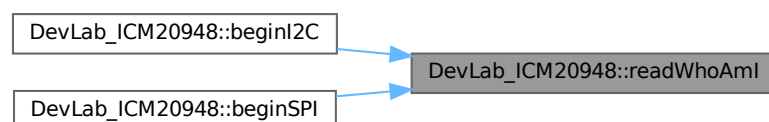
- true → Read successful
- false → Operation failed

Definition at line 118 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.2.3.24 selectBank()

```
bool DevLab_ICM20948::selectBank (
    uint8_t bank)
```

selectBank

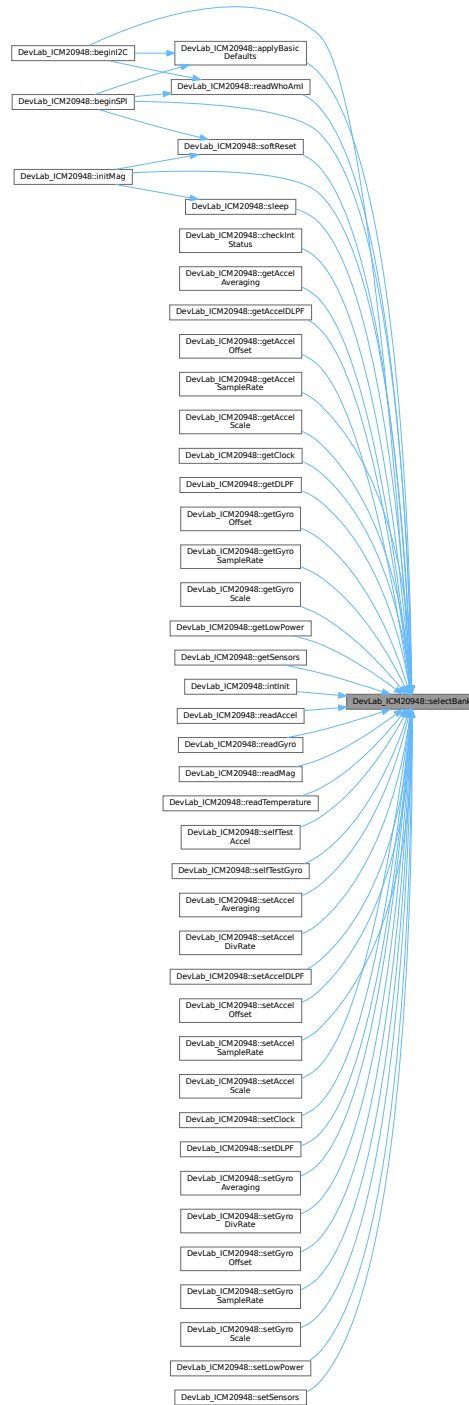
- Select USER BANK (0–3)
- Used to access different register groups inside ICM20948

Returns:

- true → Bank switch successful
- false → Write failed

Definition at line [132](#) of file [DevLab_ICM20948.cpp](#).

Here is the caller graph for this function:



6.2.3.25 selfTestAccel()

```

bool DevLab_ICM20948::selfTestAccel (
    bool x,
    bool y,
    bool z)

```

selfTestAccel

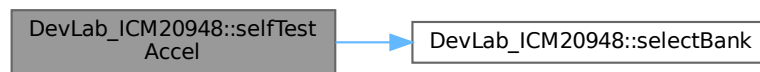
- Enable or disable accelerometer self-test per axis

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 826 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.26 selfTestGyro()

```
bool DevLab_ICM20948::selfTestGyro (  
    bool x,  
    bool y,  
    bool z)
```

selfTestGyro

- Enable or disable gyroscope self-test per axis

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 683 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.27 setAccelAveraging()

```
bool DevLab_ICM20948::setAccelAveraging (
    ICM20948_Accel_Average avg)
```

setAccelAveraging

- Configure accelerometer averaging / decimation (DEC3)
- Controls internal averaging of accel samples

Flow:

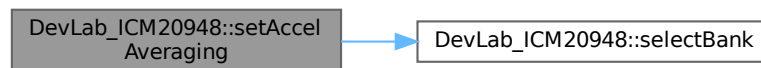
- Validate bus pointer
- Select USER BANK 2
- Write DEC3 bits [1:0]

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 952 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.28 setAccelDivRate()

```
bool DevLab_ICM20948::setAccelDivRate (
    uint16_t divisor)
```

Definition at line 748 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.29 setAccelDLPF()

```
bool DevLab_ICM20948::setAccelDLPF (
    uint8_t dlpf,
    bool bypass)
```

setAccelDLPF

- Configure accelerometer digital low-pass filter (DLPF)
- Option to bypass filter (full bandwidth)

Flow:

- Validate bus pointer
- Select USER BANK 2
- Set or clear bypass bit
- Configure DLPF bits if not bypassed

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 897 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.30 setAccelOffset()

```
bool DevLab_ICM20948::setAccelOffset (
    int16_t offsetX,
    int16_t offsetY,
    int16_t offsetZ)
```

setAccelOffset

- Set accelerometer offset values for X, Y, Z axes
- Writes offset registers in USER BANK 1

Flow:

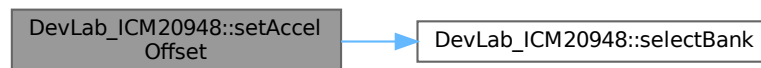
- Validate bus pointer
- Select USER BANK 1
- Pack offsets into byte array
- Write to offset registers

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 1124 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.31 setAccelSampleRate()

```
bool DevLab_ICM20948::setAccelSampleRate (  
    uint16_t sampleRate)
```

setAccelSampleRate

- Set accelerometer output data rate (ODR)
- Uses 1125 Hz base rate with 12-bit divider

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 709 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.32 setAccelScale()

```
bool DevLab_ICM20948::setAccelScale (
    ICM20948_Accel_FullScale fullScale)
```

setAccelScale

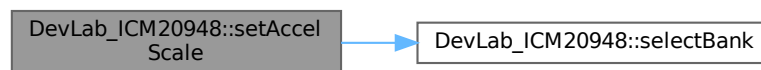
- Set accelerometer full-scale range (FS_SEL)
- Controls measurement range ($\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$)

Returns:

- true $\rightarrow$ Operation successful
- false $\rightarrow$ Operation failed

Definition at line 857 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.33 setClock()

```
bool DevLab_ICM20948::setClock (
    ICM20948_Clock_Source clock)
```

setClock

- Set clock source (CLKSEL [2:0])
- Selects internal clock for sensor operation

Returns:

- true $\rightarrow$ Operation successful
- false $\rightarrow$ Operation failed

Definition at line 520 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.34 setDLPF()

```
bool DevLab_ICM20948::setDLPF (
    ICM20948_Gyro_DLPF dlpf,
    bool bypass)
```

setDLPF

- Configure gyroscope digital low-pass filter (DLPF)
- Option to bypass filter (full bandwidth)

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 593 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.35 setGyroAveraging()

```
bool DevLab_ICM20948::setGyroAveraging (
    ICM20948_Gyro_Average avg)
```

Definition at line 970 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:

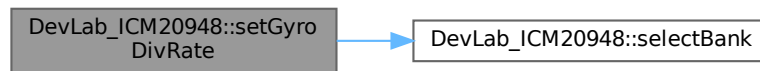


6.2.3.36 setGyroDivRate()

```
bool DevLab_ICM20948::setGyroDivRate (
    uint8_t divisor)
```

Definition at line 780 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.37 setGyroOffset()

```
bool DevLab_ICM20948::setGyroOffset (
    uint16_t offsetX,
    uint16_t offsetY,
    uint16_t offsetZ)
```

setGyroOffset

- Set gyroscope offset values for X, Y, Z axes
- Writes offset registers in USER BANK 2

Flow:

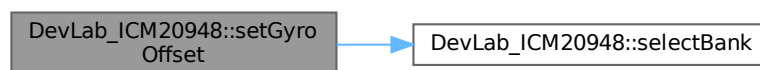
- Validate bus pointer
- Select USER BANK 2
- Pack offsets into byte array
- Write to offset registers

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 1072 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.38 setGyroSampleRate()

```
bool DevLab_ICM20948::setGyroSampleRate (
    float sampleRate)
```

setGyroSampleRate

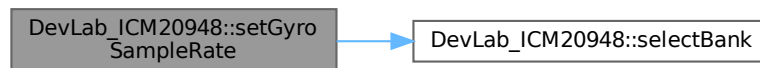
- Set gyroscope sample rate
- Uses internal divider (SRD) based on 1100 Hz base rate

Returns:

- true → Operation successful
- false → Invalid input or write failed

Definition at line 550 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.39 setGyroScale()

```
bool DevLab_ICM20948::setGyroScale (
    ICM20948_Gyro_FullScale fullScale)
```

setGyroScale

- Set gyroscope full-scale range (FS_SEL)
- Controls sensitivity (dps range)

Flow:

- Validate bus pointer
- Select USER BANK 2
- Write FS_SEL bits [2:1]

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 644 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.40 setLowPower()

```
bool DevLab_ICM20948::setLowPower (  
    bool enable)
```

setLowPower

- Enable or disable low power mode
- Controls LP_EN bit in PWR_MGMT_1

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 488 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.41 setMagOpMode()

```
bool DevLab_ICM20948::setMagOpMode (
    ICM20948_Op_Mode opMode)
```

setMagOpMode

- Set magnetometer operation mode

Returns:

- true → Mode set successfully
- false → Write failed

Definition at line 410 of file [DevLab_ICM20948.cpp](#).

6.2.3.42 setSensors()

```
bool DevLab_ICM20948::setSensors (
    bool accel_on,
    bool gyro_on,
    bool temp_on)
```

setSensors

- Enable or disable accel, gyro, and temperature sensor
- Controls power gating via PWR_MGMT_2 and TEMP_DIS

Flow:

- Validate bus pointer
- Select USER BANK 0
- Build PWR_MGMT_2 mask
- Configure temperature enable/disable

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 1006 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



6.2.3.43 sleep()

```
bool DevLab_ICM20948::sleep (  
    bool en)
```

sleep

- Enable or disable sleep mode
- Maintains clock source (CLKSEL = 1)

Flow:

- Select USER BANK 0
- Set or clear SLEEP bit

Returns:

- true → Operation successful
- false → Operation failed

Definition at line 164 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.2.3.44 softReset()

```
bool DevLab_ICM20948::softReset ()
```

softReset

- Perform software reset of ICM20948
- Resets all internal registers to default state

Flow:

- Select USER BANK 0
- Set DEVICE_RESET bit
- Wait for device to restart

Returns:

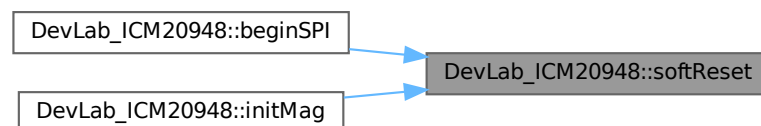
- true → Reset successful
- false → Operation failed

Definition at line 144 of file [DevLab_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



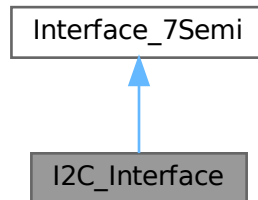
The documentation for this class was generated from the following files:

- [src/DevLab_ICM20948.h](#)
- [src/DevLab_ICM20948.cpp](#)

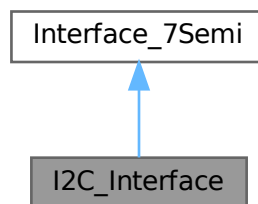
6.3 I2C_Interface Class Reference

```
#include <7Semi_I2C_Interface.h>
```

Inheritance diagram for I2C_Interface:



Collaboration diagram for I2C_Interface:



Public Member Functions

- bool [beginI2C](#) (uint8_t addr, TwoWire &wire, uint32_t speed, uint8_t sda=255, uint8_t scl=255)
- int8_t [read](#) (uint8_t reg, uint8_t *data, uint32_t len) override
- bool [beginSPI](#) (uint8_t, SPIClass &, uint32_t, uint8_t, uint8_t, uint8_t) override
- int8_t [write](#) (uint8_t reg, const uint8_t *data, uint32_t len) override

Public Member Functions inherited from [Interface_7Semi](#)

- virtual [~Interface_7Semi](#) ()

Public Attributes

- TwoWire * [i2c](#) = nullptr
- uint8_t [address](#) = 0

Additional Inherited Members

Static Protected Member Functions inherited from [Interface_7Semi](#)

- static void [delay_us](#) (uint32_t us)

6.3.1 Detailed Description

Definition at line 5 of file [7Semi_I2C_Interface.h](#).

6.3.2 Member Function Documentation

6.3.2.1 beginI2C()

```
bool I2C_Interface::beginI2C (
    uint8_t address,
    TwoWire & wire,
    uint32_t speed,
    uint8_t sda = 255,
    uint8_t scl = 255) [inline], [virtual]
```

[beginI2C\(\)](#)

- Initializes I2C peripheral
- Configures SDA / SCL pins if supported
- Sets communication clock

Returns:

- true → Initialization successful
- false → Initialization failed

Implements [Interface_7Semi](#).

Definition at line 11 of file [7Semi_I2C_Interface.h](#).

6.3.2.2 beginSPI()

```
bool I2C_Interface::beginSPI (
    uint8_t ,
    SPIClass & ,
    uint32_t ,
    uint8_t ,
    uint8_t ,
    uint8_t ) [inline], [override], [virtual]
```

Implements [Interface_7Semi](#).

Definition at line 69 of file [7Semi_I2C_Interface.h](#).

6.3.2.3 read()

```
int8_t I2C_Interface::read (
    uint8_t reg,
    uint8_t * data,
    uint32_t len) [inline], [override], [virtual]
```

read()

- Reads data from device register

Parameters:

- reg : Register address
- data : Output buffer
- len : Number of bytes

Returns:

- 0 → Success
- <0 → Error

Implements [Interface_7Semi](#).

Definition at line 32 of file [7Semi_I2C_Interface.h](#).

6.3.2.4 write()

```
int8_t I2C_Interface::write (
    uint8_t reg,
    const uint8_t * data,
    uint32_t len) [inline], [override], [virtual]
```

write()

- Writes data to device register

Returns:

- 0 → Success
- <0 → Error

Implements [Interface_7Semi](#).

Definition at line 75 of file [7Semi_I2C_Interface.h](#).

6.3.3 Member Data Documentation

6.3.3.1 address

```
uint8_t I2C_Interface::address = 0
```

Definition at line 9 of file [7Semi_I2C_Interface.h](#).

6.3.3.2 i2c

```
TwoWire* I2C_Interface::i2c = nullptr
```

Definition at line 8 of file [7Semi_I2C_Interface.h](#).

The documentation for this class was generated from the following file:

- [src/7Semi_I2C_Interface.h](#)

6.4 ICM20948_IntEnableConfig Struct Reference

```
#include <DevLab_ICM20948.h>
```

Public Attributes

- uint8_t [wofEn](#): 1
- uint8_t [womIntEn](#): 1
- uint8_t [pllRdyEn](#): 1
- uint8_t [dmpInt1En](#): 1
- uint8_t [i2cMstIntEn](#): 1
- uint8_t [rawDataRdyEn](#): 1
- uint8_t [fifoOvfEn](#) [5]
- uint8_t [fifoWmEn](#) [5]

6.4.1 Detailed Description

Definition at line 107 of file [DevLab_ICM20948.h](#).

6.4.2 Member Data Documentation

6.4.2.1 dmpInt1En

```
uint8_t ICM20948_IntEnableConfig::dmpInt1En
```

Definition at line 113 of file [DevLab_ICM20948.h](#).

6.4.2.2 fifoOvfEn

```
uint8_t ICM20948_IntEnableConfig::fifoOvfEn[5]
```

Definition at line 120 of file [DevLab_ICM20948.h](#).

6.4.2.3 fifoWmEn

```
uint8_t ICM20948_IntEnableConfig::fifoWmEn[5]
```

Definition at line 123 of file [DevLab_ICM20948.h](#).

6.4.2.4 i2cMstIntEn

```
uint8_t ICM20948_IntEnableConfig::i2cMstIntEn
```

Definition at line 114 of file [DevLab_ICM20948.h](#).

6.4.2.5 pllRdyEn

```
uint8_t ICM20948_IntEnableConfig::pllRdyEn
```

Definition at line 112 of file [DevLab_ICM20948.h](#).

6.4.2.6 rawDataRdyEn

```
uint8_t ICM20948_IntEnableConfig::rawDataRdyEn
```

Definition at line 117 of file [DevLab_ICM20948.h](#).

6.4.2.7 wofEn

```
uint8_t ICM20948_IntEnableConfig::wofEn
```

Definition at line 110 of file [DevLab_ICM20948.h](#).

6.4.2.8 womIntEn

```
uint8_t ICM20948_IntEnableConfig::womIntEn
```

Definition at line 111 of file [DevLab_ICM20948.h](#).

The documentation for this struct was generated from the following file:

- [src/DevLab_ICM20948.h](#)

6.5 ICM20948_IntPinConfig Struct Reference

```
#include <DevLab_ICM20948.h>
```

Public Attributes

- uint8_t [activeLevel](#): 1
- uint8_t [driveMode](#): 1
- uint8_t [latchMode](#): 1
- uint8_t [clearMode](#): 1
- uint8_t [fsyncActLevel](#): 1
- uint8_t [fsyncIntEn](#): 1
- uint8_t [bypassEn](#): 1

6.5.1 Detailed Description

Definition at line 96 of file [DevLab_ICM20948.h](#).

6.5.2 Member Data Documentation

6.5.2.1 activeLevel

```
uint8_t ICM20948_IntPinConfig::activeLevel
```

Definition at line 98 of file [DevLab_ICM20948.h](#).

6.5.2.2 bypassEn

```
uint8_t ICM20948_IntPinConfig::bypassEn
```

Definition at line 104 of file [DevLab_ICM20948.h](#).

6.5.2.3 clearMode

```
uint8_t ICM20948_IntPinConfig::clearMode
```

Definition at line 101 of file [DevLab_ICM20948.h](#).

6.5.2.4 driveMode

```
uint8_t ICM20948_IntPinConfig::driveMode
```

Definition at line 99 of file [DevLab_ICM20948.h](#).

6.5.2.5 fsyncActLevel

```
uint8_t ICM20948_IntPinConfig::fsyncActLevel
```

Definition at line 102 of file [DevLab_ICM20948.h](#).

6.5.2.6 fsyncIntEn

```
uint8_t ICM20948_IntPinConfig::fsyncIntEn
```

Definition at line 103 of file [DevLab_ICM20948.h](#).

6.5.2.7 latchMode

```
uint8_t ICM20948_IntPinConfig::latchMode
```

Definition at line 100 of file [DevLab_ICM20948.h](#).

The documentation for this struct was generated from the following file:

- [src/DevLab_ICM20948.h](#)

6.6 ICM20948_IntStatus Struct Reference

```
#include <DevLab_ICM20948.h>
```

Public Attributes

- [uint8_t womInt](#): 1
- [uint8_t pllRdyInt](#): 1
- [uint8_t dmpInt1](#): 1
- [uint8_t i2cMstInt](#): 1
- [uint8_t rawDataRdy](#): 1
- [uint8_t fifoOvf](#) [5]
- [uint8_t fifoWm](#) [5]

6.6.1 Detailed Description

Definition at line 126 of file [DevLab_ICM20948.h](#).

6.6.2 Member Data Documentation

6.6.2.1 dmpInt1

```
uint8_t ICM20948_IntStatus::dmpInt1
```

Definition at line 131 of file [DevLab_ICM20948.h](#).

6.6.2.2 fifoOvf

```
uint8_t ICM20948_IntStatus::fifoOvf[5]
```

Definition at line 138 of file [DevLab_ICM20948.h](#).

6.6.2.3 fifoWm

```
uint8_t ICM20948_IntStatus::fifoWm[5]
```

Definition at line 141 of file [DevLab_ICM20948.h](#).

6.6.2.4 i2cMstInt

```
uint8_t ICM20948_IntStatus::i2cMstInt
```

Definition at line 132 of file [DevLab_ICM20948.h](#).

6.6.2.5 pllRdyInt

```
uint8_t ICM20948_IntStatus::pllRdyInt
```

Definition at line 130 of file [DevLab_ICM20948.h](#).

6.6.2.6 rawDataRdy

```
uint8_t ICM20948_IntStatus::rawDataRdy
```

Definition at line 135 of file [DevLab_ICM20948.h](#).

6.6.2.7 womInt

```
uint8_t ICM20948_IntStatus::womInt
```

Definition at line 129 of file [DevLab_ICM20948.h](#).

The documentation for this struct was generated from the following file:

- [src/DevLab_ICM20948.h](#)

6.7 icm_plat_t Struct Reference

```
#include <ICM20948_platform.h>
```

Public Attributes

- `int(* read)(uint8_t reg, uint8_t *buf, size_t len, void *user)`
- `int(* write)(uint8_t reg, const uint8_t *buf, size_t len, void *user)`
- `uint8_t(* spi_reg_rw_bits)(uint8_t reg, int is_write)`
- `void(* delay_ms)(uint32_t ms)`
- `void * user`
- `void(* spi_cs)(int level, void *user)`
- `uint8_t i2c_addr`

6.7.1 Detailed Description

7Semi comment style

- Platform glue for bus IO, delays, and timing
- Implement these for your system (I2C or SPI)

Definition at line 13 of file [ICM20948_platform.h](#).

6.7.2 Member Data Documentation

6.7.2.1 delay_ms

```
void(* icm_plat_t::delay_ms) (uint32_t ms)
```

- Millisecond delay

Definition at line 32 of file [ICM20948_platform.h](#).

6.7.2.2 i2c_addr

```
uint8_t icm_plat_t::i2c_addr
```

- Device address (I2C) or ignored for SPI if you wrap inside user

Definition at line 45 of file [ICM20948_platform.h](#).

6.7.2.3 read

```
int(* icm_plat_t::read) (uint8_t reg, uint8_t *buf, size_t len, void *user)
```

- Bus read: read len bytes from reg into buf
- Return 0 on success, nonzero on error

Definition at line 18 of file [ICM20948_platform.h](#).

6.7.2.4 spi_cs

```
void(* icm_plat_t::spi_cs) (int level, void *user)
```

- If using SPI: set chip select active (1) or inactive (0)
- For I2C: leave as NULL

Definition at line 41 of file [ICM20948_platform.h](#).

6.7.2.5 spi_reg_rw_bits

```
uint8_t(* icm_plat_t::spi_reg_rw_bits) (uint8_t reg, int is_write)
```

- Optional: SPI transfers may need register R/W bit mangling
- For I2C, leave as NULL

Definition at line 28 of file [ICM20948_platform.h](#).

6.7.2.6 user

```
void* icm_plat_t::user
```

- User context pointer passed to read/write

Definition at line 36 of file [ICM20948_platform.h](#).

6.7.2.7 write

```
int(* icm_plat_t::write) (uint8_t reg, const uint8_t *buf, size_t len, void *user)
```

- Bus write: write len bytes from buf to reg
- Return 0 on success, nonzero on error

Definition at line 23 of file [ICM20948_platform.h](#).

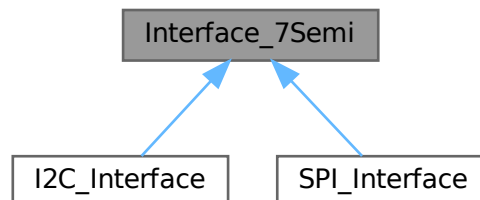
The documentation for this struct was generated from the following file:

- src/[ICM20948_platform.h](#)

6.8 Interface_7Semi Class Reference

```
#include <7Semi_Interface.h>
```

Inheritance diagram for Interface_7Semi:



Public Member Functions

- virtual [~Interface_7Semi](#) ()
- virtual bool [beginI2C](#) (uint8_t address, TwoWire &wire, uint32_t speed, uint8_t sda=255, uint8_t scl=255)=0
- virtual bool [beginSPI](#) (uint8_t cs_pin, SPIClass &wire, uint32_t speed, uint8_t csk=255, uint8_t miso=255, uint8_t mosi=255)=0
- virtual int8_t [read](#) (uint8_t reg, uint8_t *data, uint32_t len)=0
- virtual int8_t [write](#) (uint8_t reg, const uint8_t *data, uint32_t len)=0

Static Protected Member Functions

- static void [delay_us](#) (uint32_t us)

6.8.1 Detailed Description

7Semi Universal Interface Layer

- Abstract communication layer for I2C / SPI
- Ensures sensor drivers remain hardware independent

Definition at line 15 of file [7Semi_Interface.h](#).

6.8.2 Constructor & Destructor Documentation

6.8.2.1 ~Interface_7Semi()

```
virtual Interface_7Semi::~~Interface_7Semi () [inline], [virtual]
```

Definition at line 18 of file [7Semi_Interface.h](#).

6.8.3 Member Function Documentation

6.8.3.1 beginI2C()

```
virtual bool Interface_7Semi::beginI2C (  
    uint8_t address,  
    TwoWire & wire,  
    uint32_t speed,  
    uint8_t sda = 255,  
    uint8_t scl = 255) [pure virtual]
```

[beginI2C\(\)](#)

- Initializes I2C peripheral
- Configures SDA / SCL pins if supported
- Sets communication clock

Returns:

- true → Initialization successful
- false → Initialization failed

Implemented in [I2C_Interface](#), and [SPI_Interface](#).

6.8.3.2 beginSPI()

```
virtual bool Interface_7Semi::beginSPI (  
    uint8_t cs_pin,  
    SPIClass & wire,  
    uint32_t speed,  
    uint8_t csk = 255,  
    uint8_t miso = 255,  
    uint8_t mosi = 255) [pure virtual]
```

Implemented in [I2C_Interface](#), and [SPI_Interface](#).

6.8.3.3 delay_us()

```
void Interface_7Semi::delay_us (  
    uint32_t us) [inline], [static], [protected]
```

Delay wrapper

Definition at line 82 of file [7Semi_Interface.h](#).

6.8.3.4 read()

```
virtual int8_t Interface_7Semi::read (
    uint8_t reg,
    uint8_t * data,
    uint32_t len) [pure virtual]
```

read()

- Reads data from device register

Parameters:

- reg : Register address
- data : Output buffer
- len : Number of bytes

Returns:

- 0 → Success
- <0 → Error

Implemented in [I2C_Interface](#), and [SPI_Interface](#).

6.8.3.5 write()

```
virtual int8_t Interface_7Semi::write (
    uint8_t reg,
    const uint8_t * data,
    uint32_t len) [pure virtual]
```

write()

- Writes data to device register

Returns:

- 0 → Success
- <0 → Error

Implemented in [I2C_Interface](#), and [SPI_Interface](#).

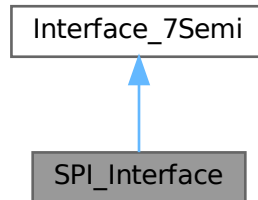
The documentation for this class was generated from the following file:

- [src/7Semi_Interface.h](#)

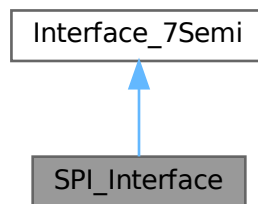
6.9 SPI_Interface Class Reference

```
#include <7Semi_SPI_Interface.h>
```

Inheritance diagram for SPI_Interface:



Collaboration diagram for SPI_Interface:



Public Member Functions

- bool [beginSPI](#) (uint8_t csPin, SPIClass &spiPort, uint32_t spiSpeed, uint8_t sck=255, uint8_t miso=255, uint8_t mosi=255) override
- bool [beginI2C](#) (uint8_t, TwoWire &, uint32_t, uint8_t, uint8_t) override
- int8_t [read](#) (uint8_t reg, uint8_t *data, uint32_t len) override
- int8_t [write](#) (uint8_t reg, const uint8_t *data, uint32_t len) override

Public Member Functions inherited from [Interface_7Semi](#)

- virtual [~Interface_7Semi](#) ()

Public Attributes

- SPIClass * [spi](#) = nullptr
- uint8_t [cs](#) = 255
- uint32_t [speed](#) = 1000000

Additional Inherited Members

Static Protected Member Functions inherited from [Interface_7Semi](#)

- static void [delay_us](#) (uint32_t us)

6.9.1 Detailed Description

7Semi SPI Interface Implementation

Definition at line 7 of file [7Semi_SPI_Interface.h](#).

6.9.2 Member Function Documentation

6.9.2.1 beginI2C()

```
bool SPI_Interface::beginI2C (
    uint8_t address,
    TwoWire & wire,
    uint32_t speed,
    uint8_t sda,
    uint8_t scl) [inline], [override], [virtual]
```

[beginI2C\(\)](#)

- Initializes I2C peripheral
- Configures SDA / SCL pins if supported
- Sets communication clock

Returns:

- true → Initialization successful
- false → Initialization failed

Implements [Interface_7Semi](#).

Definition at line 51 of file [7Semi_SPI_Interface.h](#).

6.9.2.2 beginSPI()

```
bool SPI_Interface::beginSPI (
    uint8_t csPin,
    SPIClass & spiPort,
    uint32_t spiSpeed,
    uint8_t sck = 255,
    uint8_t miso = 255,
    uint8_t mosi = 255) [inline], [override], [virtual]
```

[beginSPI\(\)](#)

- Initialize SPI interface
- Configure CS and SPI bus

Implements [Interface_7Semi](#).

Definition at line 21 of file [7Semi_SPI_Interface.h](#).

6.9.2.3 read()

```
int8_t SPI_Interface::read (
    uint8_t reg,
    uint8_t * data,
    uint32_t len) [inline], [override], [virtual]
```

[read\(\)](#)

- Read multiple bytes from register

Send register address (read)

Read data

Implements [Interface_7Semi](#).

Definition at line 62 of file [7Semi_SPI_Interface.h](#).

6.9.2.4 write()

```
int8_t SPI_Interface::write (
    uint8_t reg,
    const uint8_t * data,
    uint32_t len) [inline], [override], [virtual]
```

[write\(\)](#)

- Write multiple bytes to register

Send register address (write)

Write data

Implements [Interface_7Semi](#).

Definition at line 108 of file [7Semi_SPI_Interface.h](#).

6.9.3 Member Data Documentation

6.9.3.1 cs

```
uint8_t SPI_Interface::cs = 255
```

Definition at line 12 of file [7Semi_SPI_Interface.h](#).

6.9.3.2 speed

```
uint32_t SPI_Interface::speed = 1000000
```

Definition at line 13 of file [7Semi_SPI_Interface.h](#).

6.9.3.3 spi

```
SPIClass* SPI_Interface::spi = nullptr
```

Definition at line 11 of file [7Semi_SPI_Interface.h](#).

The documentation for this class was generated from the following file:

- [src/7Semi_SPI_Interface.h](#)

Chapter 7

File Documentation

7.1 examples/I2C_Accel/I2C_Accel.ino File Reference

7.2 I2C_Accel.ino

[Go to the documentation of this file.](#)

```
00001 /*****
00002  * @file      I2C_Accel.ino
00003  * @brief     Minimal I2C bring-up for 7Semi ICM-20948 on UNO/ESP32 +
00004  *            continuous accelerometer readout
00005  *
00006  * Features
00007  * - I2C init on default/custom pins
00008  * - IMU beginI2C() on I2C (addr 0x68/0x69)
00009  * - Manual sensor enable (setSensors)
00010  * - Accel config: scale, DLPF, averaging, sample rate
00011  * - Read accel (g) at ~2 Hz print
00012  *
00013  * Notes
00014  * - WHO_AM_I must be 0xEA
00015  * - Sensors must be explicitly enabled
00016  * - Bank switching handled internally by library
00017  *
00018  * Wiring (Arduino UNO I2C)
00019  * - SDA -> A4
00020  * - SCL -> A5
00021  * - VCC -> 3V3 (or 5V if your board supports it)
00022  * - GND -> GND
00023  *
00024  * Wiring (ESP32 default I2C)
00025  * - SDA -> GPIO21
00026  * - SCL -> GPIO22
00027  * - VCC -> 3V3
00028  * - GND -> GND
00029  *****/
00030
00031 #include <Wire.h>
00032 #include <DevLab_ICM20948.h>
00033
00034 /* ===== User Config ===== */
00035 #define SDA_PIN    6
00036 #define SCL_PIN    7
00037 #define I2C_FREQ   400000UL
00038 #define ICM_ADDR   0x69
00039
00040 DevLab_ICM20948 imu;
00041
00042 void setup() {
00043     Serial.begin(115200);
00044     delay(200);
00045     Serial.println(F("ICM-20948 (I2C) -- Accel Example"));
00046     Wire.begin(SDA_PIN, SCL_PIN);
00047     /** Initialize IMU using I2C */
00048     if (!imu.beginI2C(ICM_ADDR, Wire, 400000)) {
00049         Serial.println(F("ERROR: ICM-20948 beginI2C() failed."));
```

```

00050     while (1) delay(200);
00051 }
00052
00053 Serial.println(F("ICM-20948 ready.));
00054
00055 /** Verify device identity */
00056 uint8_t who;
00057 if (imu.readWhoAmI(who)) {
00058     Serial.print("WHO_AM_I: 0x");
00059     Serial.println(who, HEX);
00060 }
00061
00062 /** Enable only accelerometer
00063  * - accel = true
00064  * - gyro = false
00065  * - temp = false
00066  */
00067 if (!imu.setSensors(true, false, false)) {
00068     Serial.println(F("setSensors failed.));
00069 }
00070
00071 /* ----- ACCEL CONFIG -----
00072  * - Scale: ±2/±4/±8/±16 g
00073  * - DLPF: noise filtering
00074  * - Averaging: noise reduction
00075  * - Sample rate: output frequency
00076  */
00077
00078 /** Set accelerometer full-scale range */
00079 if (!imu.setAccelScale(ICM20948_Accel_FullScale::G_4)) {
00080     Serial.println(F("setAccelScale failed.));
00081 }
00082
00083 /** Enable DLPF (recommended)
00084  * - bypass = false → DLPF enabled
00085  */
00086 if (!imu.setAccelDLPF(ACCEL_DLPFCFG_3, false)) {
00087     Serial.println(F("setAccelDLPF failed.));
00088 }
00089
00090 /** Set averaging (noise reduction) */
00091 if (!imu.setAccelAveraging(ICM20948_Accel_Average::AVG_8)) {
00092     Serial.println(F("setAccelAveraging failed.));
00093 }
00094
00095 /** Set output data rate (ODR)
00096  * - Base = 1125 Hz
00097  * - ODR = 1125 / (1 + divider)
00098  * - Example: 225 Hz
00099  */
00100 if (!imu.setAccelSampleRate(225)) {
00101     Serial.println(F("setAccelSampleRate failed.));
00102 }
00103
00104 delay(10);
00105 }
00106
00107 void loop() {
00108     float ax, ay, az;
00109
00110     /** Read accelerometer data
00111     * - Output in g units
00112     * - Returns true on success
00113     */
00114     if (imu.readAccel(ax, ay, az)) {
00115         Serial.print("ACCEL [g]: ");
00116         Serial.print(ax, 3);
00117         Serial.print(", ");
00118         Serial.print(ay, 3);
00119         Serial.print(", ");
00120         Serial.println(az, 3);
00121     } else {
00122         Serial.println(F("ACCEL read failed));
00123     }
00124
00125     Serial.println(F("-----));
00126     delay(500);
00127 }

```

7.3 examples/I2C_Basic/I2C_Basic.ino File Reference

7.4 I2C_Basic.ino

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * @file      I2C_Basic.ino
00003  * @brief     Minimal I2C bring-up for 7Semi ICM-20948 +
00004  *           basic Accel/Gyro/Temp/Mag readout (updated API).
00005  *
00006  * Features
00007  * - I2C init (UNO / ESP32)
00008  * - beginI2C() initialization
00009  * - Manual sensor enable (setSensors)
00010  * - Read Accel / Gyro / Temp / Mag
00011  *
00012  * Notes
00013  * - WHO_AM_I must be 0xEA
00014  * - Sensors must be explicitly enabled
00015  * - Magnetometer requires initMag()
00016  *
00017  * Wiring (Arduino UNO I2C)
00018  * - SDA -> A4
00019  * - SCL -> A5
00020  * - VCC -> 3V3 (or 5V if supported)
00021  * - GND -> GND
00022  *
00023  * Wiring (ESP32 default I2C)
00024  * - SDA -> GPIO21
00025  * - SCL -> GPIO22
00026  *****/
00027 #include <Wire.h>
00028 #include <DevLab_ICM20948.h>
00029
00030 /* ===== User Config ===== */
00031 #define SDA_PIN    6
00032 #define SCL_PIN    7
00033 #define I2C_FREQ   400000UL
00034 #define ICM_ADDR   0x69
00035
00036 DevLab_ICM20948 imu;
00037
00038 void setup() {
00039     Serial.begin(115200);
00040     delay(200);
00041     Serial.println(F("ICM-20948 -- I2C Basic"));
00042     Wire.begin(SDA_PIN, SCL_PIN);
00043     /** Initialize IMU */
00044     if (!imu.beginI2C(ICM_ADDR, Wire, 400000)) {
00045         Serial.println(F("ERROR: beginI2C() failed"));
00046         while (1) delay(200);
00047     }
00048
00049     /** WHO_AM_I check */
00050     uint8_t who;
00051     if (!imu.readWhoAmI(who) || who != 0xEA) {
00052         Serial.println(F("ERROR: WHO_AM_I mismatch"));
00053         while (1) delay(200);
00054     }
00055
00056     Serial.print(F("WHO_AM_I = 0x"));
00057     Serial.println(who, HEX);
00058
00059     /** Enable all sensors */
00060     if (!imu.setSensors(true, true, true)) {
00061         Serial.println(F("ERROR: setSensors failed"));
00062     }
00063
00064     /** Initialize magnetometer */
00065     if (!imu.initMag()) {
00066         Serial.println(F("Mag init failed"));
00067     } else {
00068         Serial.println(F("Mag initialized"));
00069     }
00070
00071     Serial.println(F("Ready.\n"));
00072 }
00073
00074 void loop() {
00075     float ax, ay, az;
00076     float gx, gy, gz;

```

```

00077 float mx, my, mz;
00078 float tC;
00079
00080 /** Read Accelerometer */
00081 if (imu.readAccel(ax, ay, az)) {
00082     Serial.print(F("ACC [g]: "));
00083     Serial.print(ax, 3); Serial.print(", ");
00084     Serial.print(ay, 3); Serial.print(", ");
00085     Serial.println(az, 3);
00086 } else {
00087     Serial.println(F("ACC read failed"));
00088 }
00089
00090 /** Read Gyroscope */
00091 if (imu.readGyro(gx, gy, gz)) {
00092     Serial.print(F("GYR [dps]: "));
00093     Serial.print(gx, 2); Serial.print(", ");
00094     Serial.print(gy, 2); Serial.print(", ");
00095     Serial.println(gz, 2);
00096 } else {
00097     Serial.println(F("GYR read failed"));
00098 }
00099
00100 /** Read Magnetometer */
00101 if (imu.readMag(mx, my, mz)) {
00102     Serial.print(F("MAG [uT]: "));
00103     Serial.print(mx, 2); Serial.print(", ");
00104     Serial.print(my, 2); Serial.print(", ");
00105     Serial.println(mz, 2);
00106 } else {
00107     Serial.println(F("MAG read failed"));
00108 }
00109
00110 /** Read Temperature */
00111 if (imu.readTemperature(tC)) {
00112     Serial.print(F("TMP [C]: "));
00113     Serial.println(tC, 2);
00114 } else {
00115     Serial.println(F("TMP read failed"));
00116 }
00117
00118 Serial.println(F("-----"));
00119 delay(500);
00120 }
00121
00122 /* Note on magnetometer:
00123 * - Uses internal I2C master (AK09916 via ICM20948)
00124 * - initMag() must be called before readMag()
00125 * - If values are zero:
00126 *   → check initMag()
00127 *   → verify power + pull-ups
00128 */

```

7.5 examples/I2C_Magneto/I2C_Magneto.ino File Reference

7.6 I2C_Magneto.ino

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * @file      I2C_Magneto.ino
00003  * @brief     Minimal I2C bring-up for 7Semi ICM-20948 +
00004  *            AK09916 magnetometer readout (updated API).
00005  *
00006  * Features
00007  * - I2C init on default/custom pins
00008  * - IMU beginI2C() initialization
00009  * - Manual sensor enable
00010  * - Magnetometer initialization (initMag)
00011  * - Read magnetometer (uT) at ~2 Hz
00012  *
00013  * Wiring (Arduino UNO, I2C)
00014  * - SDA -> A4
00015  * - SCL -> A5
00016  * - VCC -> 3V3 (or 5V if board supports it)
00017  * - GND -> GND
00018  *
00019  * Wiring (ESP32, I2C default)
00020  * - SDA -> GPIO21

```



```

00021 * - SCL -> GPIO22
00022 * - VCC -> 3V3
00023 * - GND -> GND
00024 *
00025 * Notes
00026 * - WHO_AM_I must be 0xEA
00027 * - initMag() must be called before readMag()
00028 * - Magnetometer runs via internal I2C master
00029 *
00030 * Author : 7Semi
00031 * License : MIT
00032 *****/
00033 #include <Wire.h>
00034 #include <DevLab_ICM20948.h>
00035
00036 /* ===== User Config ===== */
00037 #define SDA_PIN 6
00038 #define SCL_PIN 7
00039 #define I2C_FREQ 400000UL
00040 #define ICM_ADDR 0x69
00041
00042 DevLab_ICM20948 imu;
00043
00044 void setup() {
00045   Serial.begin(115200);
00046   delay(200);
00047   Serial.println(F("ICM-20948 (I2C) -- Magnetometer Example"));
00048   Wire.begin(SDA_PIN, SCL_PIN);
00049   /** Initialize IMU */
00050   if (!imu.beginI2C(ICM_ADDR, Wire, 400000)) {
00051     Serial.println(F("ERROR: beginI2C() failed.));
00052     while (1) delay(200);
00053   }
00054
00055   Serial.println(F("ICM-20948 ready (I2C).));
00056
00057   /** Verify device */
00058   uint8_t who;
00059   if (!imu.readWhoAmI(who) || who != 0xEA) {
00060     Serial.println(F("ERROR: WHO_AM_I mismatch"));
00061     while (1) delay(200);
00062   }
00063
00064   Serial.print(F("WHO_AM_I = 0x"));
00065   Serial.println(who, HEX);
00066
00067   /** Enable required sensors (mag uses internal I2C master)
00068    * - accel/gyro not strictly required but safe to keep ON
00069    */
00070   if (!imu.setSensors(true, true, false)) {
00071     Serial.println(F("setSensors failed.));
00072   }
00073
00074   /** Initialize magnetometer */
00075   if (!imu.initMag()) {
00076     Serial.println(F("ERROR: initMag() failed"));
00077     while (1) delay(200);
00078   }
00079
00080   Serial.println(F("Magnetometer ready"));
00081 }
00082
00083 void loop() {
00084   float mx, my, mz;
00085
00086   /** Read magnetometer data
00087    * - Output in microtesla (uT)
00088    * - Returns true on success
00089    */
00090   if (imu.readMag(mx, my, mz)) {
00091     Serial.print(F("MAG [uT]: "));
00092     Serial.print(mx, 2); Serial.print(F(", "));
00093     Serial.print(my, 2); Serial.print(F(", "));
00094     Serial.println(mz, 2);
00095   } else {
00096     Serial.println(F("Mag read failed"));
00097   }
00098
00099   Serial.println(F("-----"));
00100   delay(500);
00101 }

```

7.7 examples/SPI_Basic/SPI_Basic.ino File Reference

7.8 SPI_Basic.ino

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * @file      SPI_Basic.ino
00003  * @brief     Minimal SPI bring-up for 7Semi ICM-20948 +
00004  *           basic Accel/Gyro/Temp readout (updated API).
00005  *
00006  * Features
00007  * - SPI init (UNO / ESP32)
00008  * - beginSPI() initialization
00009  * - Manual sensor enable
00010  * - Read Accel / Gyro / Temp
00011  *
00012  * Notes
00013  * - WHO_AM_I must be 0xEA
00014  * - Sensors must be explicitly enabled
00015  * - SPI speed ~1MHz recommended during init
00016  *
00017  * Wiring (Arduino UNO SPI)
00018  * - SCK -> D13
00019  * - MOSI -> D11
00020  * - MISO -> D12
00021  * - CS -> D10 (changeable; update CS_PIN)
00022  * - VCC -> 3V3
00023  * - GND -> GND
00024  *
00025  * Wiring (ESP32 VSPI default)
00026  * - SCK -> GPIO18
00027  * - MOSI -> GPIO23
00028  * - MISO -> GPIO19
00029  * - CS -> GPIO5
00030  *****/
00031 #include <7Semi_ICM20948.h>
00032
00033 static const uint8_t CS_PIN = 10;
00034
00035 /** IMU instance */
00036 ICM20948_7Semi imu;
00037
00038 void setup() {
00039     Serial.begin(115200);
00040     delay(200);
00041     Serial.println(F("ICM-20948 -- SPI Basic"));
00042
00043     /** Initialize IMU using SPI */
00044     if (!imu.beginSPI(CS_PIN, SPI, 1000000)) {
00045         Serial.println(F("ERROR: beginSPI() failed"));
00046         while (1) delay(200);
00047     }
00048
00049     /** Enable all sensors
00050     * - accel = true
00051     * - gyro = true
00052     * - temp = true
00053     */
00054     if (!imu.setSensors(true, true, true)) {
00055         Serial.println(F("ERROR: setSensors failed"));
00056     }
00057
00058     Serial.println(F("Ready.\n"));
00059 }
00060
00061 void loop() {
00062     float ax, ay, az;
00063     float gx, gy, gz;
00064     float tC;
00065
00066     /** Read Accelerometer */
00067     if (imu.readAccel(ax, ay, az)) {
00068         Serial.print(F("ACC [g]: "));
00069         Serial.print(ax, 3); Serial.print(", ");
00070         Serial.print(ay, 3); Serial.print(", ");
00071         Serial.print(az, 3);
00072     } else {
00073         Serial.println(F("ACC read failed"));
00074     }
00075 }
00076

```

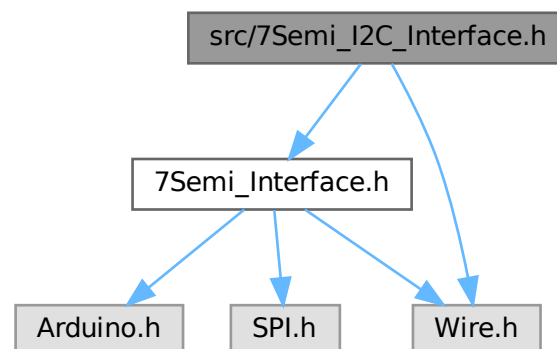
```
00077  /** Read Gyroscope */
00078  if (imu.readGyro(gx, gy, gz)) {
00079      Serial.print(F("GYR [dps]: "));
00080      Serial.print(gx, 2); Serial.print(", ");
00081      Serial.print(gy, 2); Serial.print(", ");
00082      Serial.println(gz, 2);
00083  } else {
00084      Serial.println(F("GYR read failed"));
00085  }
00086
00087  /** Read Temperature */
00088  if (imu.readTemperature(tC)) {
00089      Serial.print(F("TMP [C]: "));
00090      Serial.println(tC, 2);
00091  } else {
00092      Serial.println(F("TMP read failed"));
00093  }
00094
00095  Serial.println(F("-----"));
00096  delay(500);
00097 }
```

7.9 src/7Semi_I2C_Interface.h File Reference

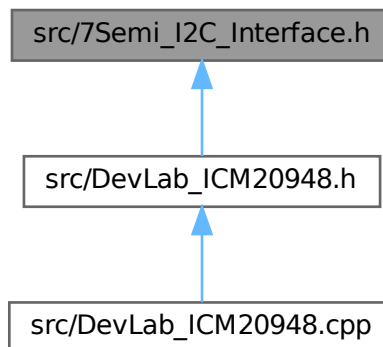
```
#include "7Semi_Interface.h"
```

```
#include <Wire.h>
```

Include dependency graph for 7Semi_I2C_Interface.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [I2C_Interface](#)

7.10 7Semi_I2C_Interface.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "7Semi_Interface.h"
00003 #include <Wire.h>
00004
00005 class I2C_Interface : public Interface_7Semi
00006 {
00007 public:
00008     TwoWire *i2c = nullptr;
00009     uint8_t address = 0;
00010
00011     bool beginI2C(uint8_t addr, TwoWire &wire, uint32_t speed,
00012                 uint8_t sda = 255, uint8_t scl = 255)
00013     {
00014         address = addr;
00015         i2c = &wire;
00016
00017         #if defined(ESP32)
00018             if (sda != 255 && scl != 255)
00019                 i2c->begin(sda, scl);
00020             else
00021                 i2c->begin();
00022         #else
00023             i2c->begin();
00024         #endif
00025         // i2c->setClock(speed);
00026
00027         // Probe device
00028         i2c->beginTransmission(address);
00029         return (i2c->endTransmission() == 0);
00030     }
00031
00032     int8_t read(uint8_t reg, uint8_t *data, uint32_t len) override
00033     {
00034         if (!i2c)
00035             return -1;
00036         i2c->beginTransmission(address);
00037         i2c->write(reg);
00038
00039         if (i2c->endTransmission(false) != 0)
  
```

```

00040         return -1;
00041
00042     delay(1);
00043
00044     if (len > 255)
00045         return -3;
00046     uint8_t received = i2c->requestFrom(address, (uint8_t)len);
00047     if (received != len)
00048         return -2;
00049
00050     for (uint32_t i = 0; i < len; i++)
00051     {
00052         if (!i2c->available())
00053             return -4;
00054         data[i] = i2c->read();
00055     }
00056     // Serial.print("R-Reg: ");
00057     // Serial.print(reg, HEX);
00058     // Serial.print(" |Data: ");
00059     // for (int i = 0; i < len; i++)
00060     // {
00061     //     Serial.print(" ");
00062     //     Serial.print(data[i], HEX);
00063     // }
00064     // Serial.println();
00065
00066     return 0;
00067 }
00068
00069 bool beginSPI(uint8_t, SPIClass &, uint32_t,
00070              uint8_t, uint8_t, uint8_t) override
00071 {
00072     return false;
00073 }
00074
00075 int8_t write(uint8_t reg, const uint8_t *data, uint32_t len) override
00076 {
00077     if (!i2c)
00078         return -1;
00079
00080     i2c->beginTransaction(address);
00081     i2c->write(reg);
00082     for (uint32_t i = 0; i < len; i++)
00083         i2c->write(data[i]);
00084
00085     uint8_t status = i2c->endTransmission();
00086
00087     // Serial.print("W-Reg: ");
00088     // Serial.print(reg, HEX);
00089     // Serial.print(" |Data: ");
00090     // for (int i = 0; i < len; i++)
00091     // {
00092     //     Serial.print(" ");
00093     //     Serial.print(data[i], HEX);
00094     // }
00095     // Serial.println();
00096     return (status == 0) ? 0 : -status;
00097 }
00098 };

```

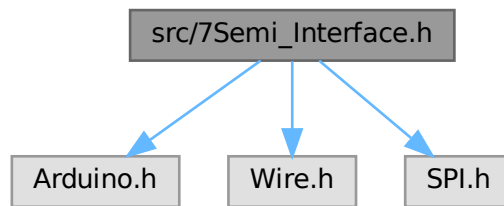
7.11 src/7Semi_Interface.h File Reference

```

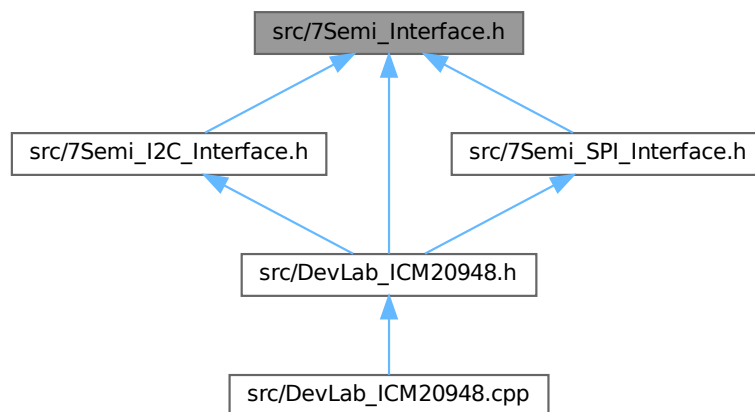
#include <Arduino.h>
#include <Wire.h>
#include <SPI.h>

```

Include dependency graph for 7Semi_Interface.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Interface_7Semi](#)

7.12 7Semi_Interface.h

[Go to the documentation of this file.](#)

```

00001 // #pragma once
00002 #ifndef INTERFACE_7SEMI_H
00003 #define INTERFACE_7SEMI_H
00004
00005 #include <Arduino.h>
00006 #include <Wire.h>
00007 #include <SPI.h>
00008
00009 /**
00010  * 7Semi Universal Interface Layer
00011  *

```

```

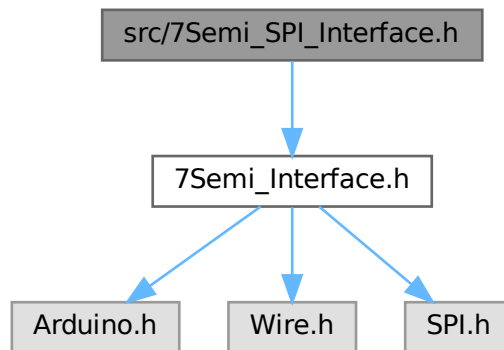
00012  * - Abstract communication layer for I2C / SPI
00013  * - Ensures sensor drivers remain hardware independent
00014  */
00015  class Interface_7Semi {
00016  public:
00017
00018      virtual ~Interface_7Semi() {}
00019
00020      /**
00021       * beginI2C()
00022       *
00023       * - Initializes I2C peripheral
00024       * - Configures SDA / SCL pins if supported
00025       * - Sets communication clock
00026       *
00027       * Returns:
00028       * - true → Initialization successful
00029       * - false → Initialization failed
00030       */
00031      virtual bool beginI2C(
00032          uint8_t address,
00033          TwoWire &wire,
00034          uint32_t speed,
00035          uint8_t sda = 255,
00036          uint8_t scl = 255) = 0;
00037
00038      virtual bool beginSPI(
00039          uint8_t cs_pin,
00040          SPIClass &wire,
00041          uint32_t speed,
00042          uint8_t csk = 255,
00043          uint8_t miso = 255,
00044          uint8_t mosi = 255) = 0;
00045
00046      /**
00047       * read()
00048       *
00049       * - Reads data from device register
00050       *
00051       * Parameters:
00052       * - reg : Register address
00053       * - data : Output buffer
00054       * - len : Number of bytes
00055       *
00056       * Returns:
00057       * - 0 → Success
00058       * - <0 → Error
00059       */
00060
00061      virtual int8_t read(
00062          uint8_t reg,
00063          uint8_t *data,
00064          uint32_t len) = 0;
00065
00066      /**
00067       * write()
00068       *
00069       * - Writes data to device register
00070       *
00071       * Returns:
00072       * - 0 → Success
00073       * - <0 → Error
00074       */
00075      virtual int8_t write(
00076          uint8_t reg,
00077          const uint8_t *data,
00078          uint32_t len) = 0;
00079
00080  protected:
00081      /** Delay wrapper */
00082      static void delay_us(uint32_t us)
00083      {
00084          delayMicroseconds(us);
00085      }
00086  };
00087  #endif

```

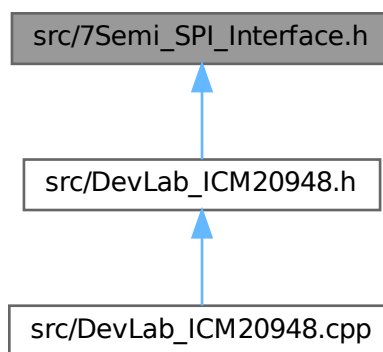
7.13 src/7Semi_SPI_Interface.h File Reference

```
#include "7Semi_Interface.h"
```

Include dependency graph for 7Semi_SPI_Interface.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [SPI_Interface](#)

7.14 7Semi_SPI_Interface.h

[Go to the documentation of this file.](#)


```

00001 #pragma once
00002 #include "7Semi_Interface.h"
00003
00004 /**
00005  * 7Semi SPI Interface Implementation
00006  */
00007 class SPI_Interface : public Interface_7Semi
00008 {
00009
00010 public:
00011     SPIClass *spi = nullptr;
00012     uint8_t cs = 255;
00013     uint32_t speed = 1000000;
00014
00015     /**
00016      * beginSPI()
00017      *
00018      * - Initialize SPI interface
00019      * - Configure CS and SPI bus
00020      */
00021     bool beginSPI(uint8_t csPin,
00022                   SPIClass &spiPort,
00023                   uint32_t spiSpeed,
00024                   uint8_t sck = 255,
00025                   uint8_t miso = 255,
00026                   uint8_t mosi = 255) override
00027     {
00028         if (csPin == 255)
00029             return false;
00030
00031         spi = &spiPort;
00032         cs = csPin;
00033         speed = spiSpeed;
00034
00035         pinMode(cs, OUTPUT);
00036         digitalWrite(cs, HIGH);
00037
00038         #if defined(ESP32)
00039             if (sck != 255 && miso != 255 && mosi != 255)
00040                 spi->begin(sck, miso, mosi, cs);
00041             else
00042                 spi->begin();
00043         #else
00044             spi->begin();
00045         #endif
00046
00047         delay(1);
00048         return true;
00049     }
00050
00051     bool beginI2C(uint8_t, TwoWire &, uint32_t,
00052                  uint8_t, uint8_t) override
00053     {
00054         return false;
00055     }
00056
00057     /**
00058      * read()
00059      *
00060      * - Read multiple bytes from register
00061      */
00062     int8_t read(uint8_t reg, uint8_t *data, uint32_t len) override
00063     {
00064         if (!spi || !data || len == 0)
00065             return -1;
00066
00067         SPISettings settings(speed, MSBFIRST, SPI_MODE0);
00068
00069         spi->beginTransaction(settings);
00070         digitalWrite(cs, LOW);
00071         delayMicroseconds(10);
00072
00073         /**
00074          * Send register address (read)
00075          */
00076         spi->transfer(reg | 0x80);
00077
00078         delayMicroseconds(10);
00079
00080         /**
00081          * Read data
00082          */
00083         for (uint32_t i = 0; i < len; i++)
00084             data[i] = spi->transfer(0x00);
00085
00086         digitalWrite(cs, HIGH);
00087         delayMicroseconds(10);

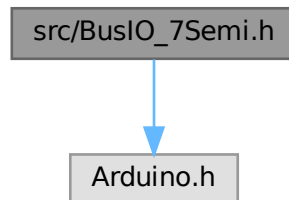
```

```
00088         spi->endTransaction();
00089
00090         // Serial.print("R-Reg: ");
00091         // Serial.print(reg, HEX);
00092         // Serial.print(" |Data: ");
00093         // for (int i = 0; i < len; i++)
00094         // {
00095         //     Serial.print(" ");
00096         //     Serial.print(data[i], HEX);
00097         // }
00098         // Serial.println();
00099
00100         return 0;
00101     }
00102
00103     /**
00104     * write()
00105     *
00106     * - Write multiple bytes to register
00107     */
00108     int8_t write(uint8_t reg, const uint8_t *data, uint32_t len) override
00109     {
00110         if (!spi || !data || len == 0)
00111             return -1;
00112
00113         SPISettings settings(speed, MSBFIRST, SPI_MODE0);
00114
00115         spi->beginTransaction(settings);
00116         digitalWrite(cs, LOW);
00117         delayMicroseconds(10);
00118
00119         /**
00120         * Send register address (write)
00121         */
00122         spi->transfer(reg & 0x7F);
00123
00124         /**
00125         * Write data
00126         */
00127         for (uint32_t i = 0; i < len; i++)
00128             spi->transfer(data[i]);
00129
00130         digitalWrite(cs, HIGH);
00131         delayMicroseconds(10);
00132         spi->endTransaction();
00133
00134         // Serial.print("W-Reg: ");
00135         // Serial.print(reg, HEX);
00136         // Serial.print(" |Data: ");
00137         // for (int i = 0; i < len; i++)
00138         // {
00139         //     Serial.print(" ");
00140         //     Serial.print(data[i], HEX);
00141         // }
00142         // Serial.println();
00143
00144         return 0;
00145     }
00146 };
```

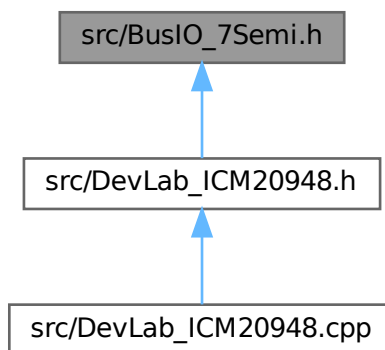
7.15 src/BusIO_7Semi.h File Reference

```
#include <Arduino.h>
```

Include dependency graph for BusIO_7Semi.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [BusIO_7Semi< Bus >](#)

7.16 BusIO_7Semi.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include <Arduino.h>
00003
00004 template <typename Bus>
00005 class BusIO_7Semi
00006 {
```

```

00007
00008 public:
00009     /**
00010      * Constructor
00011      *
00012      * - Stores interface reference
00013      */
00014     BusIO_7Semi(Bus &busRef) : bus(busRef) {}
00015
00016     /** read 8-bit */
00017     inline bool read(uint8_t reg, uint8_t &value)
00018     {
00019         return (bus.read(reg, &value, 1) == 0);
00020     }
00021
00022     /** read 16-bit */
00023     inline bool read(uint8_t reg, uint16_t &value)
00024     {
00025         uint8_t data[2];
00026         if (bus.read(reg, data, 2) != 0)
00027             return false;
00028         value = (data[0] << 8) | data[1];
00029         return true;
00030     }
00031
00032     /** read burst */
00033     inline bool read(uint8_t reg, uint8_t *data, uint32_t len)
00034     {
00035         return (bus.read(reg, data, len) == 0);
00036     }
00037
00038     /** write 8-bit */
00039     inline bool write(uint8_t reg, uint8_t value)
00040     {
00041         return (bus.write(reg, &value, 1) == 0);
00042     }
00043
00044     /** write 16-bit */
00045     inline bool write(uint8_t reg, uint16_t value)
00046     {
00047         uint8_t data[2] = {(uint8_t)(value >> 8), (uint8_t)(value & 0xFF)};
00048         return (bus.write(reg, data, 2) == 0);
00049     }
00050
00051     /** write burst */
00052     inline bool write(uint8_t reg, const uint8_t *data, uint32_t len)
00053     {
00054         return (bus.write(reg, data, len) == 0);
00055     }
00056
00057     /**
00058      * readBits (8-bit register)
00059      */
00060     bool readBits(uint8_t reg, uint8_t pos, uint8_t len, uint8_t &value)
00061     {
00062         if (len == 0 || len > 8 || (pos + len) > 8)
00063             return false;
00064
00065         uint8_t reg_val;
00066         if (!read(reg, reg_val))
00067             return false;
00068
00069         uint8_t mask = ((uint8_t)1 << len) - 1;
00070
00071         value = (reg_val >> pos) & mask;
00072         return true;
00073     }
00074
00075     /**
00076      * readBits (16-bit register)
00077      */
00078     bool readBits(uint8_t reg, uint8_t pos, uint8_t len, uint16_t &value)
00079     {
00080         if (len == 0 || len > 16 || (pos + len) > 16)
00081             return false;
00082
00083         uint8_t reg_val;
00084         if (!read(reg, reg_val))
00085             return false;
00086
00087         uint16_t mask = ((uint16_t)1 << len) - 1;
00088
00089         value = (reg_val >> pos) & mask;
00090         return true;
00091     }
00092
00093     /**

```

```

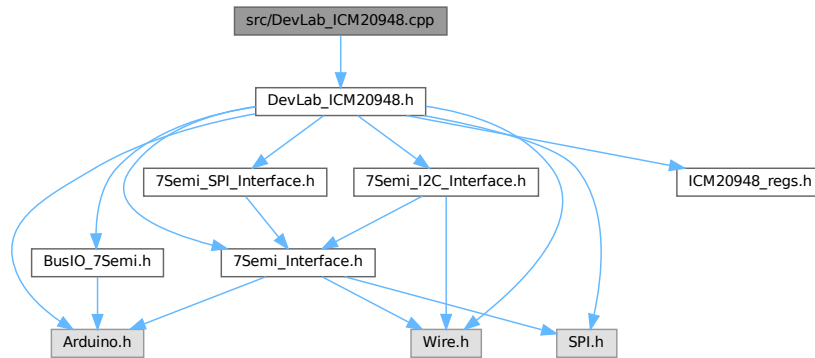
00094     * writeBits (8-bit register)
00095     *
00096     * - Modifies specific bits in 8-bit register
00097     */
00098     bool writeBits(uint8_t reg, uint8_t pos, uint8_t len, uint8_t value)
00099     {
00100         if (len == 0 || len > 8 || (pos + len) > 8)
00101             return false;
00102
00103         uint8_t reg_val;
00104         if (!read(reg, reg_val))
00105             return false;
00106
00107         uint8_t mask = ((uint8_t)1 << len) - 1;
00108
00109         reg_val &= ~(mask << pos);
00110         reg_val |= (value & mask) << pos;
00111
00112         return write(reg, reg_val);
00113     }
00114
00115     /**
00116     * writeBits (16-bit register)
00117     *
00118     * - Modifies specific bits in 16-bit register
00119     */
00120     bool writeBits(uint8_t reg, uint8_t pos, uint8_t len, uint16_t value)
00121     {
00122         if (len == 0 || len > 16 || (pos + len) > 16)
00123             return false;
00124
00125         uint8_t reg_val;
00126         if (!read(reg, reg_val))
00127             return false;
00128
00129         uint16_t mask = ((uint16_t)1 << len) - 1;
00130
00131         reg_val &= ~(mask << pos);
00132         reg_val |= (value & mask) << pos;
00133
00134         return write(reg, reg_val);
00135     }
00136
00137     bool readBit(uint8_t reg, uint8_t pos, uint8_t &value)
00138     {
00139         return readBits(reg, pos, 1, value);
00140     }
00141
00142     bool readBit(uint8_t reg, uint8_t pos, uint16_t &value)
00143     {
00144         return readBits(reg, pos, 1, value);
00145     }
00146
00147     bool writeBit(uint8_t reg, uint8_t pos, uint8_t value)
00148     {
00149         return writeBits(reg, pos, 1, (uint8_t)value );
00150     }
00151     bool writeBit(uint8_t reg, uint8_t pos, uint16_t value)
00152     {
00153         return writeBits(reg, pos, 1, value);
00154     }
00155
00156 private:
00157     Bus &bus;
00158 };

```

7.17 src/DevLab_ICM20948.cpp File Reference

```
#include "DevLab_ICM20948.h"
```

Include dependency graph for DevLab_ICM20948.cpp:



7.18 DevLab_ICM20948.cpp

[Go to the documentation of this file.](#)

```

00001 #include "DevLab_ICM20948.h"
00002
00003 /** - Construct with default I2C address; call begin() to attach bus */
00004 DevLab_ICM20948::DevLab_ICM20948() {}
00005
00006 bool DevLab_ICM20948::beginI2C(uint8_t address, TwoWire &i2cPort, uint32_t i2cSpeed)
00007 {
00008     // Free previous BusIO instance
00009     if (bus)
00010     {
00011         delete bus;
00012         bus = nullptr;
00013     }
00014
00015     // Assign I2C interface implementation
00016     iface = &i2c;
00017
00018     // Initialize I2C (400kHz) and probe device
00019     if (!i2c.beginI2C(address, i2cPort, i2cSpeed))
00020         return false;
00021
00022     // Create BusIO abstraction layer
00023     bus = new BusIO_7Semi<Interface_7Semi>(*iface);
00024     if (!bus)
00025         return false;
00026
00027     // Allow device to stabilize after power-up
00028     delay(100);
00029
00030     // Read WHO_AM_I register
00031     uint8_t who_am_i;
00032     if (!readWhoAmI(who_am_i))
00033         return false;
00034
00035     // Validate device ID (expected 0xEA)
00036     if (who_am_i != WHO_AM_I_VAL)
00037         return false;
00038
00039     if (!selectBank(2))
00040         return false;
00041
00042     if (!bus->write(ODR_ALIGN_EN, (uint8_t)0x01))
00043         return false;
00044
00045     // Enable I2C interface (I2C_IF_DIS = 0)

```

```

00046     if (!bus->writeBit(USER_CTRL, 4, (uint8_t)0x00))
00047         return false;
00048
00049     // Set clock source to PLL (auto select best clock)
00050     if (!bus->write(PWR_MGMT_1, (uint8_t)0x01))
00051         return false;
00052
00053     if(!applyBasicDefaults())
00054         return false;
00055
00056     // Initialization successful
00057     return true;
00058 }
00059
00060 bool DevLab_ICM20948::beginSPI(uint8_t csPin, SPIClass &spiPort, uint32_t spiSpeed)
00061 {
00062     // Free previous BusIO instance
00063     if (bus)
00064     {
00065         delete bus;
00066         bus = nullptr;
00067     }
00068
00069     // Assign SPI interface implementation
00070     iface = &spi;
00071
00072     // Initialize SPI (1MHz) and probe device
00073     if (!spi.beginSPI(csPin, spiPort, spiSpeed))
00074         return false;
00075
00076     // Create BusIO abstraction layer
00077     bus = new BusIO_7Semi<Interface_7Semi>(*iface);
00078     if (!bus)
00079         return false;
00080
00081     softReset();
00082
00083     // Allow device to stabilize after power-up
00084     delay(100);
00085
00086     // Read WHO_AM_I register
00087     uint8_t who_am_i;
00088
00089     if (!readWhoAmI(who_am_i))
00090         return false;
00091
00092     // Validate device ID (expected 0xEA)
00093     if (who_am_i == WHO_AM_I_VAL)
00094         return false;
00095
00096     if (!selectBank(2))
00097         return false;
00098
00099     if (!bus->write(ODR_ALIGN_EN, (uint8_t)0x01))
00100         return false;
00101
00102     // Enable I2C interface (I2C_IF_DIS = 0)
00103     if (!bus->writeBit(USER_CTRL, 4, (uint8_t)0x00))
00104         return false;
00105
00106     // Wake device from sleep mode (SLEEP = 0)
00107     if (!bus->write(PWR_MGMT_1, (uint8_t)0x01))
00108         return false;
00109
00110     if(!applyBasicDefaults())
00111         return false;
00112
00113     // Initialization successful
00114     return true;
00115 }
00116
00117 bool DevLab_ICM20948::readWhoAmI(uint8_t &whoAmI)
00118 {
00119     // Select USER BANK 0
00120     if (!selectBank(0))
00121         return false;
00122
00123     // Read WHO_AM_I register
00124     if (!bus->read(WHO_AM_I, whoAmI))
00125         return false;
00126
00127     // Success
00128     return true;
00129 }
00130
00131 bool DevLab_ICM20948::selectBank(uint8_t bank)

```

```

00133 {
00134     if (bank > 3)
00135         return false;
00136
00137     // Mask bank value to valid range (0-3) and shift to bits [5:4]
00138     uint8_t v = (bank & 0x03) << 4;
00139
00140     // Write bank selection to REG_BANK_SEL register
00141     return bus->write(REG_BANK_SEL, v);
00142 }
00143
00144 bool DevLab_ICM20948::softReset()
00145 {
00146     // Select USER BANK 0
00147     if (!selectBank(0))
00148         return false;
00149
00150     // Set DEVICE_RESET bit (bit 7)
00151     if (!bus->write(PWR_MGMT_1, (uint8_t)0x80))
00152         return false;
00153
00154     // Allow device to reset
00155     delay(100);
00156
00157     selectBank(2);
00158     bus->write(ODR_ALIGN_EN, (uint8_t)0x01);
00159
00160     // Success
00161     return true;
00162 }
00163
00164 bool DevLab_ICM20948::sleep(bool en)
00165 {
00166     // Select USER BANK 0
00167     if (!selectBank(0))
00168         return false;
00169     uint8_t v = 1;
00170     if(en)
00171         v |= 1<<6;
00172
00173     // Set or clear SLEEP bit (bit 6)
00174     if (!bus->write(PWR_MGMT_1, v))
00175         return false;
00176
00177     // Success
00178     return true;
00179 }
00180
00181 bool DevLab_ICM20948::applyBasicDefaults()
00182 {
00183     // Select USER BANK 0
00184     if (!selectBank(0))
00185         return false;
00186
00187     // Wake device and set clock source (CLKSEL=1, SLEEP=0)
00188     if (!bus->write(PWR_MGMT_1, (uint8_t)0x01))
00189         return false;
00190
00191     if (!bus->write(PWR_MGMT_2, (uint8_t)0x00))
00192         return false;
00193
00194     // Allow device to stabilize
00195     delay(10);
00196
00197     // Disable duty-cycling (LP mode off)
00198     if (!bus->write(LP_CONFIG, (uint8_t)0x00))
00199         return false;
00200
00201     // Select USER BANK 2
00202     if (!selectBank(2))
00203         return false;
00204
00205     // Configure gyro (DLPF enabled, FS = 2000 dps)
00206     if (!bus->write(GYRO_CONFIG_1, (uint8_t)0x1F))
00207         return false;
00208
00209     // Set gyro sample rate divider (SRD = 0 → max rate)
00210     if (!bus->write(GYRO_SMPLRT_DIV, (uint8_t)0x00))
00211         return false;
00212
00213     // Configure accel (DLPF enabled, FS = 16g)
00214     if (!bus->write(ACCEL_CONFIG, (uint8_t)0x1F))
00215         return false;
00216
00217     // Set accel sample rate divider high byte
00218     if (!bus->write(ACCEL_SMPLRT_DIV_1, (uint8_t)0x00))
00219

```



```

00220     return false;
00221
00222 // Set accel sample rate divider low byte
00223 if (!bus->write(ACCEL_SMPLRT_DIV_2, (uint8_t)0x00))
00224     return false;
00225
00226 // Select USER BANK 0
00227 if (!selectBank(0))
00228     return false;
00229
00230 // Configure interrupt pin (open-drain, active-low, latch)
00231 if (!bus->write(INT_PIN_CFG, (uint8_t)0x30))
00232     return false;
00233
00234 // Configuration successful
00235 return true;
00236 }
00237
00238 bool DevLab_ICM20948::readAccel(float &x, float &y, float &z)
00239 {
00240     // Select USER BANK 0
00241     if (!selectBank(0))
00242         return false;
00243
00244     // Read 6 bytes (X, Y, Z)
00245     uint8_t raw[6];
00246     if (!bus->read(ACCEL_XOUT_H, raw, 6))
00247         return false;
00248
00249     // Convert raw data to g
00250     x = (int16_t)((raw[0] << 8) | raw[1]) / mg_per_lsb;
00251     y = (int16_t)((raw[2] << 8) | raw[3]) / mg_per_lsb;
00252     z = (int16_t)((raw[4] << 8) | raw[5]) / mg_per_lsb;
00253
00254     // Success
00255     return true;
00256 }
00257
00258 bool DevLab_ICM20948::readGyro(float &x, float &y, float &z)
00259 {
00260     // Select USER BANK 0
00261     if (!selectBank(0))
00262         return false;
00263
00264     // Read 6 bytes (X, Y, Z)
00265     uint8_t raw[6];
00266     if (!bus->read(GYRO_XOUT_H, raw, 6))
00267         return false;
00268
00269     // Convert raw data to dps
00270     x = (int16_t)((raw[0] << 8) | raw[1]) / degree_per_second;
00271     y = (int16_t)((raw[2] << 8) | raw[3]) / degree_per_second;
00272     z = (int16_t)((raw[4] << 8) | raw[5]) / degree_per_second;
00273
00274     // Success
00275     return true;
00276 }
00277
00278 bool DevLab_ICM20948::readTemperature(float &temperature)
00279 {
00280     // Select USER BANK 0
00281     if (!selectBank(0))
00282         return false;
00283
00284     // Initialize accumulator
00285     temperature = 0;
00286
00287     // Read and average 5 samples
00288     for (int i = 0; i < 5; i++)
00289     {
00290         // Read temperature registers
00291         uint8_t raw[2];
00292         if (!bus->read(TEMP_OUT_H, raw, 2))
00293             return false;
00294
00295         // Convert raw to signed value
00296         int16_t temp = (int16_t)((raw[0] << 8) | raw[1]);
00297
00298         // Convert to °C and accumulate
00299         temperature += temp / 333.87f + 21.0f;
00300     }
00301
00302     // Compute average temperature
00303     temperature /= 5.0f;
00304
00305     // Success
00306     return true;

```

```

00307 }
00308
00309
00310 bool DevLab_ICM20948::readMag(float &x, float &y, float &z)
00311 {
00312     // Select USER BANK 0
00313     if (!selectBank(0))
00314         return false;
00315
00316     // Read 8 bytes (ST1 + XYZ + ST2)
00317     uint8_t buf[8];
00318     if (!bus->read(EXT_SLV_SENS_DATA_00, buf, 8))
00319         return false;
00320
00321     int16_t mx = (int16_t)(buf[1] | (buf[2] << 8));
00322     int16_t my = (int16_t)(buf[3] | (buf[4] << 8));
00323     int16_t mz = (int16_t)(buf[5] | (buf[6] << 8));
00324
00325     // Convert to  $\mu$ T (AK09916 sensitivity)
00326     x = mx * 0.15f;
00327     y = my * 0.15f;
00328     z = mz * 0.15f;
00329
00330     // Success
00331     return true;
00332 }
00333
00334 bool DevLab_ICM20948::initMag()
00335 {
00336     if (!bus)
00337         return false;
00338
00339     softReset();
00340
00341     sleep(false);
00342
00343     if (!selectBank(0))
00344         return false;
00345     // Enable I2C master mode and disable I2C slave mode
00346     if (!bus->write(USER_CTRL, (uint8_t)USER_CTRL_I2C_MST_EN))
00347         return false;
00348
00349     if (!selectBank(3))
00350         return false;
00351
00352     if (!bus->write(I2C_MST_CTRL, (uint8_t)0x07))
00353         return false;
00354
00355     delay(10);
00356
00357     //Soft Reset
00358     writeSlave4(AK_CNTL3, 0x01);
00359
00360     delay(100);
00361
00362     uint8_t i, who;
00363     for (i = 0; i < 10; i++)
00364     {
00365         readSlave4(AK_WIA2, who);
00366         if (who == AK_WIA2_VAL)
00367         {
00368             //Mode 3 (continuous measurement 100Hz)
00369             writeSlave4(AK_CNTL2, 0x08);
00370
00371             delay(10);
00372
00373             // ---- Setup auto-read (SLV0) ----
00374             if (!selectBank(3))
00375                 return false;
00376
00377             if (!bus->write(I2C_SLV0_ADDR, (uint8_t)(AK09916_I2C_ADDR | 0x80)))
00378                 return false;
00379
00380             if (!bus->write(I2C_SLV0_REG, (uint8_t)AK_ST1))
00381                 return false;
00382
00383             if (!bus->write(I2C_SLV0_CTRL, (uint8_t)(I2C_SLVx_EN | 8)))
00384                 return false;
00385
00386             return true;
00387         }
00388
00389         writeSlave4(AK_CNTL2, 0x08);
00390
00391         delay(10);
00392
00393         // ---- Setup auto-read (SLV0) ----

```

```

00394     if (!selectBank(3))
00395         return false;
00396
00397     if (!bus->write(I2C_SLV0_ADDR, (uint8_t)(AK09916_I2C_ADDR | 0x80)))
00398         return false;
00399
00400     if (!bus->write(I2C_SLV0_REG, (uint8_t)AK_ST1))
00401         return false;
00402
00403     if (!bus->write(I2C_SLV0_CTRL, (uint8_t)(I2C_SLVx_EN | 8)))
00404         return false;
00405
00406     return false;
00407 }
00408 }
00409
00410 bool DevLab_ICM20948::setMagOpMode(ICM20948_Op_Mode opMode)
00411 {
00412     // Write operation mode to magnetometer
00413     writeSlave4(AK_CNTL2, (uint8_t)opMode);
00414
00415     return true;
00416 }
00417
00418 bool DevLab_ICM20948::writeSlave4(uint8_t reg, uint8_t value)
00419 {
00420     // Select USER BANK 3
00421     if (!selectBank(3))
00422         return false;
00423
00424     // 7 - 1:r , 0:W
00425     //[6:0]: I2C slave address (AK09916_I2C_ADDR)
00426     // Set slave address (write)
00427     if (!bus->write(I2C_SLV4_ADDR, (uint8_t)AK09916_I2C_ADDR))
00428         return false;
00429
00430     // Set data to write to slave
00431     if (!bus->write(I2C_SLV4_DO, value))
00432         return false;
00433
00434     // Set target register
00435     if (!bus->write(I2C_SLV4_REG, reg))
00436         return false;
00437
00438     // Start transaction (EN bit)
00439     if (!bus->write(I2C_SLV4_CTRL, (uint8_t)128))
00440         return false;
00441     // Wait for completion (with timeout)
00442     uint32_t start = millis();
00443     uint8_t status;
00444
00445     while (millis() - start < 10)
00446     {
00447         if (bus->read(I2C_SLV4_CTRL, status))
00448             if (!(status & 0x80))
00449                 return true;
00450     }
00451
00452     // Timeout
00453     return false;
00454 }
00455
00456 bool DevLab_ICM20948::readSlave4(uint8_t reg, uint8_t &value)
00457 {
00458     // Select USER BANK 3
00459     if (!selectBank(3))
00460         return false;
00461
00462     // Set slave address (read)
00463     if (!bus->write(I2C_SLV4_ADDR, (uint8_t)(AK09916_I2C_ADDR | 0x80)))
00464         return false;
00465
00466     // Set target register
00467     if (!bus->write(I2C_SLV4_REG, reg))
00468         return false;
00469
00470     // Start transaction
00471     if (!bus->write(I2C_SLV4_CTRL, (uint8_t)128))
00472         return false;
00473     // Wait for completion
00474     uint32_t start = millis();
00475     uint8_t status;
00476     while (millis() - start < 10)
00477     {
00478         if (bus->read(I2C_SLV4_CTRL, status))
00479             if (!(status & 0x80))
00480                 { if (!bus->read(I2C_SLV4_DI, value))

```

```

00481     return false;
00482     return true;
00483 }
00484 }
00485     return false;
00486 }
00487
00488 bool DevLab_ICM20948::setLowPower(bool enable)
00489 {
00490     // Select USER BANK 0
00491     if (!selectBank(0))
00492         return false;
00493
00494     // Set or clear LP_EN bit (bit 5)
00495     if (!bus->writeBit(PWR_MGMT_1, 5, (uint8_t)enable))
00496         return false;
00497
00498     // Success
00499     return true;
00500 }
00501
00502 bool DevLab_ICM20948::getLowPower(bool &enable)
00503 {
00504     // Select USER BANK 0
00505     if (!selectBank(0))
00506         return false;
00507
00508     // Read LP_EN bit (bit 5)
00509     uint8_t v = 0;
00510     if (!bus->readBit(PWR_MGMT_1, 5, v))
00511         return false;
00512
00513     // Convert bit value to boolean
00514     enable = (v == 1);
00515
00516     // Success
00517     return true;
00518 }
00519
00520 bool DevLab_ICM20948::setClock(ICM20948_Clock_Source clock)
00521 {
00522     // Select USER BANK 0
00523     if (!selectBank(0))
00524         return false;
00525
00526     // Set CLKSEL bits [2:0]
00527     if (!bus->write(PWR_MGMT_1, (uint8_t)clock))
00528         return false;
00529
00530     // Success
00531     return true;
00532 }
00533
00534 bool DevLab_ICM20948::getClock(uint8_t &clock)
00535 {
00536     // Select USER BANK 0
00537     if (!selectBank(0))
00538         return false;
00539
00540     // Read CLKSEL bits [2:0]
00541     if (!bus->read(PWR_MGMT_1, clock))
00542         return false;
00543
00544     clock &= 0x07;
00545
00546     // Success
00547     return true;
00548 }
00549
00550 bool DevLab_ICM20948::setGyroSampleRate(float sampleRate)
00551 {
00552     // Validate sample rate range (approx 4.3 Hz to 1100 Hz)
00553     if ((sampleRate < 4.3f) || (sampleRate > 1100.0f))
00554         return false;
00555
00556     // Compute divider value
00557     uint8_t v = (uint8_t)((1100.0f / sampleRate) - 1.0f);
00558
00559     // Select USER BANK 2
00560     if (!selectBank(2))
00561         return false;
00562
00563     // Write sample rate divider
00564     if (!bus->write(GYRO_SMPLRT_DIV, v))
00565         return false;
00566
00567     // Success

```

```

00568     return true;
00569 }
00570
00571 bool DevLab_ICM20948::getGyroSampleRate(float &sampleRate)
00572 {
00573     // Validate bus pointer
00574     if (!bus)
00575         return false;
00576
00577     // Select USER BANK 2
00578     if (!selectBank(2))
00579         return false;
00580
00581     // Read sample rate divider
00582     uint8_t v = 0;
00583     if (!bus->read(GYRO_SMPLRT_DIV, v))
00584         return false;
00585
00586     // Compute sample rate
00587     sampleRate = 1100.0f / (1.0f + v);
00588
00589     // Success
00590     return true;
00591 }
00592
00593 bool DevLab_ICM20948::setDLPF(ICM20948_Gyro_DLPF dlpf, bool bypass)
00594 {
00595     // Validate bus pointer
00596     if (!bus)
00597         return false;
00598
00599     // Select USER BANK 2
00600     if (!selectBank(2))
00601         return false;
00602
00603     // Enable or disable DLPF bypass (bit 0)
00604     if (!bus->writeBit(GYRO_CONFIG_1, 0, (uint8_t)bypass))
00605         return false;
00606
00607     // If bypass enabled, skip DLPF configuration
00608     if (bypass)
00609         return true;
00610
00611     // Set DLPF configuration bits [5:3]
00612     if (!bus->writeBits(GYRO_CONFIG_1, 3, 3, (uint8_t)dlpf))
00613         return false;
00614
00615     // Success
00616     return true;
00617 }
00618
00619 bool DevLab_ICM20948::getDLPF(uint8_t &dlpf, bool &bypass)
00620 {
00621     // Validate bus pointer
00622     if (!bus)
00623         return false;
00624
00625     // Select USER BANK 2
00626     if (!selectBank(2))
00627         return false;
00628
00629     // Read GYRO_CONFIG_1 register
00630     uint8_t v = 0;
00631     if (!bus->read(GYRO_CONFIG_1, v))
00632         return false;
00633
00634     // Extract bypass bit (bit 0)
00635     bypass = (v & 0x01);
00636
00637     // Extract DLPF bits [5:3]
00638     dlpf = (v >> 3) & 0x07;
00639
00640     // Success
00641     return true;
00642 }
00643
00644 bool DevLab_ICM20948::setGyroScale(ICM20948_Gyro_FullScale fullScale)
00645 {
00646     // Validate bus pointer
00647     if (!bus)
00648         return false;
00649
00650     // Select USER BANK 2
00651     if (!selectBank(2))
00652         return false;
00653
00654     // Set full-scale range bits [2:1]

```

```

00655     if (!bus->writeBits(GYRO_CONFIG_1, 1, 2, (uint8_t)fullScale))
00656         return false;
00657
00658     // Update internal scale (LSB per g)
00659     degree_per_second = 32768.0f / (250.0f * (1 << ((uint8_t)fullScale)));
00660
00661     // Success
00662     return true;
00663 }
00664
00665 bool DevLab_ICM20948::getGyroScale(uint8_t &fullScale)
00666 {
00667     // Validate bus pointer
00668     if (!bus)
00669         return false;
00670
00671     // Select USER BANK 2
00672     if (!selectBank(2))
00673         return false;
00674
00675     // Read full-scale range bits [2:1]
00676     if (!bus->readBits(GYRO_CONFIG_1, 1, 2, fullScale))
00677         return false;
00678
00679     // Success
00680     return true;
00681 }
00682
00683 bool DevLab_ICM20948::selfTestGyro(bool x, bool y, bool z)
00684 {
00685     // Validate bus pointer
00686     if (!bus)
00687         return false;
00688
00689     // Select USER BANK 2
00690     if (!selectBank(2))
00691         return false;
00692
00693     // Set Z-axis self-test (bit 3)
00694     if (!bus->writeBit(GYRO_CONFIG_2, 3, (uint8_t)z))
00695         return false;
00696
00697     // Set Y-axis self-test (bit 4)
00698     if (!bus->writeBit(GYRO_CONFIG_2, 4, (uint8_t)y))
00699         return false;
00700
00701     // Set X-axis self-test (bit 5)
00702     if (!bus->writeBit(GYRO_CONFIG_2, 5, (uint8_t)x))
00703         return false;
00704
00705     // Success
00706     return true;
00707 }
00708
00709 bool DevLab_ICM20948::setAccelSampleRate(uint16_t sampleRate)
00710 {
00711     // Validate bus pointer
00712     if (!bus)
00713         return false;
00714
00715     // Select USER BANK 2
00716     if (!selectBank(2))
00717         return false;
00718
00719     // Validate input
00720     if (sampleRate == 0)
00721         return false;
00722
00723     // Clamp maximum rate
00724     if (sampleRate >= 1125)
00725         return false;
00726
00727     // Compute divider (rounded)
00728     uint16_t div = (1125.0f / sampleRate) - 1;
00729
00730     if (div > 4095u)
00731         div = 4095u;
00732
00733     // Split divider into high and low bytes
00734     uint8_t msb = (uint8_t)((div >> 8) & 0x0F);
00735     uint8_t lsb = (uint8_t)(div & 0xFF);
00736
00737     // Write divider high byte
00738     if (!bus->write(ACCEL_SMPLRT_DIV_1, msb))
00739         return false;
00740
00741     // Write divider low byte

```

```

00742     if (!bus->write(ACCEL_SMPLRT_DIV_2, lsb))
00743         return false;
00744
00745     // Success
00746     return true;
00747 }
00748 bool DevLab_ICM20948::setAccelDivRate(uint16_t divisor)
00749 {
00750     if (!bus)
00751         return false;
00752
00753     // Select USER BANK 2
00754     if (!selectBank(2))
00755         return false;
00756
00757     // Validate input
00758     if (divisor == 0)
00759         return false;
00760
00761     if (divisor > 4095u)
00762         divisor = 4095u;
00763
00764     // Split divider into high and low bytes
00765     uint8_t msb = (uint8_t)((divisor >> 8) & 0x0F);
00766     uint8_t lsb = (uint8_t)(divisor & 0xFF);
00767
00768     // Write divider high byte
00769     if (!bus->write(ACCEL_SMPLRT_DIV_1, msb))
00770         return false;
00771
00772     // Write divider low byte
00773     if (!bus->write(ACCEL_SMPLRT_DIV_2, lsb))
00774         return false;
00775
00776     // Success
00777     return true;
00778 }
00779
00780 bool DevLab_ICM20948::setGyroDivRate(uint8_t divisor)
00781 {
00782     if (!bus)
00783         return false;
00784
00785     // Select USER BANK 2
00786     if (!selectBank(2))
00787         return false;
00788
00789     // Write sample rate divider
00790     if (!bus->write(GYRO_SMPLRT_DIV, divisor))
00791         return false;
00792
00793     // Success
00794     return true;
00795 }
00796 bool DevLab_ICM20948::getAccelSampleRate(float &sampleRate)
00797 {
00798     // Validate bus pointer
00799     if (!bus)
00800         return false;
00801
00802     // Select USER BANK 2
00803     if (!selectBank(2))
00804         return false;
00805
00806     // Read divider high byte
00807     uint8_t msb = 0;
00808     if (!bus->read(ACCEL_SMPLRT_DIV_1, msb))
00809         return false;
00810
00811     // Read divider low byte
00812     uint8_t lsb = 0;
00813     if (!bus->read(ACCEL_SMPLRT_DIV_2, lsb))
00814         return false;
00815
00816     // Combine 12-bit divider
00817     uint16_t div = ((msb & 0x0F) << 8) | lsb;
00818
00819     // Compute sample rate
00820     sampleRate = 1125.0f / (1.0f + div);
00821
00822     // Success
00823     return true;
00824 }
00825
00826 bool DevLab_ICM20948::selfTestAccel(bool x, bool y, bool z)
00827 {
00828     // Validate bus pointer

```

```

00829     if (!bus)
00830         return false;
00831
00832     // Select USER BANK 2
00833     if (!selectBank(2))
00834         return false;
00835
00836     if (x)
00837     { // Set X-axis self-test (bit 7)
00838         if (!bus->writeBit(ACCEL_CONFIG_2, 7, (uint8_t)x))
00839             return false;
00840     }
00841
00842     if (y)
00843     { // Set Y-axis self-test (bit 6)
00844         if (!bus->writeBit(ACCEL_CONFIG_2, 6, (uint8_t)y))
00845             return false;
00846     }
00847
00848     if (z)
00849     { // Set Z-axis self-test (bit 5)
00850         if (!bus->writeBit(ACCEL_CONFIG_2, 5, (uint8_t)z))
00851             return false;
00852     }
00853     // Success
00854     return true;
00855 }
00856
00857 bool DevLab_ICM20948::setAccelScale(ICM20948_Accel_FullScale fullScale)
00858 {
00859     // Validate bus pointer
00860     if (!bus)
00861         return false;
00862
00863     // Select USER BANK 2
00864     if (!selectBank(2))
00865         return false;
00866
00867     // Set full-scale bits [2:1]
00868     if (!bus->writeBits(ACCEL_CONFIG, 1, 2, (uint8_t)fullScale))
00869         return false;
00870
00871     // Update internal scale (LSB per g)
00872     mg_per_lsb = 16384.0f / (1 << (uint8_t)fullScale);
00873
00874     // Success
00875     return true;
00876 }
00877
00878 bool DevLab_ICM20948::getAccelScale(uint8_t &fullScale)
00879 {
00880     // Validate bus pointer
00881     if (!bus)
00882         return false;
00883
00884     // Select USER BANK 2
00885     if (!selectBank(2))
00886         return false;
00887
00888     // Read full-scale bits [2:1]
00889     if (!bus->readBits(ACCEL_CONFIG, 1, 2, fullScale))
00890         return false;
00891
00892     // Success
00893     return true;
00894 }
00895
00896
00897 bool DevLab_ICM20948::setAccelDLPF(uint8_t dlpf, bool bypass)
00898 {
00899     // Validate bus pointer
00900     if (!bus)
00901         return false;
00902
00903     // Select USER BANK 2
00904     if (!selectBank(2))
00905         return false;
00906
00907     // Set or clear DLPF bypass (bit 0)
00908     if (!bus->writeBit(ACCEL_CONFIG, 0, (uint8_t)bypass))
00909         return false;
00910
00911     // If bypass enabled, skip DLPF configuration
00912     if (bypass)
00913         return true;
00914
00915     // Limit DLPF value to valid range

```



```

00916     dlpf &= 0x07;
00917
00918     // Set DLPF configuration bits [5:3]
00919     if (!bus->writeBits(ACCEL_CONFIG, 3, 3, dlpf))
00920         return false;
00921
00922     // Success
00923     return true;
00924 }
00925
00926 bool DevLab_ICM20948::getAccelDLPF(uint8_t &dlpf, bool &bypass)
00927 {
00928     // Validate bus pointer
00929     if (!bus)
00930         return false;
00931
00932     // Select USER BANK 2
00933     if (!selectBank(2))
00934         return false;
00935
00936     // Read ACCEL_CONFIG register
00937     uint8_t v = 0;
00938     if (!bus->read(ACCEL_CONFIG, v))
00939         return false;
00940
00941     // Extract bypass bit (bit 0)
00942     bypass = (v & 0x01);
00943
00944     // Extract DLPF bits [5:3]
00945     dlpf = (v >> 3) & 0x07;
00946
00947     // Success
00948     return true;
00949 }
00950
00951
00952 bool DevLab_ICM20948::setAccelAveraging(ICM20948_Accel_Average avg)
00953 {
00954     // Validate bus pointer
00955     if (!bus)
00956         return false;
00957
00958     // Select USER BANK 2
00959     if (!selectBank(2))
00960         return false;
00961
00962     // Set DEC3 averaging bits [1:0]
00963     if (!bus->writeBits(ACCEL_CONFIG_2, 0, 2, (uint8_t)avg))
00964         return false;
00965
00966     // Success
00967     return true;
00968 }
00969
00970 bool DevLab_ICM20948::setGyroAveraging(ICM20948_Gyro_Average avg)
00971 {
00972     // Validate bus pointer
00973     if (!bus)
00974         return false;
00975
00976     // Select USER BANK 2
00977     if (!selectBank(2))
00978         return false;
00979
00980     // Set DEC3 averaging bits [1:0]
00981     if (!bus->writeBits(GYRO_CONFIG_2, 0, 2, (uint8_t)avg))
00982         return false;
00983
00984     // Success
00985     return true;
00986 }
00987
00988 bool DevLab_ICM20948::getAccelAveraging(uint8_t &avg)
00989 {
00990     // Validate bus pointer
00991     if (!bus)
00992         return false;
00993
00994     // Select USER BANK 2
00995     if (!selectBank(2))
00996         return false;
00997
00998     // Read DEC3 averaging bits [1:0]
00999     if (!bus->readBits(ACCEL_CONFIG_2, 0, 2, avg))
01000         return false;
01001
01002     // Success

```

```

01003     return true;
01004 }
01005
01006 bool DevLab_ICM20948::setSensors(bool accel_on, bool gyro_on, bool temp_on)
01007 {
01008     // Validate bus pointer
01009     if (!bus)
01010         return false;
01011
01012     // Select USER BANK 0
01013     if (!selectBank(0))
01014         return false;
01015
01016     // Build PWR_MGMT_2 mask
01017     uint8_t v = 0;
01018
01019     // Disable accel axes if not enabled
01020     if (!accel_on)
01021         v |= 0x38; // bits [5:3]
01022
01023     // Disable gyro axes if not enabled
01024     if (!gyro_on)
01025         v |= 0x07; // bits [2:0]
01026
01027     // Write sensor enable/disable mask
01028     if (!bus->write(PWR_MGMT_2, v))
01029         return false;
01030
01031     // Set or clear TEMP_DIS bit (bit 3)
01032     if (!bus->writeBit(PWR_MGMT_1, 3, (uint8_t)!temp_on))
01033         return false;
01034
01035     // Success
01036     return true;
01037 }
01038
01039 bool DevLab_ICM20948::getSensors(bool &accel_on, bool &gyro_on, bool &temp_on)
01040 {
01041     // Validate bus pointer
01042     if (!bus)
01043         return false;
01044
01045     // Select USER BANK 0
01046     if (!selectBank(0))
01047         return false;
01048
01049     // Read PWR_MGMT_2 register
01050     uint8_t v = 0;
01051     if (!bus->read(PWR_MGMT_2, v))
01052         return false;
01053
01054     // Decode gyro state (bits [2:0])
01055     gyro_on = ((v & 0x07) == 0);
01056
01057     // Decode accel state (bits [5:3])
01058     accel_on = ((v & 0x38) == 0);
01059
01060     // Read TEMP_DIS bit (bit 3)
01061     uint8_t t = 0;
01062     if (!bus->readBit(PWR_MGMT_1, 3, t))
01063         return false;
01064
01065     // Decode temperature state
01066     temp_on = (t == 0);
01067
01068     // Success
01069     return true;
01070 }
01071
01072 bool DevLab_ICM20948::setGyroOffset(uint16_t offsetX, uint16_t offsetY, uint16_t offsetZ)
01073 {
01074     // Validate bus pointer
01075     if (!bus)
01076         return false;
01077
01078     // Select USER BANK 2
01079     if (!selectBank(2))
01080         return false;
01081
01082     // Pack offset values into byte array
01083     uint8_t data[6] = {
01084         (uint8_t)(offsetX >> 8),
01085         (uint8_t)(offsetX & 0xFF),
01086         (uint8_t)(offsetY >> 8),
01087         (uint8_t)(offsetY & 0xFF),
01088         (uint8_t)(offsetZ >> 8),
01089         (uint8_t)(offsetZ & 0xFF)};

```

```

01090
01091 // Write offset registers
01092 if (!bus->write(XG_OFFS_USRH, data, 6))
01093     return false;
01094
01095 // Success
01096 return true;
01097 }
01098
01099 bool DevLab_ICM20948::getGyroOffset(int16_t &offsetX, int16_t &offsetY, int16_t &offsetZ)
01100 {
01101     // Validate bus pointer
01102     if (!bus)
01103         return false;
01104
01105     // Select USER BANK 2
01106     if (!selectBank(2))
01107         return false;
01108
01109     // Read offset registers
01110     uint8_t data[6];
01111     if (!bus->read(XG_OFFS_USRH, data, 6))
01112         return false;
01113
01114     // Convert to signed values
01115     offsetX = (int16_t)((data[0] << 8) | data[1]);
01116     offsetY = (int16_t)((data[2] << 8) | data[3]);
01117     offsetZ = (int16_t)((data[4] << 8) | data[5]);
01118
01119     // Success
01120     return true;
01121 }
01122
01123
01124 bool DevLab_ICM20948::setAccelOffset(int16_t offsetX, int16_t offsetY, int16_t offsetZ)
01125 {
01126     // Validate bus pointer
01127     if (!bus)
01128         return false;
01129
01130     // Select USER BANK 1 (accel offsets are here)
01131     if (!selectBank(1))
01132         return false;
01133
01134     // Pack offset values into byte array
01135     uint8_t data[6] = {
01136         (uint8_t)(offsetX >> 8),
01137         (uint8_t)(offsetX & 0xFF),
01138         (uint8_t)(offsetY >> 8),
01139         (uint8_t)(offsetY & 0xFF),
01140         (uint8_t)(offsetZ >> 8),
01141         (uint8_t)(offsetZ & 0xFF)};
01142
01143     // Write offset registers
01144     if (!bus->write(XA_OFFS_H, data, 6))
01145         return false;
01146
01147     // Success
01148     return true;
01149 }
01150
01151 bool DevLab_ICM20948::getAccelOffset(int16_t &offsetX, int16_t &offsetY, int16_t &offsetZ)
01152 {
01153     // Validate bus pointer
01154     if (!bus)
01155         return false;
01156
01157     // Select USER BANK 1
01158     if (!selectBank(1))
01159         return false;
01160
01161     // Read offset registers
01162     uint8_t data[6];
01163     if (!bus->read(XA_OFFS_H, data, 6))
01164         return false;
01165
01166     // Convert to signed values
01167     offsetX = (int16_t)((data[0] << 8) | data[1]);
01168     offsetY = (int16_t)((data[2] << 8) | data[3]);
01169     offsetZ = (int16_t)((data[4] << 8) | data[5]);
01170
01171     // Success
01172     return true;
01173 }
01174
01175
01176 bool DevLab_ICM20948::intInit(const ICM20948_IntPinConfig &cfg) {

```

```

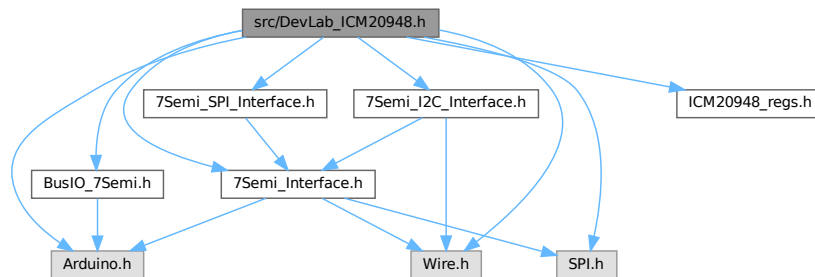
01177 // Validate bus pointer
01178 if (!bus)
01179     return false;
01180
01181 if (!selectBank(0))
01182     return false;
01183
01184 uint8_t reg = (cfg.activeLevel << 7) |
01185               (cfg.driveMode << 6) |
01186               (cfg.latchMode << 5) |
01187               (cfg.clearMode << 4) |
01188               (cfg.fsyncActLevel << 3) |
01189               (cfg.fsyncIntEn << 2) |
01190               (cfg.bypassEn << 1);
01191 // Initialize INT1
01192 // Configure interrupt pin (open-drain, active-low, latch)
01193 if (!bus->write(INT_PIN_CFG, reg))
01194     return false;
01195
01196 // Configuration successful
01197 return true;
01198 }
01199
01200 bool DevLab_ICM20948::intEnableConfig(const ICM20948_IntEnableConfig &cfg)
01201 {
01202     if (!bus) return false;
01203     if (!selectBank(0)) return false;
01204
01205     // INT_ENABLE
01206     uint8_t reg0 = (cfg.wofEn << 7) |
01207                   (cfg.womIntEn << 3) |
01208                   (cfg.pllRdyEn << 2) |
01209                   (cfg.dmpInt1En << 1) |
01210                   (cfg.i2cMstIntEn << 0);
01211     if (!bus->write(INT_ENABLE, reg0)) return false;
01212
01213     // INT_ENABLE_1
01214     if (!bus->write(INT_ENABLE_1, (uint8_t)(cfg.rawDataRdyEn & 0x01))) return false;
01215
01216     // INT_ENABLE_2 -- construye máscara desde el array
01217     uint8_t ovfMask = 0;
01218     for (uint8_t i = 0; i < 5; i++)
01219         ovfMask |= (cfg.fifoOvfEn[i] ? BIT(i) : 0);
01220     if (!bus->write(INT_ENABLE_2, ovfMask)) return false;
01221
01222     // INT_ENABLE_3 -- igual
01223     uint8_t wmMask = 0;
01224     for (uint8_t i = 0; i < 5; i++)
01225         wmMask |= (cfg.fifoWmEn[i] ? BIT(i) : 0);
01226     if (!bus->write(INT_ENABLE_3, wmMask)) return false;
01227
01228     return true;
01229 }
01230
01231 bool DevLab_ICM20948::checkIntStatus(ICM20948_IntStatus &status)
01232 {
01233     if (!bus) return false;
01234     if (!selectBank(0)) return false;
01235
01236     // Leer los 4 registros de status en una sola operación de burst
01237     uint8_t raw[4];
01238     if (!bus->read(INT_STATUS, raw, 4)) return false;
01239
01240     // INT_STATUS (0x19)
01241     status.womInt = (raw[0] & STS_WOM_INT) ? 1 : 0;
01242     status.pllRdyInt = (raw[0] & STS_PLL_RDY_INT) ? 1 : 0;
01243     status.dmpInt1 = (raw[0] & STS_DMP_INT1) ? 1 : 0;
01244     status.i2cMstInt = (raw[0] & STS_I2C_MST_INT) ? 1 : 0;
01245
01246     // INT_STATUS_1 (0x1A)
01247     status.rawDataRdy = (raw[1] & STS_RAW_DATA_0_RDY_INT) ? 1 : 0;
01248
01249     // INT_STATUS_2 (0x1B) -- FIFO overflow canal por canal
01250     for (uint8_t i = 0; i < 5; i++)
01251         status.fifoOvf[i] = (raw[2] & BIT(i)) ? 1 : 0;
01252
01253     // INT_STATUS_3 (0x1C) -- FIFO watermark canal por canal
01254     for (uint8_t i = 0; i < 5; i++)
01255         status.fifoWm[i] = (raw[3] & BIT(i)) ? 1 : 0;
01256
01257     return true;
01258 }

```

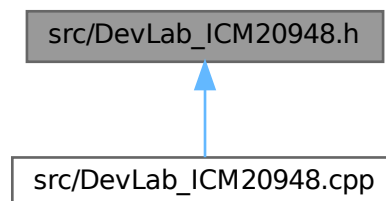
7.19 src/DevLab_ICM20948.h File Reference

```
#include <Arduino.h>
#include <Wire.h>
#include <SPI.h>
#include "ICM20948_regs.h"
#include "7Semi_Interface.h"
#include "7Semi_I2C_Interface.h"
#include "7Semi_SPI_Interface.h"
#include "BusIO_7Semi.h"
```

Include dependency graph for DevLab_ICM20948.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [ICM20948_IntPinConfig](#)
- struct [ICM20948_IntEnableConfig](#)
- struct [ICM20948_IntStatus](#)
- class [DevLab_ICM20948](#)

Enumerations

- enum class [ICM20948_Op_Mode](#) : uint8_t {
[MODE_POWER_DOWN](#) = 0x00 , [MODE_SINGLE_MEASUREMENT](#) = 0x01 , [MODE_CONTINUOUS_1](#) = 0x02 , [MODE_CONTINUOUS_2](#) = 0x04 ,
[MODE_CONTINUOUS_3](#) = 0x06 , [MODE_CONTINUOUS_4](#) = 0x08 , [MODE_SELF_TEST](#) = 0x10 }
- enum class [ICM20948_Clock_Source](#) : uint8_t {
[INTERNAL_20MHZ_0](#) = 0x00 , [AUTO_PLL_1](#) = 0x01 , [AUTO_PLL_2](#) = 0x02 , [AUTO_PLL_3](#) = 0x03 ,
[AUTO_PLL_4](#) = 0x04 , [AUTO_PLL_5](#) = 0x05 , [INTERNAL_20MHZ_6](#) = 0x06 , [STOP_CLOCK](#) = 0x07 }
- enum class [ICM20948_Gyro_DLPF](#) : uint8_t {
[DLPF_196HZ](#) = 0x00 , [DLPF_151HZ](#) = 0x01 , [DLPF_119HZ](#) = 0x02 , [DLPF_51HZ](#) = 0x03 ,
[DLPF_23HZ](#) = 0x04 , [DLPF_11HZ](#) = 0x05 , [DLPF_5HZ](#) = 0x06 , [DLPF_361HZ](#) = 0x07 }
- enum class [ICM20948_Gyro_Average](#) : uint8_t {
[AVG_1](#) = 0x00 , [AVG_2](#) = 0x01 , [AVG_4](#) = 0x02 , [AVG_8](#) = 0x03 ,
[AVG_16](#) = 0x04 , [AVG_32](#) = 0x05 , [AVG_64](#) = 0x06 , [AVG_128](#) = 0x07 }
- enum class [ICM20948_Gyro_FullScale](#) : uint8_t { [DPS_250](#) = 0x00 , [DPS_500](#) = 0x01 , [DPS_1000](#) = 0x02 ,
[DPS_2000](#) = 0x03 }
- enum class [ICM20948_Accel_FullScale](#) : uint8_t { [G_2](#) = 0x00 , [G_4](#) = 0x01 , [G_8](#) = 0x02 , [G_16](#) = 0x03 }
- enum class [ICM20948_Accel_Average](#) : uint8_t { [AVG_1_OR_4](#) = 0x00 , [AVG_8](#) = 0x01 , [AVG_16](#) = 0x02 ,
[AVG_32](#) = 0x03 }
- enum class [ICM20948_Accel_DLPFCFG](#) : uint8_t {
[DLPF0_246HZ](#) = 0x00 , [DLPF_246HZ](#) = 0x01 , [DLPF_111HZ](#) = 0x02 , [DLPF_50HZ](#) = 0x03 ,
[DLPF_24HZ](#) = 0x04 , [DLPF_12HZ](#) = 0x05 , [DLPF_6HZ](#) = 0x06 , [DLPF_473HZ](#) = 0x07 }

7.19.1 Enumeration Type Documentation

7.19.1.1 ICM20948_Accel_Average

```
enum class ICM20948\_Accel\_Average : uint8_t [strong]
```

Enumerator

AVG_1_OR_4	
AVG_8	
AVG_16	
AVG_32	

Definition at line 76 of file [DevLab_ICM20948.h](#).

7.19.1.2 ICM20948_Accel_DLPFCFG

```
enum class ICM20948\_Accel\_DLPFCFG : uint8_t [strong]
```

Enumerator

DLPF0_246HZ	
DLPF_246HZ	

DLPF_111HZ	
DLPF_50HZ	
DLPF_24HZ	
DLPF_12HZ	
DLPF_6HZ	
DLPF_473HZ	

Definition at line 84 of file [DevLab_ICM20948.h](#).

7.19.1.3 ICM20948_Accel_FullScale

```
enum class ICM20948_Accel_FullScale : uint8_t [strong]
```

Enumerator

G_2	
G_4	
G_8	
G_16	

Definition at line 68 of file [DevLab_ICM20948.h](#).

7.19.1.4 ICM20948_Clock_Source

```
enum class ICM20948_Clock_Source : uint8_t [strong]
```

Enumerator

INTERNAL_20MHZ_0	
AUTO_PLL_1	
AUTO_PLL_2	
AUTO_PLL_3	
AUTO_PLL_4	
AUTO_PLL_5	
INTERNAL_20MHZ_6	

STOP_CLOCK	
------------	--

Definition at line 25 of file [DevLab_ICM20948.h](#).

7.19.1.5 ICM20948_Gyro_Average

```
enum class ICM20948_Gyro_Average : uint8_t [strong]
```

Enumerator

AVG_1	
AVG_2	
AVG_4	
AVG_8	
AVG_16	
AVG_32	
AVG_64	
AVG_128	

Definition at line 49 of file [DevLab_ICM20948.h](#).

7.19.1.6 ICM20948_Gyro_DLPF

```
enum class ICM20948_Gyro_DLPF : uint8_t [strong]
```

Enumerator

DLPF_196HZ	
DLPF_151HZ	
DLPF_119HZ	
DLPF_51HZ	
DLPF_23HZ	
DLPF_11HZ	
DLPF_5HZ	
DLPF_361HZ	

Definition at line 37 of file [DevLab_ICM20948.h](#).

7.19.1.7 ICM20948_Gyro_FullScale

```
enum class ICM20948_Gyro_FullScale : uint8_t [strong]
```

Enumerator

DPS_250	
DPS_500	
DPS_1000	
DPS_2000	

Definition at line 60 of file [DevLab_ICM20948.h](#).

7.19.1.8 ICM20948_Op_Mode

```
enum class ICM20948_Op_Mode : uint8_t [strong]
```

Enumerator

MODE_POWER_DOWN	
MODE_SINGLE_MEASUREMENT	
MODE_CONTINUOUS_1	
MODE_CONTINUOUS_2	
MODE_CONTINUOUS_3	
MODE_CONTINUOUS_4	
MODE_SELF_TEST	

Definition at line 14 of file [DevLab_ICM20948.h](#).

7.20 DevLab_ICM20948.h

[Go to the documentation of this file.](#)

```
00001 #ifndef ICM20948_DEVLAB_H
00002 #define ICM20948_DEVLAB_H
00003
00004 #include <Arduino.h>
00005 #include <Wire.h>
00006 #include <SPI.h>
00007 #include "ICM20948_regs.h"
00008
00009 #include "7Semi_Interface.h"
00010 #include "7Semi_I2C_Interface.h"
00011 #include "7Semi_SPI_Interface.h"
00012 #include "BusIO_7Semi.h"
00013
00014 enum class ICM20948_Op_Mode : uint8_t
00015 {
00016     MODE_POWER_DOWN = 0x00,
00017     MODE_SINGLE_MEASUREMENT = 0x01,
00018     MODE_CONTINUOUS_1 = 0x02,
```

```

00019  MODE_CONTINUOUS_2 = 0x04,
00020  MODE_CONTINUOUS_3 = 0x06,
00021  MODE_CONTINUOUS_4 = 0x08,
00022  MODE_SELF_TEST = 0x10
00023 };
00024
00025 enum class ICM20948_Clock_Source : uint8_t
00026 {
00027     INTERNAL_20MHZ_0 = 0x00, // Internal oscillator
00028     AUTO_PLL_1 = 0x01,      // Auto select (PLL if ready)
00029     AUTO_PLL_2 = 0x02,
00030     AUTO_PLL_3 = 0x03,
00031     AUTO_PLL_4 = 0x04,
00032     AUTO_PLL_5 = 0x05,
00033     INTERNAL_20MHZ_6 = 0x06, // Internal oscillator
00034     STOP_CLOCK = 0x07       // Stops clock, timing generator reset
00035 };
00036
00037 enum class ICM20948_Gyro_DLPF : uint8_t
00038 {
00039     DLPF_196HZ = 0x00, // BW 196.6 Hz, NBW 229.8 Hz
00040     DLPF_151HZ = 0x01, // BW 151.8 Hz, NBW 187.6 Hz
00041     DLPF_119HZ = 0x02, // BW 119.5 Hz, NBW 154.3 Hz
00042     DLPF_51HZ = 0x03,  // BW 51.2 Hz, NBW 73.3 Hz
00043     DLPF_23HZ = 0x04,  // BW 23.9 Hz, NBW 35.9 Hz
00044     DLPF_11HZ = 0x05,  // BW 11.6 Hz, NBW 17.8 Hz
00045     DLPF_5HZ = 0x06,   // BW 5.7 Hz, NBW 8.9 Hz
00046     DLPF_361HZ = 0x07 // BW 361.4 Hz, NBW 376.5 Hz
00047 };
00048
00049 enum class ICM20948_Gyro_Average : uint8_t
00050 {
00051     AVG_1 = 0x00,      // 1x Average (no averaging)
00052     AVG_2 = 0x01,      // 2x Average
00053     AVG_4 = 0x02,      // 4x Average
00054     AVG_8 = 0x03,      // 8x Average
00055     AVG_16 = 0x04,     // Average 16 samples
00056     AVG_32 = 0x05,     // Average 32 samples
00057     AVG_64 = 0x06,     // Average 64 samples
00058     AVG_128 = 0x07     // Average 128 samples
00059 };
00060 enum class ICM20948_Gyro_FullScale : uint8_t
00061 {
00062     DPS_250 = 0x00,    // ±250 dps
00063     DPS_500 = 0x01,    // ±500 dps
00064     DPS_1000 = 0x02,   // ±1000 dps
00065     DPS_2000 = 0x03    // ±2000 dps
00066 };
00067
00068 enum class ICM20948_Accel_FullScale : uint8_t
00069 {
00070     G_2 = 0x00, // ±2g
00071     G_4 = 0x01, // ±4g
00072     G_8 = 0x02, // ±8g
00073     G_16 = 0x03 // ±16g
00074 };
00075
00076 enum class ICM20948_Accel_Average : uint8_t
00077 {
00078     AVG_1_OR_4 = 0x00, // Depends on ACCEL_FCHOICE
00079     AVG_8 = 0x01,      // Average 8 samples
00080     AVG_16 = 0x02,     // Average 16 samples
00081     AVG_32 = 0x03      // Average 32 samples
00082 };
00083
00084 enum class ICM20948_Accel_DLPFCFG : uint8_t
00085 {
00086     DLPF0_246HZ = 0x00, // BW 246 Hz, NBW 265 Hz
00087     DLPF_246HZ = 0x01,  // BW 246 Hz, NBW 265 Hz
00088     DLPF_111HZ = 0x02,  // BW 111 Hz, NBW 136 Hz
00089     DLPF_50HZ = 0x03,   // BW 50 Hz, NBW 68.8 Hz
00090     DLPF_24HZ = 0x04,   // BW 24 Hz, NBW 34.4 Hz
00091     DLPF_12HZ = 0x05,   // BW 12 Hz, NBW 17 Hz
00092     DLPF_6HZ = 0x06,    // BW 6 Hz, NBW 8.3 Hz
00093     DLPF_473HZ = 0x07,  // BW 473 Hz, NBW 499 Hz
00094 };
00095
00096 struct ICM20948_IntPinConfig
00097 {
00098     uint8_t activeLevel : 1; // bit 7 | 0 = active high, 1 = active low
00099     uint8_t driveMode : 1;  // bit 6 | 0 = push-pull, 1 = open-drain
00100     uint8_t latchMode : 1;  // bit 5 | 0 = pulse 50µs, 1 = latch held
00101     uint8_t clearMode : 1;  // bit 4 | 0 = read INT_STATUS, 1 = any read
00102     uint8_t fsyncActLevel : 1; // bit 3 | 0 = FSYNC active high, 1 = FSYNC active low
00103     uint8_t fsyncIntEn : 1;  // bit 2 | 0 = FSYNC disabled, 1 = FSYNC as interrupt
00104     uint8_t bypassEn : 1;    // bit 1 | 0 = normal, 1 = I2C bypass mode
00105 };

```

```

00106
00107 struct ICM20948_IntEnableConfig
00108 {
00109     // --- INT_ENABLE (0x10) ---
00110     uint8_t wofEn      : 1; // Wake on FSYNC
00111     uint8_t womIntEn   : 1; // Wake on motion
00112     uint8_t pllRdyEn   : 1; // PLL ready
00113     uint8_t dmpInt1En  : 1; // DMP interrupt
00114     uint8_t i2cMstIntEn : 1; // I2C master interrupt
00115
00116     // --- INT_ENABLE_1 (0x11) ---
00117     uint8_t rawDataRdyEn : 1; // Raw data ready
00118
00119     // --- INT_ENABLE_2 (0x12) --- canal por canal
00120     uint8_t fifoOvfEn[5]; // [0]-[4]: 1 = overflow interrupt ON para ese canal
00121
00122     // --- INT_ENABLE_3 (0x13) --- canal por canal
00123     uint8_t fifoWmEn[5]; // [0]-[4]: 1 = watermark interrupt ON para ese canal
00124 };
00125
00126 struct ICM20948_IntStatus
00127 {
00128     // --- INT_STATUS (0x19) ---
00129     uint8_t womInt      : 1; // Wake on motion ocurrió
00130     uint8_t pllRdyInt   : 1; // PLL listo
00131     uint8_t dmpInt1     : 1; // DMP generó interrupción
00132     uint8_t i2cMstInt   : 1; // I2C master generó interrupción
00133
00134     // --- INT_STATUS_1 (0x1A) ---
00135     uint8_t rawDataRdy : 1; // Datos de sensores listos para leer
00136
00137     // --- INT_STATUS_2 (0x1B) ---
00138     uint8_t fifoOvf[5]; // [0]-[4]: FIFO overflow por canal
00139
00140     // --- INT_STATUS_3 (0x1C) ---
00141     uint8_t fifoWm[5]; // [0]-[4]: FIFO watermark por canal
00142 };
00143
00144 /**
00145  * Class
00146  * - Manages ICM-20948 over I2C (SPI path disabled in this cut)
00147  * - Caches scale factors for LSB->physical conversions
00148  */
00149 class DevLab_ICM20948
00150 {
00151 public:
00152     DevLab_ICM20948();
00153
00154     /**
00155      * beginI2C
00156      *
00157      * - Initialize ICM20948 using I2C interface
00158      * - Sets up communication layer (Interface + BusIO)
00159      * - Verifies device identity (WHO_AM_I)
00160      * - Configures basic power and clock settings
00161      *
00162      * Returns:
00163      * - true → Device initialized successfully
00164      * - false → Communication or configuration failed
00165      */
00166     bool beginI2C(uint8_t address = 0x69, TwoWire &i2cPort = Wire, uint32_t i2cSpeed = 400000);
00167
00168     bool beginSPI(uint8_t csPin, SPIClass &spiPort = SPI, uint32_t spiSpeed = 1000000);
00169
00170     /**
00171      * readWhoAmI
00172      *
00173      * - Read WHO_AM_I register (device ID)
00174      * - Used during initialization to verify ICM20948
00175      * - Expected value: 0xEA
00176      *
00177      * Returns:
00178      * - true → Read successful
00179      * - false → Operation failed
00180      */
00181     bool readWhoAmI(uint8_t &whoAmI);
00182
00183     /**
00184      * selectBank
00185      *
00186      * - Select USER BANK (0-3)
00187      * - Used to access different register groups inside ICM20948
00188      *
00189      * Returns:
00190      * - true → Bank switch successful
00191      * - false → Write failed
00192

```

```

00193     */
00194     bool selectBank(uint8_t bank);
00195
00196     /**
00197     * softReset
00198     *
00199     * - Perform software reset of ICM20948
00200     * - Resets all internal registers to default state
00201     *
00202     * Flow:
00203     * - Select USER BANK 0
00204     * - Set DEVICE_RESET bit
00205     * - Wait for device to restart
00206     *
00207     * Returns:
00208     * - true → Reset successful
00209     * - false → Operation failed
00210     */
00211     bool softReset();
00212
00213     /**
00214     * sleep
00215     *
00216     * - Enable or disable sleep mode
00217     * - Maintains clock source (CLKSEL = 1)
00218     *
00219     * Flow:
00220     * - Select USER BANK 0
00221     * - Set or clear SLEEP bit
00222     *
00223     * Returns:
00224     * - true → Operation successful
00225     * - false → Operation failed
00226     */
00227     bool sleep(bool en);
00228
00229     /**
00230     * applyBasicDefaults
00231     *
00232     * - Apply basic sensor configuration
00233     * - Configure power, filters, ranges, and sampling rates
00234     *
00235     * Returns:
00236     * - true → Configuration successful
00237     * - false → Any register write failed
00238     */
00239     bool applyBasicDefaults();
00240
00241     /**
00242     * readAccel
00243     *
00244     * - Read accelerometer data (X, Y, Z)
00245     * - Convert raw values to g using LSB scale
00246     *
00247     * Returns:
00248     * - true → Read successful
00249     * - false → Read failed
00250     */
00251     bool readAccel(float &x, float &y, float &z);
00252
00253     /**
00254     * readGyro
00255     *
00256     * - Read gyroscope data (X, Y, Z)
00257     * - Convert raw values to dps using LSB scale
00258     *
00259     * Returns:
00260     * - true → Read successful
00261     * - false → Read failed
00262     */
00263     bool readGyro(float &x, float &y, float &z);
00264
00265     /**
00266     * readTemperature
00267     *
00268     * - Read temperature sensor
00269     * - Convert raw values to °C
00270     *
00271     * Returns:
00272     * - true → Read successful
00273     * - false → Read failed
00274     */
00275     bool readTemperature(float &temperature);
00276
00277     /**
00278     * readMag
00279     *

```

```

00280     * - Read magnetometer data via internal I2C master
00281     * - Data comes from EXT_SLV_SENS_DATA registers
00282     * - Convert raw values to  $\mu$ T
00283     *
00284     * Returns:
00285     * - true → Read successful
00286     * - false → Read failed
00287     */
00288     bool readMag(float &x, float &y, float &z);
00289
00290     /**
00291     * initMag
00292     *
00293     * - Initialize AK09916 magnetometer using internal I2C master
00294     * - Verifies WHO_AM_I and enables continuous mode
00295     * - Configures SLV0 for automatic data read
00296     *
00297     * Returns:
00298     * - true → Initialization successful
00299     * - false → Operation failed
00300     */
00301     bool initMag();
00302
00303     /**
00304     * setMagOpMode
00305     *
00306     * - Set magnetometer operation mode
00307     *
00308     * Returns:
00309     * - true → Mode set successfully
00310     * - false → Write failed
00311     */
00312     bool setMagOpMode(ICM20948_Op_Mode opMode);
00313
00314     /**
00315     * setLowPower
00316     *
00317     * - Enable or disable low power mode
00318     * - Controls LP_EN bit in PWR_MGMT_1
00319     *
00320     * Returns:
00321     * - true → Operation successful
00322     * - false → Operation failed
00323     */
00324     bool setLowPower(bool enable);
00325
00326     /**
00327     * getLowPower
00328     *
00329     * - Read low power mode status
00330     * - Returns state of LP_EN bit
00331     *
00332     * Returns:
00333     * - true → Read successful
00334     * - false → Operation failed
00335     */
00336     bool getLowPower(bool &enable);
00337
00338     /**
00339     * setClock
00340     *
00341     * - Set clock source (CLKSEL [2:0])
00342     * - Selects internal clock for sensor operation
00343     *
00344     * Returns:
00345     * - true → Operation successful
00346     * - false → Operation failed
00347     */
00348     bool setClock(ICM20948_Clock_Source clock);
00349
00350     /**
00351     * getClock
00352     *
00353     * - Read current clock source (CLKSEL [2:0])
00354     *
00355     * Returns:
00356     * - true → Read successful
00357     * - false → Operation failed
00358     */
00359     bool getClock(uint8_t &clock);
00360
00361     /**
00362     * setGyroSampleRate
00363     *
00364     * - Set gyroscope sample rate
00365     * - Uses internal divider (SRD) based on 1100 Hz base rate
00366     *

```

```

00367     * Returns:
00368     * - true → Operation successful
00369     * - false → Invalid input or write failed
00370     */
00371     bool setGyroSampleRate(float sampleRate);
00372
00373     /**
00374     * getGyroSampleRate
00375     *
00376     * - Get current gyroscope sample rate
00377     * - Computes value from divider register
00378     *
00379     * Returns:
00380     * - true → Read successful
00381     * - false → Operation failed
00382     */
00383     bool getGyroSampleRate(float &sampleRate);
00384     bool setGyroDivRate(uint8_t divisor);
00385     /**
00386     * setDLPF
00387     *
00388     * - Configure gyroscope digital low-pass filter (DLPF)
00389     * - Option to bypass filter (full bandwidth)
00390     *
00391     * Returns:
00392     * - true → Operation successful
00393     * - false → Operation failed
00394     */
00395     bool setDLPF(ICM20948_Gyro_DLPF dlpf, bool bypass);
00396
00397     /**
00398     * getDLPF
00399     *
00400     * - Read gyroscope DLPF configuration
00401     * - Returns filter setting and bypass state
00402     *
00403     * Returns:
00404     * - true → Read successful
00405     * - false → Operation failed
00406     */
00407     bool getDLPF(uint8_t &dlpf, bool &bypass);
00408
00409     /**
00410     * setGyroScale
00411     *
00412     * - Set gyroscope full-scale range (FS_SEL)
00413     * - Controls sensitivity (dps range)
00414     *
00415     * Flow:
00416     * - Validate bus pointer
00417     * - Select USER BANK 2
00418     * - Write FS_SEL bits [2:1]
00419     *
00420     * Returns:
00421     * - true → Operation successful
00422     * - false → Operation failed
00423     */
00424     bool setGyroScale(ICM20948_Gyro_FullScale fullScale);
00425     bool setGyroAveraging(ICM20948_Gyro_Average avg);
00426     /**
00427     * getGyroScale
00428     *
00429     * - Get current gyroscope full-scale range (FS_SEL)
00430     *
00431     * Returns:
00432     * - true → Read successful
00433     * - false → Operation failed
00434     */
00435     bool getGyroScale(uint8_t &fullScale);
00436
00437     /**
00438     * selfTestGyro
00439     *
00440     * - Enable or disable gyroscope self-test per axis
00441     *
00442     * Returns:
00443     * - true → Operation successful
00444     * - false → Operation failed
00445     */
00446     bool selfTestGyro(bool x, bool y, bool z);
00447
00448     bool setAccelDivRate(uint16_t divisor);
00449     /**
00450     * setAccelSampleRate
00451     *
00452     * - Set accelerometer output data rate (ODR)
00453     * - Uses 1125 Hz base rate with 12-bit divider

```

```
00454     *
00455     * Returns:
00456     * - true → Operation successful
00457     * - false → Operation failed
00458     */
00459 bool setAccelSampleRate(uint16_t sampleRate);
00460
00461 /**
00462     * getAccelSampleRate
00463     *
00464     * - Get accelerometer output data rate (ODR)
00465     * - Computes value from divider registers
00466     *
00467     * Returns:
00468     * - true → Read successful
00469     * - false → Operation failed
00470     */
00471 bool getAccelSampleRate(float &sampleRate);
00472
00473 /**
00474     * selfTestAccel
00475     *
00476     * - Enable or disable accelerometer self-test per axis
00477     *
00478     * Returns:
00479     * - true → Operation successful
00480     * - false → Operation failed
00481     */
00482 bool selfTestAccel(bool x, bool y, bool z);
00483
00484 /**
00485     * setAccelScale
00486     *
00487     * - Set accelerometer full-scale range (FS_SEL)
00488     * - Controls measurement range (±2g, ±4g, ±8g, ±16g)
00489     *
00490     * Returns:
00491     * - true → Operation successful
00492     * - false → Operation failed
00493     */
00494 bool setAccelScale(ICM20948_Accel_FullScale fullScale);
00495
00496 /**
00497     * getAccelScale
00498     *
00499     * - Get current accelerometer full-scale range (FS_SEL)
00500     *
00501     * Returns:
00502     * - true → Read successful
00503     * - false → Operation failed
00504     */
00505 bool getAccelScale(uint8_t &fullScale);
00506
00507 /**
00508     * setAccelDLPF
00509     *
00510     * - Configure accelerometer digital low-pass filter (DLPF)
00511     * - Option to bypass filter (full bandwidth)
00512     *
00513     * Flow:
00514     * - Validate bus pointer
00515     * - Select USER BANK 2
00516     * - Set or clear bypass bit
00517     * - Configure DLPF bits if not bypassed
00518     *
00519     * Returns:
00520     * - true → Operation successful
00521     * - false → Operation failed
00522     */
00523 bool setAccelDLPF(uint8_t dlpf, bool bypass);
00524
00525 /**
00526     * getAccelDLPF
00527     *
00528     * - Read accelerometer DLPF configuration
00529     * - Returns filter setting and bypass state
00530     *
00531     * Flow:
00532     * - Validate bus pointer
00533     * - Select USER BANK 2
00534     * - Read ACCEL_CONFIG register
00535     * - Extract bypass and DLPF bits
00536     *
00537     * Returns:
00538     * - true → Read successful
00539     * - false → Operation failed
00540     */
```

```
00541 bool getAccelDLPF(uint8_t &dlpf, bool &bypass);
00542
00543 /**
00544  * setAccelAveraging
00545  *
00546  * - Configure accelerometer averaging / decimation (DEC3)
00547  * - Controls internal averaging of accel samples
00548  *
00549  * Flow:
00550  * - Validate bus pointer
00551  * - Select USER BANK 2
00552  * - Write DEC3 bits [1:0]
00553  *
00554  * Returns:
00555  * - true → Operation successful
00556  * - false → Operation failed
00557  */
00558 bool setAccelAveraging(ICM20948_Accel_Average avg);
00559
00560 /**
00561  * getAccelAveraging
00562  *
00563  * - Read accelerometer averaging / decimation setting (DEC3)
00564  *
00565  * Flow:
00566  * - Validate bus pointer
00567  * - Select USER BANK 2
00568  * - Read DEC3 bits [1:0]
00569  *
00570  * Returns:
00571  * - true → Read successful
00572  * - false → Operation failed
00573  */
00574 bool getAccelAveraging(uint8_t &avg);
00575
00576 /**
00577  * setSensors
00578  *
00579  * - Enable or disable accel, gyro, and temperature sensor
00580  * - Controls power gating via PWR_MGMT_2 and TEMP_DIS
00581  *
00582  * Flow:
00583  * - Validate bus pointer
00584  * - Select USER BANK 0
00585  * - Build PWR_MGMT_2 mask
00586  * - Configure temperature enable/disable
00587  *
00588  * Returns:
00589  * - true → Operation successful
00590  * - false → Operation failed
00591  */
00592 bool setSensors(bool accel_on, bool gyro_on, bool temp_on);
00593
00594 /**
00595  * getSensors
00596  *
00597  * - Read accel, gyro, and temperature enable state
00598  *
00599  * Flow:
00600  * - Validate bus pointer
00601  * - Select USER BANK 0
00602  * - Read PWR_MGMT_2 register
00603  * - Read TEMP_DIS bit
00604  * - Decode enable states
00605  *
00606  * Returns:
00607  * - true → Read successful
00608  * - false → Operation failed
00609  */
00610 bool getSensors(bool &accel_on, bool &gyro_on, bool &temp_on);
00611
00612 /**
00613  * setGyroOffset
00614  *
00615  * - Set gyroscope offset values for X, Y, Z axes
00616  * - Writes offset registers in USER BANK 2
00617  *
00618  * Flow:
00619  * - Validate bus pointer
00620  * - Select USER BANK 2
00621  * - Pack offsets into byte array
00622  * - Write to offset registers
00623  *
00624  * Returns:
00625  * - true → Operation successful
00626  * - false → Operation failed
00627  */
```



```

00628 bool setGyroOffset(uint16_t offsetX, uint16_t offsetY, uint16_t offsetZ);
00629
00630 /**
00631  * getGyroOffset
00632  *
00633  * - Read gyroscope offset values for X, Y, Z axes
00634  * - Reads offset registers from USER BANK 2
00635  *
00636  * Flow:
00637  * - Validate bus pointer
00638  * - Select USER BANK 2
00639  * - Read offset registers
00640  * - Convert to signed values
00641  *
00642  * Returns:
00643  * - true → Read successful
00644  * - false → Operation failed
00645  */
00646 bool getGyroOffset(int16_t &offsetX, int16_t &offsetY, int16_t &offsetZ);
00647
00648 /**
00649  * setAccelOffset
00650  *
00651  * - Set accelerometer offset values for X, Y, Z axes
00652  * - Writes offset registers in USER BANK 1
00653  *
00654  * Flow:
00655  * - Validate bus pointer
00656  * - Select USER BANK 1
00657  * - Pack offsets into byte array
00658  * - Write to offset registers
00659  *
00660  * Returns:
00661  * - true → Operation successful
00662  * - false → Operation failed
00663  */
00664 bool setAccelOffset(int16_t offsetX, int16_t offsetY, int16_t offsetZ);
00665
00666 /**
00667  * getAccelOffset
00668  *
00669  * - Read accelerometer offset values for X, Y, Z axes
00670  * - Reads offset registers from USER BANK 1
00671  *
00672  * Flow:
00673  * - Validate bus pointer
00674  * - Select USER BANK 1
00675  * - Read offset registers
00676  * - Convert to signed values
00677  *
00678  * Returns:
00679  * - true → Read successful
00680  * - false → Operation failed
00681  */
00682 bool getAccelOffset(int16_t &offsetX, int16_t &offsetY, int16_t &offsetZ);
00683
00684 bool intInit();
00685
00686 bool checkIntStatus(ICM20948_IntStatus &status);
00687
00688 private:
00689 I2C_Interface i2c;
00690 SPI_Interface spi;
00691
00692 Interface_7Semi *iface = nullptr;
00693
00694 BusIO_7Semi<Interface_7Semi> *bus = nullptr;
00695
00696 float mg_per_lsb = 16384.0f; // LSB/g at ±16g
00697 float degree_per_second = 131.072f; // LSB/dps at ±2000 dps
00698
00699 bool writeSlave4(uint8_t reg, uint8_t value);
00700 bool readSlave4(uint8_t reg, uint8_t &value);
00701 };
00702
00703
00704 #endif /* ICM20948_DEVLAB_H */

```

7.21 src/ICM20948_platform.h File Reference

Classes

- struct [icm_plat_t](#)

7.22 ICM20948_platform.h

[Go to the documentation of this file.](#)

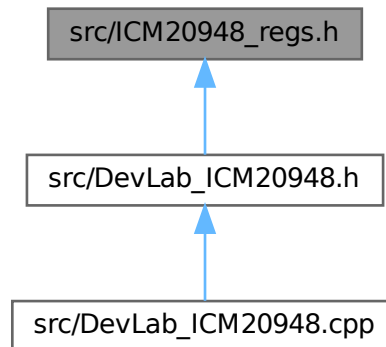
```

00001 #ifndef ICM20948_PLATFORM_H
00002 #define ICM20948_PLATFORM_H
00003
00004 // #include <stdint.h>
00005 // #include <stddef.h>
00006
00007 /**
00008  * 7Semi comment style
00009  * - Platform glue for bus IO, delays, and timing
00010  * - Implement these for your system (I2C or SPI)
00011  */
00012
00013 typedef struct {
00014     /**
00015      * - Bus read: read `len` bytes from `reg` into `buf`
00016      * - Return 0 on success, nonzero on error
00017      */
00018     int (*read)(uint8_t reg, uint8_t *buf, size_t len, void *user);
00019     /**
00020      * - Bus write: write `len` bytes from `buf` to `reg`
00021      * - Return 0 on success, nonzero on error
00022      */
00023     int (*write)(uint8_t reg, const uint8_t *buf, size_t len, void *user);
00024     /**
00025      * - Optional: SPI transfers may need register R/W bit mangling
00026      * - For I2C, leave as NULL
00027      */
00028     uint8_t (*spi_reg_rw_bits)(uint8_t reg, int is_write);
00029     /**
00030      * - Millisecond delay
00031      */
00032     void (*delay_ms)(uint32_t ms);
00033     /**
00034      * - User context pointer passed to read/write
00035      */
00036     void *user;
00037     /**
00038      * - If using SPI: set chip select active (1) or inactive (0)
00039      * - For I2C: leave as NULL
00040      */
00041     void (*spi_cs)(int level, void *user);
00042     /**
00043      * - Device address (I2C) or ignored for SPI if you wrap inside user
00044      */
00045     uint8_t i2c_addr;
00046 } icm_plat_t;
00047
00048 #endif

```

7.23 src/ICM20948_regs.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- #define [WHO_AM_I](#) 0x00 /* WHO_AM_I[7:0] */
- #define [WHO_AM_I_VAL](#) 0xEA
- #define [USER_CTRL](#) 0x03 /* DMP_EN FIFO_EN I2C_MST_EN I2C_IF_DIS DMP_RST SRAM_RST I2C←
_MST_RST - */
- #define [USER_CTRL_DMP_EN](#) BIT(7)
- #define [USER_CTRL_FIFO_EN](#) BIT(6)
- #define [USER_CTRL_I2C_MST_EN](#) BIT(5)
- #define [USER_CTRL_I2C_IF_DIS](#) BIT(4)
- #define [USER_CTRL_DMP_RST](#) BIT(3)
- #define [USER_CTRL_SRAM_RST](#) BIT(2)
- #define [USER_CTRL_I2C_MST_RST](#) BIT(1)
- #define [LP_CONFIG](#) 0x05 /* I2C_MST_CYCLE ACCEL_CYCLE GYRO_CYCLE - */
- #define [LP_I2C_MST_CYCLE](#) BIT(6)
- #define [LP_ACCEL_CYCLE](#) BIT(5)
- #define [LP_GYRO_CYCLE](#) BIT(4)
- #define [PWR_MGMT_1](#) 0x06 /* DEVICE_RESET SLEEP LP_EN - TEMP_DIS CLKSEL[2:0] */
- #define [PWR_DEVICE_RESET](#) BIT(7)
- #define [PWR_SLEEP](#) BIT(6)
- #define [PWR_LP_EN](#) BIT(5)
- #define [PWR_TEMP_DIS](#) BIT(3)
- #define [PWR_CLKSEL_MASK](#) 0x07
- #define [PWR_CLKSEL_INT_20MHZ](#) 0x01
- #define [PWR_CLKSEL_AUTO](#) 0x01 /* typical: auto selects best source */
- #define [PWR_MGMT_2](#) 0x07 /* - DISABLE_ACCEL DISABLE_GYRO */
- #define [PWR_DISABLE_ACCEL](#) BIT(3)
- #define [PWR_DISABLE_GYRO](#) BIT(0)
- #define [INT_PIN_CFG](#) 0x0F /* INT1_ACTL INT1_OPEN INT1_LATCH_INT_EN INT_ANYRD_2CLEAR
ACTL_FSYNC FSYNC_INT_MODE_EN BYPASS_EN - */
- #define [INT1_ACTL](#) BIT(7) /* active low */

- #define INT1_OPEN BIT(6) /* open-drain */
- #define INT1_LATCH_INT_EN BIT(5) /* latch until status read */
- #define INT_ANYRD_2CLEAR BIT(4)
- #define INT_ACTL_FSYNC BIT(3)
- #define FSYNC_INT_MODE_EN BIT(2)
- #define BYPASS_EN BIT(1) /* I2C bypass to aux devices */
- #define INT_ENABLE 0x10 /* REG_WOF_EN - WOM_INT_EN PLL_RDY_EN DMP_INT1_EN I2C_MST_INT_EN */
- #define INT_REG_WOF_EN BIT(7)
- #define INT_WOM_INT_EN BIT(3)
- #define INT_PLL_RDY_EN BIT(2)
- #define INT_DMP_INT1_EN BIT(1)
- #define INT_I2C_MST_INT_EN BIT(0)
- #define INT_ENABLE_1 0x11 /* RAW_DATA_0_RDY_EN */
- #define INT_RAW_DATA_0_RDY_EN BIT(0)
- #define INT_ENABLE_2 0x12 /* FIFO_OVERFLOW_EN[4:0] */
- #define INT_ENABLE_3 0x13 /* FIFO_WM_EN[4:0] */
- #define I2C_MST_STATUS 0x17 /* PASS_THROUGH, *_DONE, *_NACK, LOST_ARB */
- #define MST_PASS_THROUGH BIT(7)
- #define MST_SLV4_DONE BIT(6)
- #define MST_LOST_ARB BIT(5)
- #define MST_SLV4_NACK BIT(4)
- #define MST_SLV3_NACK BIT(3)
- #define MST_SLV2_NACK BIT(2)
- #define MST_SLV1_NACK BIT(1)
- #define MST_SLV0_NACK BIT(0)
- #define INT_STATUS 0x19 /* WOM_INT PLL_RDY_INT DMP_INT1 I2C_MST_INT */
- #define STS_WOM_INT BIT(3)
- #define STS_PLL_RDY_INT BIT(2)
- #define STS_DMP_INT1 BIT(1)
- #define STS_I2C_MST_INT BIT(0)
- #define INT_STATUS_1 0x1A /* RAW_DATA_0_RDY_INT */
- #define STS_RAW_DATA_0_RDY_INT BIT(0)
- #define INT_STATUS_2 0x1B /* FIFO_OVERFLOW_INT[4:0] */
- #define INT_STATUS_3 0x1C /* FIFO_WM_INT[4:0] */
- #define DELAY_TIMEH 0x28
- #define DELAY_TIMEL 0x29
- #define ACCEL_XOUT_H 0x2D
- #define ACCEL_XOUT_L 0x2E
- #define ACCEL_YOUT_H 0x2F
- #define ACCEL_YOUT_L 0x30
- #define ACCEL_ZOUT_H 0x31
- #define ACCEL_ZOUT_L 0x32
- #define GYRO_XOUT_H 0x33
- #define GYRO_XOUT_L 0x34
- #define GYRO_YOUT_H 0x35
- #define GYRO_YOUT_L 0x36
- #define GYRO_ZOUT_H 0x37
- #define GYRO_ZOUT_L 0x38
- #define TEMP_OUT_H 0x39
- #define TEMP_OUT_L 0x3A
- #define EXT_SLV_SENS_DATA_00 0x3B
- #define EXT_SLV_SENS_DATA_23 0x52 /* contiguous range 0x3B..0x52 */
- #define FIFO_EN_1 0x66 /* SLV3..SLV0 FIFO_EN */
- #define FIFO_EN_2 0x67 /* ACCEL GYRO_Z/Y/X TEMP FIFO_EN */

- #define FIFO_RST 0x68 /* FIFO_RESET[4:0] */
- #define FIFO_MODE 0x69 /* FIFO_MODE[4:0] */
- #define FIFO_COUNTH 0x70
- #define FIFO_COUNTL 0x71
- #define FIFO_R_W 0x72
- #define FIFO_CFG 0x76
- #define REG_BANK_SEL 0x7F /* USER_BANK[1:0] */
- #define SELF_TEST_X_GYRO 0x02 /* XG_ST_DATA[7:0] */
- #define SELF_TEST_Y_GYRO 0x03 /* YG_ST_DATA[7:0] */
- #define SELF_TEST_Z_GYRO 0x04 /* ZG_ST_DATA[7:0] */
- #define SELF_TEST_X_ACCEL 0x0E /* XA_ST_DATA[7:0] */
- #define SELF_TEST_Y_ACCEL 0x0F /* YA_ST_DATA[7:0] */
- #define SELF_TEST_Z_ACCEL 0x10 /* ZA_ST_DATA[7:0] */
- #define XA_OFFS_H 0x14 /* XA_OFFS[14:7] */
- #define XA_OFFS_L 0x15 /* XA_OFFS[6:0] */
- #define YA_OFFS_H 0x17 /* YA_OFFS[14:7] */
- #define YA_OFFS_L 0x18 /* YA_OFFS[6:0] */
- #define ZA_OFFS_H 0x1A /* ZA_OFFS[14:7] */
- #define ZA_OFFS_L 0x1B /* ZA_OFFS[6:0] */
- #define TIMEBASE_CORRECTION_PLL 0x28 /* TBC_PLL[7:0] */
- #define BANK1_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
- #define GYRO_SMPLRT_DIV 0x00 /* GYRO_SMPLRT_DIV[7:0] */
- #define GYRO_CONFIG_1 0x01 /* GYRO_DLPFCFG[2:0] GYRO_FS_SEL[1:0] GYRO_FCHOICE */
- #define GYRO_FCHOICE BIT(0) /* when 0: DLPF on, when 1: off (per datasheet) */
- #define GYRO_FS_SEL_SHIFT 1
- #define GYRO_FS_SEL_MASK (0x3u << GYRO_FS_SEL_SHIFT)
- #define GYRO_FS_250DPS (0u << GYRO_FS_SEL_SHIFT)
- #define GYRO_FS_500DPS (1u << GYRO_FS_SEL_SHIFT)
- #define GYRO_FS_1000DPS (2u << GYRO_FS_SEL_SHIFT)
- #define GYRO_FS_2000DPS (3u << GYRO_FS_SEL_SHIFT)
- #define GYRO_DLPFCFG_SHIFT 5
- #define GYRO_DLPFCFG_MASK (0x7u << GYRO_DLPFCFG_SHIFT)
- #define GYRO_CONFIG_2 0x02 /* XGYRO_CTEN YGYRO_CTEN ZGYRO_CTEN GYRO_AVGCFG[2:0] */
- #define XGYRO_CTEN BIT(5)
- #define YGYRO_CTEN BIT(4)
- #define ZGYRO_CTEN BIT(3)
- #define GYRO_AVGCFG_SHIFT 0
- #define GYRO_AVGCFG_MASK (0x7u << GYRO_AVGCFG_SHIFT)
- #define XG_OFFS_USRH 0x03
- #define XG_OFFS_USRL 0x04
- #define YG_OFFS_USRH 0x05
- #define YG_OFFS_USRL 0x06
- #define ZG_OFFS_USRH 0x07
- #define ZG_OFFS_USRL 0x08
- #define ODR_ALIGN_EN 0x09 /* ODR_ALIGN_EN */
- #define ODR_ALIGN_EN_BIT BIT(0)
- #define ACCEL_SMPLRT_DIV_1 0x10 /* ACCEL_SMPLRT_DIV[11:8] */
- #define ACCEL_SMPLRT_DIV_2 0x11 /* ACCEL_SMPLRT_DIV[7:0] */
- #define ACCEL_INTEL_CTRL 0x12 /* ACCEL_INTEL_EN ACCEL_INTEL_MODE_INT */
- #define ACCEL_INTEL_EN BIT(7)
- #define ACCEL_INTEL_MODE_INT BIT(6)
- #define ACCEL_WOM_THR 0x13 /* WOM_THRESHOLD[7:0] */
- #define ACCEL_CONFIG 0x14 /* ACCEL_DLPFCFG[2:0] ACCEL_FS_SEL[1:0] ACCEL_FCHOICE */
- #define ACCEL_FCHOICE BIT(0)

- #define ACCEL_FS_SEL_SHIFT 1
- #define ACCEL_FS_SEL_MASK (0x3u << ACCEL_FS_SEL_SHIFT)
- #define ACCEL_FS_2G (0u << ACCEL_FS_SEL_SHIFT)
- #define ACCEL_FS_4G (1u << ACCEL_FS_SEL_SHIFT)
- #define ACCEL_FS_8G (2u << ACCEL_FS_SEL_SHIFT)
- #define ACCEL_FS_16G (3u << ACCEL_FS_SEL_SHIFT)
- #define ACCEL_DLPFCFG_SHIFT 5
- #define ACCEL_DLPFCFG_MASK (0x7u << ACCEL_DLPFCFG_SHIFT)
- #define ACCEL_CONFIG_2 0x15 /* AX_ST_EN_REG AY_ST_EN_REG AZ_ST_EN_REG DEC3_CFG[1:0] */
- #define AX_ST_EN_REG BIT(7)
- #define AY_ST_EN_REG BIT(6)
- #define AZ_ST_EN_REG BIT(5)
- #define DEC3_CFG_SHIFT 0
- #define DEC3_CFG_MASK (0x3u << DEC3_CFG_SHIFT)
- #define FSYNC_CONFIG 0x52 /* DELAY_TIME_EN WOF DEGLITCH_EN WOF_EDGE_INT EXT_SYNC_SET[3:0] */
- #define DELAY_TIME_EN BIT(7)
- #define WOF DEGLITCH_EN BIT(6)
- #define WOF_EDGE_INT BIT(5)
- #define EXT_SYNC_SET_MASK 0x0F
- #define TEMP_CONFIG 0x53 /* TEMP_DLPFCFG[2:0] */
- #define TEMP_DLPFCFG_SHIFT 5
- #define TEMP_DLPFCFG_MASK (0x7u << TEMP_DLPFCFG_SHIFT)
- #define MOD_CTRL_USR 0x54 /* REG_LP_DMP_EN */
- #define REG_LP_DMP_EN BIT(7)
- #define BANK2_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
- #define I2C_MST_ODR_CONFIG 0x00 /* I2C_MST_ODR_CONFIG[3:0] */
- #define MST_ODR_CFG_MASK 0x0F
- #define I2C_MST_CTRL 0x01 /* MULT_MST_EN - I2C_MST_P_NSR I2C_MST_CLK[3:0] */
- #define MULT_MST_EN BIT(7)
- #define I2C_MST_P_NSR BIT(4)
- #define I2C_MST_CLK_MASK 0x0F
- #define I2C_MST_DELAY_CTRL 0x02 /* DELAY_ES_SHADOW + I2C_SLVx_DELAY_EN bits */
- #define DELAY_ES_SHADOW BIT(7)
- #define I2C_SLV4_DELAY_EN BIT(4)
- #define I2C_SLV3_DELAY_EN BIT(3)
- #define I2C_SLV2_DELAY_EN BIT(2)
- #define I2C_SLV1_DELAY_EN BIT(1)
- #define I2C_SLV0_DELAY_EN BIT(0)
- #define I2C_SLV0_ADDR 0x03 /* RNW + ID[6:0] */
- #define I2C_SLVx_RNW BIT(7)
- #define I2C_SLV0_REG 0x04
- #define I2C_SLV0_CTRL 0x05 /* EN BYTE_SW REG_DIS GRP LENG[3:0] */
- #define I2C_SLVx_EN BIT(7)
- #define I2C_SLVx_BYTE_SW BIT(6)
- #define I2C_SLVx_REG_DIS BIT(5)
- #define I2C_SLVx_GRP BIT(4)
- #define I2C_SLVx_LENG_MASK 0x0F
- #define I2C_SLV0_DO 0x06
- #define I2C_SLV1_ADDR 0x07
- #define I2C_SLV1_REG 0x08
- #define I2C_SLV1_CTRL 0x09
- #define I2C_SLV1_DO 0x0A
- #define I2C_SLV2_ADDR 0x0B

- #define I2C_SLV2_REG 0x0C
- #define I2C_SLV2_CTRL 0x0D
- #define I2C_SLV2_DO 0x0E
- #define I2C_SLV3_ADDR 0x0F
- #define I2C_SLV3_REG 0x10
- #define I2C_SLV3_CTRL 0x11
- #define I2C_SLV3_DO 0x12
- #define I2C_SLV4_ADDR 0x13
- #define I2C_SLV4_REG 0x14
- #define I2C_SLV4_CTRL 0x15 /* EN BYTE_SW REG_DIS DLY[4:0] */
- #define I2C_SLV4_DLY_MASK 0x1F
- #define I2C_SLV4_DO 0x16
- #define I2C_SLV4_DI 0x17
- #define BANK3_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
- #define REG_BANK_SEL_USER_BANK_SHIFT 4
- #define REG_BANK_SEL_USER_BANK_MASK (0x3u << REG_BANK_SEL_USER_BANK_SHIFT)
- #define USER_BANK_0 BANK0
- #define USER_BANK_1 BANK1
- #define USER_BANK_2 BANK2
- #define USER_BANK_3 BANK3
- #define AK09916_I2C_ADDR 0x0C
- #define AK_WIA2 0x01
- #define AK_ST1 0x10
- #define AK_HXL 0x11
- #define AK_ST2 0x18
- #define AK_CNTL2 0x31
- #define AK_CNTL3 0x32
- #define AK_WIA2_VAL 0x09 /* AK09916C WIA2 expected */
- #define INTERNAL_20MHZ 0x00 /* 0: Internal 20 MHz RC */
- #define AUTO_SEL 0x01 /* 1–5: Auto/PLL preferred (use 1 as default) */
- #define CLK_STOP 0x07 /* 7: Stop clock / timing gen reset */
- #define g2 0
- #define g4 1
- #define g8 2
- #define g16 3
- #define ACCEL_FCHOICE_BYPASS 0
- #define ACCEL_FCHOICE_DLPF 1
- #define ACCEL_DLPFCFG_0 0
- #define ACCEL_DLPFCFG_1 1
- #define ACCEL_DLPFCFG_2 2
- #define ACCEL_DLPFCFG_3 3
- #define ACCEL_DLPFCFG_4 4
- #define ACCEL_DLPFCFG_5 5
- #define ACCEL_DLPFCFG_6 6
- #define ACCEL_DLPFCFG_7 7
- #define ACCEL_DEC3_AVG_4 0u /* averages 1 or 4 (depends on FCHOICE) */
- #define ACCEL_DEC3_AVG_8 1u
- #define ACCEL_DEC3_AVG_16 2u
- #define ACCEL_DEC3_AVG_32 3u
- #define ACCEL_SMPLRT_DIV_MIN 0u
- #define ACCEL_SMPLRT_DIV_MAX 4095u
- #define ACCEL_DLPF_BASE_HZ 1125.0f
- #define ACCEL_DLPF_ODR_HZ(div)
- #define ACCEL_BYPASS_3DB_BW_HZ 1209.0f
- #define ACCEL_BYPASS_NBW_HZ 1248.0f

- #define ACCEL_BYPASS_RATE_HZ 4500.0f
- #define ACCEL_DLPF0_3DB_BW_HZ 246.0f
- #define ACCEL_DLPF0_NBW_HZ 265.0f
- #define ACCEL_DLPF1_3DB_BW_HZ 246.0f
- #define ACCEL_DLPF1_NBW_HZ 265.0f
- #define ACCEL_DLPF2_3DB_BW_HZ 111.4f
- #define ACCEL_DLPF2_NBW_HZ 136.0f
- #define ACCEL_DLPF3_3DB_BW_HZ 50.4f
- #define ACCEL_DLPF3_NBW_HZ 68.8f
- #define ACCEL_DLPF4_3DB_BW_HZ 23.9f
- #define ACCEL_DLPF4_NBW_HZ 34.4f
- #define ACCEL_DLPF5_3DB_BW_HZ 11.5f
- #define ACCEL_DLPF5_NBW_HZ 17.0f
- #define ACCEL_DLPF6_3DB_BW_HZ 5.7f
- #define ACCEL_DLPF6_NBW_HZ 8.3f
- #define ACCEL_DLPF7_3DB_BW_HZ 473.0f
- #define ACCEL_DLPF7_NBW_HZ 499.0f
- #define dps250 0
- #define dps500 1
- #define dps1000 2
- #define dps2000 3
- #define GYRO_FCHOICE_BYPASS 0
- #define GYRO_FCHOICE_DLPF 1
- #define GYRO_DLPFCFG_0 0
- #define GYRO_DLPFCFG_1 1
- #define GYRO_DLPFCFG_2 2
- #define GYRO_DLPFCFG_3 3
- #define GYRO_DLPFCFG_4 4
- #define GYRO_DLPFCFG_5 5
- #define GYRO_DLPFCFG_6 6
- #define GYRO_DLPFCFG_7 7
- #define GYRO_SMPLRT_MIN_HZ 4.3f
- #define GYRO_DLPF_BASE_HZ 1100.0f
- #define GYRO_DLPF_ODR_HZ(rate_request)
- #define GYRO_BW_ULTRA_WIDE 0 /* bypass path */
- #define GYRO_BW_WIDE 1 /* e.g., CFG 0/1 */
- #define GYRO_BW_MEDIUM 2 /* e.g., CFG 2/3 */
- #define GYRO_BW_NARROW 3 /* e.g., CFG 4/5/6/7 */
- #define Hz2 0
- #define Hz6 1
- #define Hz8 2
- #define Hz10 3
- #define Hz15 4
- #define Hz20 5
- #define Hz25 6
- #define Hz30 7
- #define LowPower 0
- #define Regular 1
- #define EnhancedRegular 2
- #define HighAccuracy 3

7.23.1 Macro Definition Documentation

7.23.1.1 ACCEL_BYPASS_3DB_BW_HZ

```
#define ACCEL_BYPASS_3DB_BW_HZ 1209.0f
```

Definition at line 369 of file [ICM20948_regs.h](#).

7.23.1.2 ACCEL_BYPASS_NBW_HZ

```
#define ACCEL_BYPASS_NBW_HZ 1248.0f
```

Definition at line 370 of file [ICM20948_regs.h](#).

7.23.1.3 ACCEL_BYPASS_RATE_HZ

```
#define ACCEL_BYPASS_RATE_HZ 4500.0f
```

Definition at line 371 of file [ICM20948_regs.h](#).

7.23.1.4 ACCEL_CONFIG

```
#define ACCEL_CONFIG 0x14 /* ACCEL_DLPFCFG[2:0] ACCEL_FS_SEL[1:0] ACCEL\_FCHOICE */
```

Definition at line 209 of file [ICM20948_regs.h](#).

7.23.1.5 ACCEL_CONFIG_2

```
#define ACCEL_CONFIG_2 0x15 /* AX\_ST\_EN\_REG AY\_ST\_EN\_REG AZ\_ST\_EN\_REG DEC3_CFG[1:0] */
```

Definition at line 220 of file [ICM20948_regs.h](#).

7.23.1.6 ACCEL_DEC3_AVG_16

```
#define ACCEL_DEC3_AVG_16 2u
```

Definition at line 356 of file [ICM20948_regs.h](#).

7.23.1.7 ACCEL_DEC3_AVG_32

```
#define ACCEL_DEC3_AVG_32 3u
```

Definition at line 357 of file [ICM20948_regs.h](#).

7.23.1.8 ACCEL_DEC3_AVG_4

```
#define ACCEL_DEC3_AVG_4 0u /* averages 1 or 4 (depends on FCHOICE) */
```

Definition at line 354 of file [ICM20948_regs.h](#).

7.23.1.9 ACCEL_DEC3_AVG_8

```
#define ACCEL_DEC3_AVG_8 1u
```

Definition at line 355 of file [ICM20948_regs.h](#).

7.23.1.10 ACCEL_DLPF0_3DB_BW_HZ

```
#define ACCEL_DLPF0_3DB_BW_HZ 246.0f
```

Definition at line 376 of file [ICM20948_regs.h](#).

7.23.1.11 ACCEL_DLPF0_NBW_HZ

```
#define ACCEL_DLPF0_NBW_HZ 265.0f
```

Definition at line 377 of file [ICM20948_regs.h](#).

7.23.1.12 ACCEL_DLPF1_3DB_BW_HZ

```
#define ACCEL_DLPF1_3DB_BW_HZ 246.0f
```

Definition at line 378 of file [ICM20948_regs.h](#).

7.23.1.13 ACCEL_DLPF1_NBW_HZ

```
#define ACCEL_DLPF1_NBW_HZ 265.0f
```

Definition at line 379 of file [ICM20948_regs.h](#).

7.23.1.14 ACCEL_DLPF2_3DB_BW_HZ

```
#define ACCEL_DLPF2_3DB_BW_HZ 111.4f
```

Definition at line 380 of file [ICM20948_regs.h](#).

7.23.1.15 ACCEL_DLPF2_NBW_HZ

```
#define ACCEL_DLPF2_NBW_HZ 136.0f
```

Definition at line 381 of file [ICM20948_regs.h](#).

7.23.1.16 ACCEL_DLPF3_3DB_BW_HZ

```
#define ACCEL_DLPF3_3DB_BW_HZ 50.4f
```

Definition at line 382 of file [ICM20948_regs.h](#).

7.23.1.17 ACCEL_DLPF3_NBW_HZ

```
#define ACCEL_DLPF3_NBW_HZ 68.8f
```

Definition at line 383 of file [ICM20948_regs.h](#).

7.23.1.18 ACCEL_DLPF4_3DB_BW_HZ

```
#define ACCEL_DLPF4_3DB_BW_HZ 23.9f
```

Definition at line 384 of file [ICM20948_regs.h](#).

7.23.1.19 ACCEL_DLPF4_NBW_HZ

```
#define ACCEL_DLPF4_NBW_HZ 34.4f
```

Definition at line 385 of file [ICM20948_regs.h](#).

7.23.1.20 ACCEL_DLPF5_3DB_BW_HZ

```
#define ACCEL_DLPF5_3DB_BW_HZ 11.5f
```

Definition at line 386 of file [ICM20948_regs.h](#).

7.23.1.21 ACCEL_DLPF5_NBW_HZ

```
#define ACCEL_DLPF5_NBW_HZ 17.0f
```

Definition at line 387 of file [ICM20948_regs.h](#).

7.23.1.22 ACCEL_DLPF6_3DB_BW_HZ

```
#define ACCEL_DLPF6_3DB_BW_HZ 5.7f
```

Definition at line 388 of file [ICM20948_regs.h](#).

7.23.1.23 ACCEL_DLPF6_NBW_HZ

```
#define ACCEL_DLPF6_NBW_HZ 8.3f
```

Definition at line 389 of file [ICM20948_regs.h](#).

7.23.1.24 ACCEL_DLPF7_3DB_BW_HZ

```
#define ACCEL_DLPF7_3DB_BW_HZ 473.0f
```

Definition at line 390 of file [ICM20948_regs.h](#).

7.23.1.25 ACCEL_DLPF7_NBW_HZ

```
#define ACCEL_DLPF7_NBW_HZ 499.0f
```

Definition at line 391 of file [ICM20948_regs.h](#).

7.23.1.26 ACCEL_DLPF_BASE_HZ

```
#define ACCEL_DLPF_BASE_HZ 1125.0f
```

Definition at line 362 of file [ICM20948_regs.h](#).

7.23.1.27 ACCEL_DLPF_ODR_HZ

```
#define ACCEL_DLPF_ODR_HZ(  
    div)
```

Value:

```
(ACCEL_DLPF_BASE_HZ / (1.0f + (float)(div)))
```

Definition at line 363 of file [ICM20948_regs.h](#).

7.23.1.28 ACCEL_DLPFCFG_0

```
#define ACCEL_DLPFCFG_0 0
```

ACCEL_DLPFCFG (0..7) — choose cutoff/noise BW set (when DLPF is ON). Tip: start with ACCEL_DLPFCFG_3 for a good noise/latency tradeoff.

Definition at line 345 of file [ICM20948_regs.h](#).

7.23.1.29 ACCEL_DLPFCFG_1

```
#define ACCEL_DLPFCFG_1 1
```

Definition at line 346 of file [ICM20948_regs.h](#).

7.23.1.30 ACCEL_DLPFCFG_2

```
#define ACCEL_DLPFCFG_2 2
```

Definition at line 347 of file [ICM20948_regs.h](#).

7.23.1.31 ACCEL_DLPFCFG_3

```
#define ACCEL_DLPFCFG_3 3
```

Definition at line 348 of file [ICM20948_regs.h](#).

7.23.1.32 ACCEL_DLPFCFG_4

```
#define ACCEL_DLPFCFG_4 4
```

Definition at line 349 of file [ICM20948_regs.h](#).

7.23.1.33 ACCEL_DLPFCFG_5

```
#define ACCEL_DLPFCFG_5 5
```

Definition at line 350 of file [ICM20948_regs.h](#).

7.23.1.34 ACCEL_DLPFCFG_6

```
#define ACCEL_DLPFCFG_6 6
```

Definition at line 351 of file [ICM20948_regs.h](#).

7.23.1.35 ACCEL_DLPFCFG_7

```
#define ACCEL_DLPFCFG_7 7
```

Definition at line 352 of file [ICM20948_regs.h](#).

7.23.1.36 ACCEL_DLPFCFG_MASK

```
#define ACCEL_DLPFCFG_MASK (0x7u << ACCEL_DLPFCFG_SHIFT)
```

Definition at line 218 of file [ICM20948_regs.h](#).

7.23.1.37 ACCEL_DLPFCFG_SHIFT

```
#define ACCEL_DLPFCFG_SHIFT 5
```

Definition at line 217 of file [ICM20948_regs.h](#).

7.23.1.38 ACCEL_FCHOICE

```
#define ACCEL_FCHOICE BIT(0)
```

Definition at line 210 of file [ICM20948_regs.h](#).

7.23.1.39 ACCEL_FCHOICE_BYPASS

```
#define ACCEL_FCHOICE_BYPASS 0
```

Path select for accel:

- ACCEL_FCHOICE_BYPASS : DLPF bypassed (very wide BW, fastest)
- ACCEL_FCHOICE_DLPF : DLPF enabled (use ACCEL_DLPFCFG + SMPLRT_DIV)

Definition at line 337 of file [ICM20948_regs.h](#).

7.23.1.40 ACCEL_FCHOICE_DLPF

```
#define ACCEL_FCHOICE_DLPF 1
```

Definition at line 338 of file [ICM20948_regs.h](#).

7.23.1.41 ACCEL_FS_16G

```
#define ACCEL_FS_16G (3u << ACCEL_FS_SEL_SHIFT)
```

Definition at line 216 of file [ICM20948_regs.h](#).

7.23.1.42 ACCEL_FS_2G

```
#define ACCEL_FS_2G (0u << ACCEL_FS_SEL_SHIFT)
```

Definition at line 213 of file [ICM20948_regs.h](#).

7.23.1.43 ACCEL_FS_4G

```
#define ACCEL_FS_4G (1u << ACCEL_FS_SEL_SHIFT)
```

Definition at line 214 of file [ICM20948_regs.h](#).

7.23.1.44 ACCEL_FS_8G

```
#define ACCEL_FS_8G (2u << ACCEL_FS_SEL_SHIFT)
```

Definition at line 215 of file [ICM20948_regs.h](#).

7.23.1.45 ACCEL_FS_SEL_MASK

```
#define ACCEL_FS_SEL_MASK (0x3u << ACCEL_FS_SEL_SHIFT)
```

Definition at line 212 of file [ICM20948_regs.h](#).

7.23.1.46 ACCEL_FS_SEL_SHIFT

```
#define ACCEL_FS_SEL_SHIFT 1
```

Definition at line 211 of file [ICM20948_regs.h](#).

7.23.1.47 ACCEL_INTEL_CTRL

```
#define ACCEL_INTEL_CTRL 0x12 /* ACCEL_INTEL_EN ACCEL_INTEL_MODE_INT */
```

Definition at line 203 of file [ICM20948_regs.h](#).

7.23.1.48 ACCEL_INTEL_EN

```
#define ACCEL_INTEL_EN BIT(7)
```

Definition at line 204 of file [ICM20948_regs.h](#).

7.23.1.49 ACCEL_INTEL_MODE_INT

```
#define ACCEL_INTEL_MODE_INT BIT(6)
```

Definition at line 205 of file [ICM20948_regs.h](#).

7.23.1.50 ACCEL_SMPLRT_DIV_1

```
#define ACCEL_SMPLRT_DIV_1 0x10 /* ACCEL_SMPLRT_DIV[11:8] */
```

Definition at line 200 of file [ICM20948_regs.h](#).

7.23.1.51 ACCEL_SMPLRT_DIV_2

```
#define ACCEL_SMPLRT_DIV_2 0x11 /* ACCEL_SMPLRT_DIV[7:0] */
```

Definition at line 201 of file [ICM20948_regs.h](#).

7.23.1.52 ACCEL_SMPLRT_DIV_MAX

```
#define ACCEL_SMPLRT_DIV_MAX 4095u
```

Definition at line 361 of file [ICM20948_regs.h](#).

7.23.1.53 ACCEL_SMPLRT_DIV_MIN

```
#define ACCEL_SMPLRT_DIV_MIN 0u
```

Definition at line 360 of file [ICM20948_regs.h](#).

7.23.1.54 ACCEL_WOM_THR

```
#define ACCEL_WOM_THR 0x13 /* WOM_THRESHOLD[7:0] */
```

Definition at line 207 of file [ICM20948_regs.h](#).

7.23.1.55 ACCEL_XOUT_H

```
#define ACCEL_XOUT_H 0x2D
```

Definition at line 111 of file [ICM20948_regs.h](#).

7.23.1.56 ACCEL_XOUT_L

```
#define ACCEL_XOUT_L 0x2E
```

Definition at line 112 of file [ICM20948_regs.h](#).

7.23.1.57 ACCEL_YOUT_H

```
#define ACCEL_YOUT_H 0x2F
```

Definition at line 113 of file [ICM20948_regs.h](#).

7.23.1.58 ACCEL_YOUT_L

```
#define ACCEL_YOUT_L 0x30
```

Definition at line 114 of file [ICM20948_regs.h](#).

7.23.1.59 ACCEL_ZOUT_H

```
#define ACCEL_ZOUT_H 0x31
```

Definition at line 115 of file [ICM20948_regs.h](#).

7.23.1.60 ACCEL_ZOUT_L

```
#define ACCEL_ZOUT_L 0x32
```

Definition at line 116 of file [ICM20948_regs.h](#).

7.23.1.61 AK09916_I2C_ADDR

```
#define AK09916_I2C_ADDR 0x0C
```

Definition at line 310 of file [ICM20948_regs.h](#).

7.23.1.62 AK_CNTL2

```
#define AK_CNTL2 0x31
```

Definition at line 315 of file [ICM20948_regs.h](#).

7.23.1.63 AK_CNTL3

```
#define AK_CNTL3 0x32
```

Definition at line 316 of file [ICM20948_regs.h](#).

7.23.1.64 AK_HXL

```
#define AK_HXL 0x11
```

Definition at line 313 of file [ICM20948_regs.h](#).

7.23.1.65 AK_ST1

```
#define AK_ST1 0x10
```

Definition at line 312 of file [ICM20948_regs.h](#).

7.23.1.66 AK_ST2

```
#define AK_ST2 0x18
```

Definition at line 314 of file [ICM20948_regs.h](#).

7.23.1.67 AK_WIA2

```
#define AK_WIA2 0x01
```

Definition at line 311 of file [ICM20948_regs.h](#).

7.23.1.68 AK_WIA2_VAL

```
#define AK_WIA2_VAL 0x09 /* AK09916C WIA2 expected */
```

Definition at line 317 of file [ICM20948_regs.h](#).

7.23.1.69 AUTO_SEL

```
#define AUTO_SEL 0x01 /* 1-5: Auto/PLL preferred (use 1 as default) */
```

Definition at line 320 of file [ICM20948_regs.h](#).

7.23.1.70 AX_ST_EN_REG

```
#define AX_ST_EN_REG BIT(7)
```

Definition at line 221 of file [ICM20948_regs.h](#).

7.23.1.71 AY_ST_EN_REG

```
#define AY_ST_EN_REG BIT(6)
```

Definition at line 222 of file [ICM20948_regs.h](#).

7.23.1.72 AZ_ST_EN_REG

```
#define AZ_ST_EN_REG BIT(5)
```

Definition at line 223 of file [ICM20948_regs.h](#).

7.23.1.73 BANK1_REG_BANK_SEL

```
#define BANK1_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
```

Definition at line 163 of file [ICM20948_regs.h](#).

7.23.1.74 BANK2_REG_BANK_SEL

```
#define BANK2_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
```

Definition at line 240 of file [ICM20948_regs.h](#).

7.23.1.75 BANK3_REG_BANK_SEL

```
#define BANK3_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
```

Definition at line 296 of file [ICM20948_regs.h](#).

7.23.1.76 BYPASS_EN

```
#define BYPASS_EN BIT(1) /* I2C bypass to aux devices */
```

Definition at line 71 of file [ICM20948_regs.h](#).

7.23.1.77 CLK_STOP

```
#define CLK_STOP 0x07 /* 7: Stop clock / timing gen reset */
```

Definition at line 321 of file [ICM20948_regs.h](#).

7.23.1.78 DEC3_CFG_MASK

```
#define DEC3_CFG_MASK (0x3u << DEC3_CFG_SHIFT)
```

Definition at line 225 of file [ICM20948_regs.h](#).

7.23.1.79 DEC3_CFG_SHIFT

```
#define DEC3_CFG_SHIFT 0
```

Definition at line 224 of file [ICM20948_regs.h](#).

7.23.1.80 DELAY_ES_SHADOW

```
#define DELAY_ES_SHADOW BIT(7)
```

Definition at line 256 of file [ICM20948_regs.h](#).

7.23.1.81 DELAY_TIME_EN

```
#define DELAY_TIME_EN BIT(7)
```

Definition at line 228 of file [ICM20948_regs.h](#).

7.23.1.82 DELAY_TIMEH

```
#define DELAY_TIMEH 0x28
```

Definition at line 107 of file [ICM20948_regs.h](#).

7.23.1.83 DELAY_TIMEL

```
#define DELAY_TIMEL 0x29
```

Definition at line 108 of file [ICM20948_regs.h](#).

7.23.1.84 dps1000

```
#define dps1000 2
```

Definition at line 398 of file [ICM20948_regs.h](#).

7.23.1.85 dps2000

```
#define dps2000 3
```

Definition at line 399 of file [ICM20948_regs.h](#).

7.23.1.86 dps250

```
#define dps250 0
```

- $\pm 250/\pm 500/\pm 1000/\pm 2000$ dps (maps to FS_SEL 0..3)

Definition at line 396 of file [ICM20948_regs.h](#).

7.23.1.87 dps500

```
#define dps500 1
```

Definition at line 397 of file [ICM20948_regs.h](#).

7.23.1.88 EnhancedRegular

```
#define EnhancedRegular 2
```

Definition at line 456 of file [ICM20948_regs.h](#).

7.23.1.89 EXT_SLV_SENS_DATA_00

```
#define EXT_SLV_SENS_DATA_00 0x3B
```

Definition at line 127 of file [ICM20948_regs.h](#).

7.23.1.90 EXT_SLV_SENS_DATA_23

```
#define EXT_SLV_SENS_DATA_23 0x52 /* contiguous range 0x3B..0x52 */
```

Definition at line 128 of file [ICM20948_regs.h](#).

7.23.1.91 EXT_SYNC_SET_MASK

```
#define EXT_SYNC_SET_MASK 0x0F
```

Definition at line 231 of file [ICM20948_regs.h](#).

7.23.1.92 FIFO_CFG

```
#define FIFO_CFG 0x76
```

Definition at line 138 of file [ICM20948_regs.h](#).

7.23.1.93 FIFO_COUNTH

```
#define FIFO_COUNTH 0x70
```

Definition at line 134 of file [ICM20948_regs.h](#).

7.23.1.94 FIFO_COUNTL

```
#define FIFO_COUNTL 0x71
```

Definition at line 135 of file [ICM20948_regs.h](#).

7.23.1.95 FIFO_EN_1

```
#define FIFO_EN_1 0x66 /* SLV3..SLV0 FIFO_EN */
```

Definition at line 130 of file [ICM20948_regs.h](#).

7.23.1.96 FIFO_EN_2

```
#define FIFO_EN_2 0x67 /* ACCEL GYRO_Z/Y/X TEMP FIFO_EN */
```

Definition at line 131 of file [ICM20948_regs.h](#).

7.23.1.97 FIFO_MODE

```
#define FIFO_MODE 0x69 /* FIFO_MODE[4:0] */
```

Definition at line 133 of file [ICM20948_regs.h](#).

7.23.1.98 FIFO_R_W

```
#define FIFO_R_W 0x72
```

Definition at line 136 of file [ICM20948_regs.h](#).

7.23.1.99 FIFO_RST

```
#define FIFO_RST 0x68 /* FIFO_RESET[4:0] */
```

Definition at line 132 of file [ICM20948_regs.h](#).

7.23.1.100 FSYNC_CONFIG

```
#define FSYNC_CONFIG 0x52 /* DELAY_TIME_EN WOF_DEGLITCH_EN WOF_EDGE_INT EXT_SYNC_SET[3:0] */
```

Definition at line 227 of file [ICM20948_regs.h](#).

7.23.1.101 FSYNC_INT_MODE_EN

```
#define FSYNC_INT_MODE_EN BIT(2)
```

Definition at line 70 of file [ICM20948_regs.h](#).

7.23.1.102 g16

```
#define g16 3
```

Definition at line 329 of file [ICM20948_regs.h](#).

7.23.1.103 g2

```
#define g2 0
```

- $\pm 2g$ / $\pm 4g$ / $\pm 8g$ / $\pm 16g$ (maps to FS_SEL 0..3)

Definition at line 326 of file [ICM20948_regs.h](#).

7.23.1.104 g4

```
#define g4 1
```

Definition at line 327 of file [ICM20948_regs.h](#).

7.23.1.105 g8

```
#define g8 2
```

Definition at line 328 of file [ICM20948_regs.h](#).

7.23.1.106 GYRO_AVGCFG_MASK

```
#define GYRO_AVGCFG_MASK (0x7u << GYRO_AVGCFG_SHIFT)
```

Definition at line 188 of file [ICM20948_regs.h](#).

7.23.1.107 GYRO_AVGCFG_SHIFT

```
#define GYRO_AVGCFG_SHIFT 0
```

Definition at line 187 of file [ICM20948_regs.h](#).

7.23.1.108 GYRO_BW_MEDIUM

```
#define GYRO_BW_MEDIUM 2 /* e.g., CFG 2/3 */
```

Definition at line 435 of file [ICM20948_regs.h](#).

7.23.1.109 GYRO_BW_NARROW

```
#define GYRO_BW_NARROW 3 /* e.g., CFG 4/5/6/7 */
```

Definition at line 436 of file [ICM20948_regs.h](#).

7.23.1.110 GYRO_BW_ULTRA_WIDE

```
#define GYRO_BW_ULTRA_WIDE 0 /* bypass path */
```

Definition at line 433 of file [ICM20948_regs.h](#).

7.23.1.111 GYRO_BW_WIDE

```
#define GYRO_BW_WIDE 1 /* e.g., CFG 0/1 */
```

Definition at line 434 of file [ICM20948_regs.h](#).

7.23.1.112 GYRO_CONFIG_1

```
#define GYRO_CONFIG_1 0x01 /* GYRO_DLPFCFG[2:0] GYRO_FS_SEL[1:0] GYRO\_FCHOICE */
```

Definition at line 172 of file [ICM20948_regs.h](#).

7.23.1.113 GYRO_CONFIG_2

```
#define GYRO_CONFIG_2 0x02 /* XGYRO\_CTEN YGYRO\_CTEN ZGYRO\_CTEN GYRO_AVGCFG[2:0] */
```

Definition at line 183 of file [ICM20948_regs.h](#).

7.23.1.114 GYRO_DLPF_BASE_HZ

```
#define GYRO_DLPF_BASE_HZ 1100.0f
```

Definition at line 426 of file [ICM20948_regs.h](#).

7.23.1.115 GYRO_DLPF_ODR_HZ

```
#define GYRO_DLPF_ODR_HZ(  
    rate_request)
```

Value:

```
(rate_request) /* symbolic; your driver computes DIV = round(1100/rate)-1 */
```

Definition at line 427 of file [ICM20948_regs.h](#).

7.23.1.116 GYRO_DLPFCFG_0

```
#define GYRO_DLPFCFG_0 0
```

GYRO_DLPFCFG (0..7) — choose cutoff/noise BW set (when DLPF is ON). Tip: start with GYRO_DLPFCFG_3 or _2 for balanced noise/latency.

Definition at line 415 of file [ICM20948_regs.h](#).

7.23.1.117 GYRO_DLPFCFG_1

```
#define GYRO_DLPFCFG_1 1
```

Definition at line 416 of file [ICM20948_regs.h](#).

7.23.1.118 GYRO_DLPFCFG_2

```
#define GYRO_DLPFCFG_2 2
```

Definition at line 417 of file [ICM20948_regs.h](#).

7.23.1.119 GYRO_DLPFCFG_3

```
#define GYRO_DLPFCFG_3 3
```

Definition at line 418 of file [ICM20948_regs.h](#).

7.23.1.120 GYRO_DLPFCFG_4

```
#define GYRO_DLPFCFG_4 4
```

Definition at line 419 of file [ICM20948_regs.h](#).

7.23.1.121 GYRO_DLPFCFG_5

```
#define GYRO_DLPFCFG_5 5
```

Definition at line 420 of file [ICM20948_regs.h](#).

7.23.1.122 GYRO_DLPFCFG_6

```
#define GYRO_DLPFCFG_6 6
```

Definition at line 421 of file [ICM20948_regs.h](#).

7.23.1.123 GYRO_DLPFCFG_7

```
#define GYRO_DLPFCFG_7 7
```

Definition at line 422 of file [ICM20948_regs.h](#).

7.23.1.124 GYRO_DLPFCFG_MASK

```
#define GYRO_DLPFCFG_MASK (0x7u << GYRO_DLPFCFG_SHIFT)
```

Definition at line 181 of file [ICM20948_regs.h](#).

7.23.1.125 GYRO_DLPFCFG_SHIFT

```
#define GYRO_DLPFCFG_SHIFT 5
```

Definition at line 180 of file [ICM20948_regs.h](#).

7.23.1.126 GYRO_FCHOICE

```
#define GYRO_FCHOICE BIT(0) /* when 0: DLPF on, when 1: off (per datasheet) */
```

Definition at line 173 of file [ICM20948_regs.h](#).

7.23.1.127 GYRO_FCHOICE_BYPASS

```
#define GYRO_FCHOICE_BYPASS 0
```

Path select for gyro:

- GYRO_FCHOICE_BYPASS : DLPF bypassed (widest BW, fastest)
- GYRO_FCHOICE_DLPF : DLPF enabled (use GYRO_DLPFCFG + SMPLRT_DIV)

Definition at line 407 of file [ICM20948_regs.h](#).

7.23.1.128 GYRO_FCHOICE_DLPF

```
#define GYRO_FCHOICE_DLPF 1
```

Definition at line 408 of file [ICM20948_regs.h](#).

7.23.1.129 GYRO_FS_1000DPS

```
#define GYRO_FS_1000DPS (2u << GYRO_FS_SEL_SHIFT)
```

Definition at line 178 of file [ICM20948_regs.h](#).

7.23.1.130 GYRO_FS_2000DPS

```
#define GYRO_FS_2000DPS (3u << GYRO_FS_SEL_SHIFT)
```

Definition at line 179 of file [ICM20948_regs.h](#).

7.23.1.131 GYRO_FS_250DPS

```
#define GYRO_FS_250DPS (0u << GYRO_FS_SEL_SHIFT)
```

Definition at line 176 of file [ICM20948_regs.h](#).

7.23.1.132 GYRO_FS_500DPS

```
#define GYRO_FS_500DPS (1u << GYRO_FS_SEL_SHIFT)
```

Definition at line 177 of file [ICM20948_regs.h](#).

7.23.1.133 GYRO_FS_SEL_MASK

```
#define GYRO_FS_SEL_MASK (0x3u << GYRO_FS_SEL_SHIFT)
```

Definition at line 175 of file [ICM20948_regs.h](#).

7.23.1.134 GYRO_FS_SEL_SHIFT

```
#define GYRO_FS_SEL_SHIFT 1
```

Definition at line 174 of file [ICM20948_regs.h](#).

7.23.1.135 GYRO_SMPLRT_DIV

```
#define GYRO_SMPLRT_DIV 0x00 /* GYRO_SMPLRT_DIV[7:0] */
```

- Gyro/Accel configuration, offsets, FSYNC, temperature filter

Definition at line 170 of file [ICM20948_regs.h](#).

7.23.1.136 GYRO_SMPLRT_MIN_HZ

```
#define GYRO_SMPLRT_MIN_HZ 4.3f
```

Definition at line [425](#) of file [ICM20948_regs.h](#).

7.23.1.137 GYRO_XOUT_H

```
#define GYRO_XOUT_H 0x33
```

Definition at line [117](#) of file [ICM20948_regs.h](#).

7.23.1.138 GYRO_XOUT_L

```
#define GYRO_XOUT_L 0x34
```

Definition at line [118](#) of file [ICM20948_regs.h](#).

7.23.1.139 GYRO_YOUT_H

```
#define GYRO_YOUT_H 0x35
```

Definition at line [119](#) of file [ICM20948_regs.h](#).

7.23.1.140 GYRO_YOUT_L

```
#define GYRO_YOUT_L 0x36
```

Definition at line [120](#) of file [ICM20948_regs.h](#).

7.23.1.141 GYRO_ZOUT_H

```
#define GYRO_ZOUT_H 0x37
```

Definition at line [121](#) of file [ICM20948_regs.h](#).

7.23.1.142 GYRO_ZOUT_L

```
#define GYRO_ZOUT_L 0x38
```

Definition at line [122](#) of file [ICM20948_regs.h](#).

7.23.1.143 HighAccuracy

```
#define HighAccuracy 3
```

Definition at line [457](#) of file [ICM20948_regs.h](#).

7.23.1.144 Hz10

```
#define Hz10 3
```

Definition at line [443](#) of file [ICM20948_regs.h](#).

7.23.1.145 Hz15

```
#define Hz15 4
```

Definition at line [444](#) of file [ICM20948_regs.h](#).

7.23.1.146 Hz2

```
#define Hz2 0
```

- Friendly ODR presets you can map to dividers

Definition at line [440](#) of file [ICM20948_regs.h](#).

7.23.1.147 Hz20

```
#define Hz20 5
```

Definition at line [445](#) of file [ICM20948_regs.h](#).

7.23.1.148 Hz25

```
#define Hz25 6
```

Definition at line [446](#) of file [ICM20948_regs.h](#).

7.23.1.149 Hz30

```
#define Hz30 7
```

Definition at line [447](#) of file [ICM20948_regs.h](#).

7.23.1.150 Hz6

```
#define Hz6 1
```

Definition at line [441](#) of file [ICM20948_regs.h](#).

7.23.1.151 Hz8

```
#define Hz8 2
```

Definition at line 442 of file [ICM20948_regs.h](#).

7.23.1.152 I2C_MST_CLK_MASK

```
#define I2C_MST_CLK_MASK 0x0F
```

Definition at line 253 of file [ICM20948_regs.h](#).

7.23.1.153 I2C_MST_CTRL

```
#define I2C_MST_CTRL 0x01 /* MULT_MST_EN - I2C_MST_P_NSR I2C_MST_CLK[3:0] */
```

Definition at line 250 of file [ICM20948_regs.h](#).

7.23.1.154 I2C_MST_DELAY_CTRL

```
#define I2C_MST_DELAY_CTRL 0x02 /* DELAY_ES_SHADOW + I2C_SLVx_DELAY_EN bits */
```

Definition at line 255 of file [ICM20948_regs.h](#).

7.23.1.155 I2C_MST_ODR_CONFIG

```
#define I2C_MST_ODR_CONFIG 0x00 /* I2C_MST_ODR_CONFIG[3:0] */
```

- I2C master (aux) interface and slave windows

Definition at line 247 of file [ICM20948_regs.h](#).

7.23.1.156 I2C_MST_P_NSR

```
#define I2C_MST_P_NSR BIT(4)
```

Definition at line 252 of file [ICM20948_regs.h](#).

7.23.1.157 I2C_MST_STATUS

```
#define I2C_MST_STATUS 0x17 /* PASS_THROUGH, *_DONE, *_NACK, LOST_ARB */
```

Definition at line 86 of file [ICM20948_regs.h](#).

7.23.1.158 I2C_SLV0_ADDR

```
#define I2C_SLV0_ADDR 0x03 /* RNW + ID[6:0] */
```

Definition at line 263 of file [ICM20948_regs.h](#).

7.23.1.159 I2C_SLV0_CTRL

```
#define I2C_SLV0_CTRL 0x05 /* EN BYTE_SW REG_DIS GRP LENG[3:0] */
```

Definition at line 266 of file [ICM20948_regs.h](#).

7.23.1.160 I2C_SLV0_DELAY_EN

```
#define I2C_SLV0_DELAY_EN BIT(0)
```

Definition at line 261 of file [ICM20948_regs.h](#).

7.23.1.161 I2C_SLV0_DO

```
#define I2C_SLV0_DO 0x06
```

Definition at line 272 of file [ICM20948_regs.h](#).

7.23.1.162 I2C_SLV0_REG

```
#define I2C_SLV0_REG 0x04
```

Definition at line 265 of file [ICM20948_regs.h](#).

7.23.1.163 I2C_SLV1_ADDR

```
#define I2C_SLV1_ADDR 0x07
```

Definition at line 274 of file [ICM20948_regs.h](#).

7.23.1.164 I2C_SLV1_CTRL

```
#define I2C_SLV1_CTRL 0x09
```

Definition at line 276 of file [ICM20948_regs.h](#).

7.23.1.165 I2C_SLV1_DELAY_EN

```
#define I2C_SLV1_DELAY_EN BIT(1)
```

Definition at line 260 of file [ICM20948_regs.h](#).

7.23.1.166 I2C_SLV1_DO

```
#define I2C_SLV1_DO 0x0A
```

Definition at line 277 of file [ICM20948_regs.h](#).

7.23.1.167 I2C_SLV1_REG

```
#define I2C_SLV1_REG 0x08
```

Definition at line 275 of file [ICM20948_regs.h](#).

7.23.1.168 I2C_SLV2_ADDR

```
#define I2C_SLV2_ADDR 0x0B
```

Definition at line 279 of file [ICM20948_regs.h](#).

7.23.1.169 I2C_SLV2_CTRL

```
#define I2C_SLV2_CTRL 0x0D
```

Definition at line 281 of file [ICM20948_regs.h](#).

7.23.1.170 I2C_SLV2_DELAY_EN

```
#define I2C_SLV2_DELAY_EN BIT(2)
```

Definition at line 259 of file [ICM20948_regs.h](#).

7.23.1.171 I2C_SLV2_DO

```
#define I2C_SLV2_DO 0x0E
```

Definition at line 282 of file [ICM20948_regs.h](#).

7.23.1.172 I2C_SLV2_REG

```
#define I2C_SLV2_REG 0x0C
```

Definition at line 280 of file [ICM20948_regs.h](#).

7.23.1.173 I2C_SLV3_ADDR

```
#define I2C_SLV3_ADDR 0x0F
```

Definition at line 284 of file [ICM20948_regs.h](#).

7.23.1.174 I2C_SLV3_CTRL

```
#define I2C_SLV3_CTRL 0x11
```

Definition at line 286 of file [ICM20948_regs.h](#).

7.23.1.175 I2C_SLV3_DELAY_EN

```
#define I2C_SLV3_DELAY_EN BIT(3)
```

Definition at line 258 of file [ICM20948_regs.h](#).

7.23.1.176 I2C_SLV3_DO

```
#define I2C_SLV3_DO 0x12
```

Definition at line 287 of file [ICM20948_regs.h](#).

7.23.1.177 I2C_SLV3_REG

```
#define I2C_SLV3_REG 0x10
```

Definition at line 285 of file [ICM20948_regs.h](#).

7.23.1.178 I2C_SLV4_ADDR

```
#define I2C_SLV4_ADDR 0x13
```

Definition at line 289 of file [ICM20948_regs.h](#).

7.23.1.179 I2C_SLV4_CTRL

```
#define I2C_SLV4_CTRL 0x15 /* EN BYTE_SW REG_DIS DLY[4:0] */
```

Definition at line 291 of file [ICM20948_regs.h](#).

7.23.1.180 I2C_SLV4_DELAY_EN

```
#define I2C_SLV4_DELAY_EN BIT(4)
```

Definition at line 257 of file [ICM20948_regs.h](#).

7.23.1.181 I2C_SLV4_DI

```
#define I2C_SLV4_DI 0x17
```

Definition at line 294 of file [ICM20948_regs.h](#).

7.23.1.182 I2C_SLV4_DLY_MASK

```
#define I2C_SLV4_DLY_MASK 0x1F
```

Definition at line 292 of file [ICM20948_regs.h](#).

7.23.1.183 I2C_SLV4_DO

```
#define I2C_SLV4_DO 0x16
```

Definition at line 293 of file [ICM20948_regs.h](#).

7.23.1.184 I2C_SLV4_REG

```
#define I2C_SLV4_REG 0x14
```

Definition at line 290 of file [ICM20948_regs.h](#).

7.23.1.185 I2C_SLVx_BYTE_SW

```
#define I2C_SLVx_BYTE_SW BIT(6)
```

Definition at line 268 of file [ICM20948_regs.h](#).

7.23.1.186 I2C_SLVx_EN

```
#define I2C_SLVx_EN BIT(7)
```

Definition at line 267 of file [ICM20948_regs.h](#).

7.23.1.187 I2C_SLVx_GRP

```
#define I2C_SLVx_GRP BIT(4)
```

Definition at line 270 of file [ICM20948_regs.h](#).

7.23.1.188 I2C_SLVx LENG_MASK

```
#define I2C_SLVx LENG_MASK 0x0F
```

Definition at line 271 of file [ICM20948_regs.h](#).

7.23.1.189 I2C_SLVx_REG_DIS

```
#define I2C_SLVx_REG_DIS BIT(5)
```

Definition at line 269 of file [ICM20948_regs.h](#).

7.23.1.190 I2C_SLVx_RNW

```
#define I2C_SLVx_RNW BIT(7)
```

Definition at line 264 of file [ICM20948_regs.h](#).

7.23.1.191 INT1_ACTL

```
#define INT1_ACTL BIT(7) /* active low */
```

Definition at line 65 of file [ICM20948_regs.h](#).

7.23.1.192 INT1_LATCH_INT_EN

```
#define INT1_LATCH_INT_EN BIT(5) /* latch until status read */
```

Definition at line 67 of file [ICM20948_regs.h](#).

7.23.1.193 INT1_OPEN

```
#define INT1_OPEN BIT(6) /* open-drain */
```

Definition at line 66 of file [ICM20948_regs.h](#).

7.23.1.194 INT_ACTL_FSYNC

```
#define INT_ACTL_FSYNC BIT(3)
```

Definition at line 69 of file [ICM20948_regs.h](#).

7.23.1.195 INT_ANYRD_2CLEAR

```
#define INT_ANYRD_2CLEAR BIT(4)
```

Definition at line 68 of file [ICM20948_regs.h](#).

7.23.1.196 INT_DMP_INT1_EN

```
#define INT_DMP_INT1_EN BIT(1)
```

Definition at line 77 of file [ICM20948_regs.h](#).

7.23.1.197 INT_ENABLE

```
#define INT_ENABLE 0x10 /* REG_WOF_EN - WOM_INT_EN PLL_RDY_EN DMP_INT1_EN I2C_MST_INT_EN */
```

Definition at line 73 of file [ICM20948_regs.h](#).

7.23.1.198 INT_ENABLE_1

```
#define INT_ENABLE_1 0x11 /* RAW_DATA_0_RDY_EN */
```

Definition at line 80 of file [ICM20948_regs.h](#).

7.23.1.199 INT_ENABLE_2

```
#define INT_ENABLE_2 0x12 /* FIFO_OVERFLOW_EN[4:0] */
```

Definition at line 83 of file [ICM20948_regs.h](#).

7.23.1.200 INT_ENABLE_3

```
#define INT_ENABLE_3 0x13 /* FIFO_WM_EN[4:0] */
```

Definition at line 84 of file [ICM20948_regs.h](#).

7.23.1.201 INT_I2C_MST_INT_EN

```
#define INT_I2C_MST_INT_EN BIT(0)
```

Definition at line 78 of file [ICM20948_regs.h](#).

7.23.1.202 INT_PIN_CFG

```
#define INT_PIN_CFG 0x0F /* INT1_ACTL INT1_OPEN INT1_LATCH_INT_EN INT_ANYRD_2CLEAR ACTL_FSYNC  
FSYNC_INT_MODE_EN BYPASS_EN - */
```

Definition at line 64 of file [ICM20948_regs.h](#).

7.23.1.203 INT_PLL_RDY_EN

```
#define INT_PLL_RDY_EN BIT(2)
```

Definition at line 76 of file [ICM20948_regs.h](#).

7.23.1.204 INT_RAW_DATA_0_RDY_EN

```
#define INT_RAW_DATA_0_RDY_EN BIT(0)
```

Definition at line 81 of file [ICM20948_regs.h](#).

7.23.1.205 INT_REG_WOF_EN

```
#define INT_REG_WOF_EN BIT(7)
```

Definition at line 74 of file [ICM20948_regs.h](#).

7.23.1.206 INT_STATUS

```
#define INT_STATUS 0x19 /* WOM_INT PLL_RDY_INT DMP_INT1 I2C_MST_INT */
```

Definition at line 96 of file [ICM20948_regs.h](#).

7.23.1.207 INT_STATUS_1

```
#define INT_STATUS_1 0x1A /* RAW_DATA_0_RDY_INT */
```

Definition at line 102 of file [ICM20948_regs.h](#).

7.23.1.208 INT_STATUS_2

```
#define INT_STATUS_2 0x1B /* FIFO_OVERFLOW_INT[4:0] */
```

Definition at line 104 of file [ICM20948_regs.h](#).

7.23.1.209 INT_STATUS_3

```
#define INT_STATUS_3 0x1C /* FIFO_WM_INT[4:0] */
```

Definition at line 105 of file [ICM20948_regs.h](#).

7.23.1.210 INT_WOM_INT_EN

```
#define INT_WOM_INT_EN BIT(3)
```

Definition at line 75 of file [ICM20948_regs.h](#).

7.23.1.211 INTERNAL_20MHZ

```
#define INTERNAL_20MHZ 0x00 /* 0: Internal 20 MHz RC */
```

Definition at line 319 of file [ICM20948_regs.h](#).

7.23.1.212 LowPower

```
#define LowPower 0
```

Generic power or fusion quality modes (BNO-style). Map to sensor-specific sequences inside your driver.

Definition at line 454 of file [ICM20948_regs.h](#).

7.23.1.213 LP_ACCEL_CYCLE

```
#define LP_ACCEL_CYCLE BIT(5)
```

Definition at line 47 of file [ICM20948_regs.h](#).

7.23.1.214 LP_CONFIG

```
#define LP_CONFIG 0x05 /* I2C_MST_CYCLE ACCEL_CYCLE GYRO_CYCLE - */
```

Definition at line 45 of file [ICM20948_regs.h](#).

7.23.1.215 LP_GYRO_CYCLE

```
#define LP_GYRO_CYCLE BIT(4)
```

Definition at line 48 of file [ICM20948_regs.h](#).

7.23.1.216 LP_I2C_MST_CYCLE

```
#define LP_I2C_MST_CYCLE BIT(6)
```

Definition at line 46 of file [ICM20948_regs.h](#).

7.23.1.217 MOD_CTRL_USR

```
#define MOD_CTRL_USR 0x54 /* REG_LP_DMP_EN */
```

Definition at line 237 of file [ICM20948_regs.h](#).

7.23.1.218 MST_LOST_ARB

```
#define MST_LOST_ARB BIT(5)
```

Definition at line 89 of file [ICM20948_regs.h](#).

7.23.1.219 MST_ODR_CFG_MASK

```
#define MST_ODR_CFG_MASK 0x0F
```

Definition at line 248 of file [ICM20948_regs.h](#).

7.23.1.220 MST_PASS_THROUGH

```
#define MST_PASS_THROUGH BIT(7)
```

Definition at line 87 of file [ICM20948_regs.h](#).

7.23.1.221 MST_SLV0_NACK

```
#define MST_SLV0_NACK BIT(0)
```

Definition at line 94 of file [ICM20948_regs.h](#).

7.23.1.222 MST_SLV1_NACK

```
#define MST_SLV1_NACK BIT(1)
```

Definition at line 93 of file [ICM20948_regs.h](#).

7.23.1.223 MST_SLV2_NACK

```
#define MST_SLV2_NACK BIT(2)
```

Definition at line 92 of file [ICM20948_regs.h](#).

7.23.1.224 MST_SLV3_NACK

```
#define MST_SLV3_NACK BIT(3)
```

Definition at line 91 of file [ICM20948_regs.h](#).

7.23.1.225 MST_SLV4_DONE

```
#define MST_SLV4_DONE BIT(6)
```

Definition at line 88 of file [ICM20948_regs.h](#).

7.23.1.226 MST_SLV4_NACK

```
#define MST_SLV4_NACK BIT(4)
```

Definition at line 90 of file [ICM20948_regs.h](#).

7.23.1.227 MULT_MST_EN

```
#define MULT_MST_EN BIT(7)
```

Definition at line 251 of file [ICM20948_regs.h](#).

7.23.1.228 ODR_ALIGN_EN

```
#define ODR_ALIGN_EN 0x09 /* ODR_ALIGN_EN */
```

Definition at line 197 of file [ICM20948_regs.h](#).

7.23.1.229 ODR_ALIGN_EN_BIT

```
#define ODR_ALIGN_EN_BIT BIT(0)
```

Definition at line 198 of file [ICM20948_regs.h](#).

7.23.1.230 PWR_CLKSEL_AUTO

```
#define PWR_CLKSEL_AUTO 0x01 /* typical: auto selects best source */
```

Definition at line 57 of file [ICM20948_regs.h](#).

7.23.1.231 PWR_CLKSEL_INT_20MHZ

```
#define PWR_CLKSEL_INT_20MHZ 0x01
```

Definition at line 56 of file [ICM20948_regs.h](#).

7.23.1.232 PWR_CLKSEL_MASK

```
#define PWR_CLKSEL_MASK 0x07
```

Definition at line 55 of file [ICM20948_regs.h](#).

7.23.1.233 PWR_DEVICE_RESET

```
#define PWR_DEVICE_RESET BIT(7)
```

Definition at line 51 of file [ICM20948_regs.h](#).

7.23.1.234 PWR_DISABLE_ACCEL

```
#define PWR_DISABLE_ACCEL BIT(3)
```

Definition at line 60 of file [ICM20948_regs.h](#).

7.23.1.235 PWR_DISABLE_GYRO

```
#define PWR_DISABLE_GYRO BIT(0)
```

Definition at line 61 of file [ICM20948_regs.h](#).

7.23.1.236 PWR_LP_EN

```
#define PWR_LP_EN BIT(5)
```

Definition at line 53 of file [ICM20948_regs.h](#).

7.23.1.237 PWR_MGMT_1

```
#define PWR_MGMT_1 0x06 /* DEVICE_RESET SLEEP LP_EN - TEMP_DIS CLKSEL[2:0] */
```

Definition at line 50 of file [ICM20948_regs.h](#).

7.23.1.238 PWR_MGMT_2

```
#define PWR_MGMT_2 0x07 /* - DISABLE_ACCEL DISABLE_GYRO */
```

Definition at line 59 of file [ICM20948_regs.h](#).

7.23.1.239 PWR_SLEEP

```
#define PWR_SLEEP BIT(6)
```

Definition at line 52 of file [ICM20948_regs.h](#).

7.23.1.240 PWR_TEMP_DIS

```
#define PWR_TEMP_DIS BIT(3)
```

Definition at line 54 of file [ICM20948_regs.h](#).

7.23.1.241 REG_BANK_SEL

```
#define REG_BANK_SEL 0x7F /* USER_BANK[1:0] */
```

Definition at line 140 of file [ICM20948_regs.h](#).

7.23.1.242 REG_BANK_SEL_USER_BANK_MASK

```
#define REG_BANK_SEL_USER_BANK_MASK (0x3u << REG_BANK_SEL_USER_BANK_SHIFT)
```

Definition at line 303 of file [ICM20948_regs.h](#).

7.23.1.243 REG_BANK_SEL_USER_BANK_SHIFT

```
#define REG_BANK_SEL_USER_BANK_SHIFT 4
```

Definition at line 302 of file [ICM20948_regs.h](#).

7.23.1.244 REG_LP_DMP_EN

```
#define REG_LP_DMP_EN BIT(7)
```

Definition at line 238 of file [ICM20948_regs.h](#).

7.23.1.245 Regular

```
#define Regular 1
```

Definition at line 455 of file [ICM20948_regs.h](#).

7.23.1.246 SELF_TEST_X_ACCEL

```
#define SELF_TEST_X_ACCEL 0x0E /* XA_ST_DATA[7:0] */
```

Definition at line 150 of file [ICM20948_regs.h](#).

7.23.1.247 SELF_TEST_X_GYRO

```
#define SELF_TEST_X_GYRO 0x02 /* XG_ST_DATA[7:0] */
```

- Self-test / accel offsets / timebase

Definition at line 147 of file [ICM20948_regs.h](#).

7.23.1.248 SELF_TEST_Y_ACCEL

```
#define SELF_TEST_Y_ACCEL 0x0F /* YA_ST_DATA[7:0] */
```

Definition at line 151 of file [ICM20948_regs.h](#).

7.23.1.249 SELF_TEST_Y_GYRO

```
#define SELF_TEST_Y_GYRO 0x03 /* YG_ST_DATA[7:0] */
```

Definition at line 148 of file [ICM20948_regs.h](#).

7.23.1.250 SELF_TEST_Z_ACCEL

```
#define SELF_TEST_Z_ACCEL 0x10 /* ZA_ST_DATA[7:0] */
```

Definition at line 152 of file [ICM20948_regs.h](#).

7.23.1.251 SELF_TEST_Z_GYRO

```
#define SELF_TEST_Z_GYRO 0x04 /* ZG_ST_DATA[7:0] */
```

Definition at line 149 of file [ICM20948_regs.h](#).

7.23.1.252 STS_DMP_INT1

```
#define STS_DMP_INT1 BIT(1)
```

Definition at line 99 of file [ICM20948_regs.h](#).

7.23.1.253 STS_I2C_MST_INT

```
#define STS_I2C_MST_INT BIT(0)
```

Definition at line 100 of file [ICM20948_regs.h](#).

7.23.1.254 STS_PLL_RDY_INT

```
#define STS_PLL_RDY_INT BIT(2)
```

Definition at line 98 of file [ICM20948_regs.h](#).

7.23.1.255 STS_RAW_DATA_0_RDY_INT

```
#define STS_RAW_DATA_0_RDY_INT BIT(0)
```

Definition at line 103 of file [ICM20948_regs.h](#).

7.23.1.256 STS_WOM_INT

```
#define STS_WOM_INT BIT(3)
```

Definition at line 97 of file [ICM20948_regs.h](#).

7.23.1.257 TEMP_CONFIG

```
#define TEMP_CONFIG 0x53 /* TEMP_DLPFCFG[2:0] */
```

Definition at line 233 of file [ICM20948_regs.h](#).

7.23.1.258 TEMP_DLPFCFG_MASK

```
#define TEMP_DLPFCFG_MASK (0x7u << TEMP_DLPFCFG_SHIFT)
```

Definition at line 235 of file [ICM20948_regs.h](#).

7.23.1.259 TEMP_DLPFCFG_SHIFT

```
#define TEMP_DLPFCFG_SHIFT 5
```

Definition at line 234 of file [ICM20948_regs.h](#).

7.23.1.260 TEMP_OUT_H

```
#define TEMP_OUT_H 0x39
```

Definition at line 123 of file [ICM20948_regs.h](#).

7.23.1.261 TEMP_OUT_L

```
#define TEMP_OUT_L 0x3A
```

Definition at line 124 of file [ICM20948_regs.h](#).

7.23.1.262 TIMEBASE_CORRECTION_PLL

```
#define TIMEBASE_CORRECTION_PLL 0x28 /* TBC_PLL[7:0] */
```

Definition at line 161 of file [ICM20948_regs.h](#).

7.23.1.263 USER_BANK_0

```
#define USER_BANK_0 BANK0
```

Definition at line 304 of file [ICM20948_regs.h](#).

7.23.1.264 USER_BANK_1

```
#define USER_BANK_1 BANK1
```

Definition at line 305 of file [ICM20948_regs.h](#).

7.23.1.265 USER_BANK_2

```
#define USER_BANK_2 BANK2
```

Definition at line 306 of file [ICM20948_regs.h](#).

7.23.1.266 USER_BANK_3

```
#define USER_BANK_3 BANK3
```

Definition at line 307 of file [ICM20948_regs.h](#).

7.23.1.267 USER_CTRL

```
#define USER_CTRL 0x03 /* DMP_EN FIFO_EN I2C_MST_EN I2C_IF_DIS DMP_RST SRAM_RST I2C_MST_RST -  
*/
```

Definition at line 35 of file [ICM20948_regs.h](#).

7.23.1.268 USER_CTRL_DMP_EN

```
#define USER_CTRL_DMP_EN BIT(7)
```

Definition at line 37 of file [ICM20948_regs.h](#).

7.23.1.269 USER_CTRL_DMP_RST

```
#define USER_CTRL_DMP_RST BIT(3)
```

Definition at line 41 of file [ICM20948_regs.h](#).

7.23.1.270 USER_CTRL_FIFO_EN

```
#define USER_CTRL_FIFO_EN BIT(6)
```

Definition at line 38 of file [ICM20948_regs.h](#).

7.23.1.271 USER_CTRL_I2C_IF_DIS

```
#define USER_CTRL_I2C_IF_DIS BIT(4)
```

Definition at line 40 of file [ICM20948_regs.h](#).

7.23.1.272 USER_CTRL_I2C_MST_EN

```
#define USER_CTRL_I2C_MST_EN BIT(5)
```

Definition at line 39 of file [ICM20948_regs.h](#).

7.23.1.273 USER_CTRL_I2C_MST_RST

```
#define USER_CTRL_I2C_MST_RST BIT(1)
```

Definition at line 43 of file [ICM20948_regs.h](#).

7.23.1.274 USER_CTRL_SRAM_RST

```
#define USER_CTRL_SRAM_RST BIT(2)
```

Definition at line 42 of file [ICM20948_regs.h](#).

7.23.1.275 WHO_AM_I

```
#define WHO_AM_I 0x00 /* WHO_AM_I[7:0] */
```

7Semi comment style

- Full ICM-20948 register map (Banks 0–3) + key bit fields
- Simple, portable #defines for firmware use
- Datasheet naming kept where practical; fields grouped by bank

Notes

- WHO_AM_I expected value: 0xEA
- Select register bank via REG_BANK_SEL in any bank
- Use BANK(n) helper or write raw values 0x00/0x10/0x20/0x30
- Identity / power / interrupts / sensor data

Definition at line 32 of file [ICM20948_regs.h](#).

7.23.1.276 WHO_AM_I_VAL

```
#define WHO_AM_I_VAL 0xEA
```

Definition at line 33 of file [ICM20948_regs.h](#).

7.23.1.277 WOF_DEGLITCH_EN

```
#define WOF_DEGLITCH_EN BIT(6)
```

Definition at line 229 of file [ICM20948_regs.h](#).

7.23.1.278 WOF_EDGE_INT

```
#define WOF_EDGE_INT BIT(5)
```

Definition at line 230 of file [ICM20948_regs.h](#).

7.23.1.279 XA_OFFS_H

```
#define XA_OFFS_H 0x14 /* XA_OFFS[14:7] */
```

Definition at line 154 of file [ICM20948_regs.h](#).

7.23.1.280 XA_OFFS_L

```
#define XA_OFFS_L 0x15 /* XA_OFFS[6:0] */
```

Definition at line 155 of file [ICM20948_regs.h](#).

7.23.1.281 XG_OFFS_USRH

```
#define XG_OFFS_USRH 0x03
```

Definition at line 190 of file [ICM20948_regs.h](#).

7.23.1.282 XG_OFFS_USRL

```
#define XG_OFFS_USRL 0x04
```

Definition at line 191 of file [ICM20948_regs.h](#).

7.23.1.283 XGYRO_CTEN

```
#define XGYRO_CTEN BIT(5)
```

Definition at line 184 of file [ICM20948_regs.h](#).

7.23.1.284 YA_OFFS_H

```
#define YA_OFFS_H 0x17 /* YA_OFFS[14:7] */
```

Definition at line 156 of file [ICM20948_regs.h](#).

7.23.1.285 YA_OFFS_L

```
#define YA_OFFS_L 0x18 /* YA_OFFS[6:0] */
```

Definition at line 157 of file [ICM20948_regs.h](#).

7.23.1.286 YG_OFFS_USRH

```
#define YG_OFFS_USRH 0x05
```

Definition at line 192 of file [ICM20948_regs.h](#).

7.23.1.287 YG_OFFS_USRL

```
#define YG_OFFS_USRL 0x06
```

Definition at line 193 of file [ICM20948_regs.h](#).

7.23.1.288 YGYRO_CTEN

```
#define YGYRO_CTEN BIT(4)
```

Definition at line 185 of file [ICM20948_regs.h](#).

7.23.1.289 ZA_OFFS_H

```
#define ZA_OFFS_H 0x1A /* ZA_OFFS[14:7] */
```

Definition at line 158 of file [ICM20948_regs.h](#).

7.23.1.290 ZA_OFFS_L

```
#define ZA_OFFS_L 0x1B /* ZA_OFFS[6:0] */
```

Definition at line 159 of file [ICM20948_regs.h](#).

7.23.1.291 ZG_OFFS_USRH

```
#define ZG_OFFS_USRH 0x07
```

Definition at line 194 of file [ICM20948_regs.h](#).

7.23.1.292 ZG_OFFS_USRL

```
#define ZG_OFFS_USRL 0x08
```

Definition at line 195 of file [ICM20948_regs.h](#).

7.23.1.293 ZGYRO_CTEN

```
#define ZGYRO_CTEN BIT(3)
```

Definition at line 186 of file [ICM20948_regs.h](#).

7.24 ICM20948_regs.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ICM20948_REGS_H
00002 #define ICM20948_REGS_H
00003
00004 /**
00005  * 7Semi comment style
00006  * - Full ICM-20948 register map (Banks 0-3) + key bit fields
00007  * - Simple, portable #defines for firmware use
00008  * - Datasheet naming kept where practical; fields grouped by bank
00009  *
00010  * Notes
00011  * - WHO_AM_I expected value: 0xEA
00012  * - Select register bank via REG_BANK_SEL in any bank
00013  * - Use BANK(n) helper or write raw values 0x00/0x10/0x20/0x30
00014  */
00015
00016 /* ----- Common helpers ----- */
00017 #ifndef BIT
00018 #define BIT(n) (1u < (n))
00019 #endif
00020
00021 // #define BANK(n) ((uint8_t)((n) < 4))
00022 // #define BANK0 BANK(0)
00023 // #define BANK1 BANK(1)
00024 // #define BANK2 BANK(2)
00025 // #define BANK3 BANK(3)
00026
00027 /* ===== */
00028 /* BANK 0 */
00029 /* ===== */
00030
00031 /** - Identity / power / interrupts / sensor data */
00032 #define WHO_AM_I 0x00 /* WHO_AM_I[7:0] */
00033 #define WHO_AM_I_VAL 0xEA
00034
00035 #define USER_CTRL 0x03 /* DMP_EN FIFO_EN I2C_MST_EN I2C_IF_DIS DMP_RST SRAM_RST I2C_MST_RST - */
00036 /* bits */
00037 #define USER_CTRL_DMP_EN BIT(7)
00038 #define USER_CTRL_FIFO_EN BIT(6)
00039 #define USER_CTRL_I2C_MST_EN BIT(5)
00040 #define USER_CTRL_I2C_IF_DIS BIT(4)
00041 #define USER_CTRL_DMP_RST BIT(3)
00042 #define USER_CTRL_SRAM_RST BIT(2)
00043 #define USER_CTRL_I2C_MST_RST BIT(1)
00044
00045 #define LP_CONFIG 0x05 /* I2C_MST_CYCLE ACCEL_CYCLE GYRO_CYCLE - */
00046 #define LP_I2C_MST_CYCLE BIT(6)
00047 #define LP_ACCEL_CYCLE BIT(5)
00048 #define LP_GYRO_CYCLE BIT(4)
00049
00050 #define PWR_MGMT_1 0x06 /* DEVICE_RESET SLEEP LP_EN - TEMP_DIS CLKSEL[2:0] */
00051 #define PWR_DEVICE_RESET BIT(7)
00052 #define PWR_SLEEP BIT(6)
00053 #define PWR_LP_EN BIT(5)
00054 #define PWR_TEMP_DIS BIT(3)
00055 #define PWR_CLKSEL_MASK 0x07
00056 #define PWR_CLKSEL_INT_20MHZ 0x01
00057 #define PWR_CLKSEL_AUTO 0x01 /* typical: auto selects best source */
00058
00059 #define PWR_MGMT_2 0x07 /* - DISABLE_ACCEL DISABLE_GYRO */
00060 #define PWR_DISABLE_ACCEL BIT(3)
00061 #define PWR_DISABLE_GYRO BIT(0)
00062
00063 //Interrupt Configs register
00064 #define INT_PIN_CFG 0x0F /* INT1_ACTL INT1_OPEN INT1_LATCH_INT_EN INT_ANYRD_2CLEAR ACTL_FSYNC
FSYNC_INT_MODE_EN BYPASS_EN - */
00065 #define INT1_ACTL BIT(7) /* active low */
00066 #define INT1_OPEN BIT(6) /* open-drain */
00067 #define INT1_LATCH_INT_EN BIT(5) /* latch until status read */
00068 #define INT_ANYRD_2CLEAR BIT(4)
00069 #define INT_ACTL_FSYNC BIT(3)
00070 #define FSYNC_INT_MODE_EN BIT(2)
00071 #define BYPASS_EN BIT(1) /* I2C bypass to aux devices */
00072
00073 #define INT_ENABLE 0x10 /* REG_WOF_EN - WOM_INT_EN PLL_RDY_EN DMP_INT1_EN I2C_MST_INT_EN */
00074 #define INT_REG_WOF_EN BIT(7)
00075 #define INT_WOM_INT_EN BIT(3)
00076 #define INT_PLL_RDY_EN BIT(2)
00077 #define INT_DMP_INT1_EN BIT(1)
00078 #define INT_I2C_MST_INT_EN BIT(0)
00079
00080 #define INT_ENABLE_1 0x11 /* RAW_DATA_0_RDY_EN */
00081 #define INT_RAW_DATA_0_RDY_EN BIT(0)

```



```

00082
00083 #define INT_ENABLE_2 0x12 /* FIFO_OVERFLOW_EN[4:0] */
00084 #define INT_ENABLE_3 0x13 /* FIFO_WM_EN[4:0] */
00085
00086 #define I2C_MST_STATUS 0x17 /* PASS_THROUGH, *_DONE, *_NACK, LOST_ARB */
00087 #define MST_PASS_THROUGH BIT(7)
00088 #define MST_SLV4_DONE BIT(6)
00089 #define MST_LOST_ARB BIT(5)
00090 #define MST_SLV4_NACK BIT(4)
00091 #define MST_SLV3_NACK BIT(3)
00092 #define MST_SLV2_NACK BIT(2)
00093 #define MST_SLV1_NACK BIT(1)
00094 #define MST_SLV0_NACK BIT(0)
00095
00096 #define INT_STATUS 0x19 /* WOM_INT PLL_RDY_INT DMP_INT1 I2C_MST_INT */
00097 #define STS_WOM_INT BIT(3)
00098 #define STS_PLL_RDY_INT BIT(2)
00099 #define STS_DMP_INT1 BIT(1)
00100 #define STS_I2C_MST_INT BIT(0)
00101
00102 #define INT_STATUS_1 0x1A /* RAW_DATA_0_RDY_INT */
00103 #define STS_RAW_DATA_0_RDY_INT BIT(0)
00104 #define INT_STATUS_2 0x1B /* FIFO_OVERFLOW_INT[4:0] */
00105 #define INT_STATUS_3 0x1C /* FIFO_WM_INT[4:0] */
00106
00107 #define DELAY_TIMEH 0x28
00108 #define DELAY_TIMEL 0x29
00109
00110 /* - Sensor outputs */
00111 #define ACCEL_XOUT_H 0x2D
00112 #define ACCEL_XOUT_L 0x2E
00113 #define ACCEL_YOUT_H 0x2F
00114 #define ACCEL_YOUT_L 0x30
00115 #define ACCEL_ZOUT_H 0x31
00116 #define ACCEL_ZOUT_L 0x32
00117 #define GYRO_XOUT_H 0x33
00118 #define GYRO_XOUT_L 0x34
00119 #define GYRO_YOUT_H 0x35
00120 #define GYRO_YOUT_L 0x36
00121 #define GYRO_ZOUT_H 0x37
00122 #define GYRO_ZOUT_L 0x38
00123 #define TEMP_OUT_H 0x39
00124 #define TEMP_OUT_L 0x3A
00125
00126 /* - External sensor shadow data (aux I2C) */
00127 #define EXT_SLV_SENS_DATA_00 0x3B
00128 #define EXT_SLV_SENS_DATA_23 0x52 /* contiguous range 0x3B..0x52 */
00129
00130 #define FIFO_EN_1 0x66 /* SLV3..SLV0 FIFO_EN */
00131 #define FIFO_EN_2 0x67 /* ACCEL GYRO_Z/Y/X TEMP FIFO_EN */
00132 #define FIFO_RST 0x68 /* FIFO_RESET[4:0] */
00133 #define FIFO_MODE 0x69 /* FIFO_MODE[4:0] */
00134 #define FIFO_COUNTH 0x70
00135 #define FIFO_COUNTL 0x71
00136 #define FIFO_R_W 0x72
00137 // #define DATA_RDY_STATUS 0x74 /* WOF_STATUS RAW_DATA_RDY[3:0] */
00138 #define FIFO_CFG 0x76
00139
00140 #define REG_BANK_SEL 0x7F /* USER_BANK[1:0] */
00141
00142 /* ===== */
00143 /* BANK 1 */
00144 /* ===== */
00145
00146 /** - Self-test / accel offsets / timebase */
00147 #define SELF_TEST_X_GYRO 0x02 /* XG_ST_DATA[7:0] */
00148 #define SELF_TEST_Y_GYRO 0x03 /* YG_ST_DATA[7:0] */
00149 #define SELF_TEST_Z_GYRO 0x04 /* ZG_ST_DATA[7:0] */
00150 #define SELF_TEST_X_ACCEL 0x0E /* XA_ST_DATA[7:0] */
00151 #define SELF_TEST_Y_ACCEL 0x0F /* YA_ST_DATA[7:0] */
00152 #define SELF_TEST_Z_ACCEL 0x10 /* ZA_ST_DATA[7:0] */
00153
00154 #define XA_OFFS_H 0x14 /* XA_OFFS[14:7] */
00155 #define XA_OFFS_L 0x15 /* XA_OFFS[6:0] */
00156 #define YA_OFFS_H 0x17 /* YA_OFFS[14:7] */
00157 #define YA_OFFS_L 0x18 /* YA_OFFS[6:0] */
00158 #define ZA_OFFS_H 0x1A /* ZA_OFFS[14:7] */
00159 #define ZA_OFFS_L 0x1B /* ZA_OFFS[6:0] */
00160
00161 #define TIMEBASE_CORRECTION_PLL 0x28 /* TBC_PLL[7:0] */
00162
00163 #define BANK1_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
00164
00165 /* ===== */
00166 /* BANK 2 */
00167 /* ===== */
00168

```

```

00169 /** - Gyro/Accel configuration, offsets, FSYNC, temperature filter */
00170 #define GYRO_SMPLRT_DIV 0x00 /* GYRO_SMPLRT_DIV[7:0] */
00171
00172 #define GYRO_CONFIG_1 0x01 /* GYRO_DLPFCFG[2:0] GYRO_FS_SEL[1:0] GYRO_FCHOICE */
00173 #define GYRO_FCHOICE BIT(0) /* when 0: DLPF on, when 1: off (per datasheet) */
00174 #define GYRO_FS_SEL_SHIFT 1
00175 #define GYRO_FS_SEL_MASK (0x3u < GYRO_FS_SEL_SHIFT)
00176 #define GYRO_FS_250DPS (0u < GYRO_FS_SEL_SHIFT)
00177 #define GYRO_FS_500DPS (1u < GYRO_FS_SEL_SHIFT)
00178 #define GYRO_FS_1000DPS (2u < GYRO_FS_SEL_SHIFT)
00179 #define GYRO_FS_2000DPS (3u < GYRO_FS_SEL_SHIFT)
00180 #define GYRO_DLPFCFG_SHIFT 5
00181 #define GYRO_DLPFCFG_MASK (0x7u < GYRO_DLPFCFG_SHIFT)
00182
00183 #define GYRO_CONFIG_2 0x02 /* XGYRO_CTEN YGYRO_CTEN ZGYRO_CTEN GYRO_AVGCFG[2:0] */
00184 #define XGYRO_CTEN BIT(5)
00185 #define YGYRO_CTEN BIT(4)
00186 #define ZGYRO_CTEN BIT(3)
00187 #define GYRO_AVGCFG_SHIFT 0
00188 #define GYRO_AVGCFG_MASK (0x7u < GYRO_AVGCFG_SHIFT)
00189
00190 #define XG_OFFS_USRH 0x03
00191 #define XG_OFFS_USRL 0x04
00192 #define YG_OFFS_USRH 0x05
00193 #define YG_OFFS_USRL 0x06
00194 #define ZG_OFFS_USRH 0x07
00195 #define ZG_OFFS_USRL 0x08
00196
00197 #define ODR_ALIGN_EN 0x09 /* ODR_ALIGN_EN */
00198 #define ODR_ALIGN_EN_BIT BIT(0)
00199
00200 #define ACCEL_SMPLRT_DIV_1 0x10 /* ACCEL_SMPLRT_DIV[11:8] */
00201 #define ACCEL_SMPLRT_DIV_2 0x11 /* ACCEL_SMPLRT_DIV[7:0] */
00202
00203 #define ACCEL_INTEL_CTRL 0x12 /* ACCEL_INTEL_EN ACCEL_INTEL_MODE_INT */
00204 #define ACCEL_INTEL_EN BIT(7)
00205 #define ACCEL_INTEL_MODE_INT BIT(6)
00206
00207 #define ACCEL_WOM_THR 0x13 /* WOM_THRESHOLD[7:0] */
00208
00209 #define ACCEL_CONFIG 0x14 /* ACCEL_DLPFCFG[2:0] ACCEL_FS_SEL[1:0] ACCEL_FCHOICE */
00210 #define ACCEL_FCHOICE BIT(0)
00211 #define ACCEL_FS_SEL_SHIFT 1
00212 #define ACCEL_FS_SEL_MASK (0x3u < ACCEL_FS_SEL_SHIFT)
00213 #define ACCEL_FS_2G (0u < ACCEL_FS_SEL_SHIFT)
00214 #define ACCEL_FS_4G (1u < ACCEL_FS_SEL_SHIFT)
00215 #define ACCEL_FS_8G (2u < ACCEL_FS_SEL_SHIFT)
00216 #define ACCEL_FS_16G (3u < ACCEL_FS_SEL_SHIFT)
00217 #define ACCEL_DLPFCFG_SHIFT 5
00218 #define ACCEL_DLPFCFG_MASK (0x7u < ACCEL_DLPFCFG_SHIFT)
00219
00220 #define ACCEL_CONFIG_2 0x15 /* AX_ST_EN_REG AY_ST_EN_REG AZ_ST_EN_REG DEC3_CFG[1:0] */
00221 #define AX_ST_EN_REG BIT(7)
00222 #define AY_ST_EN_REG BIT(6)
00223 #define AZ_ST_EN_REG BIT(5)
00224 #define DEC3_CFG_SHIFT 0
00225 #define DEC3_CFG_MASK (0x3u < DEC3_CFG_SHIFT)
00226
00227 #define FSYNC_CONFIG 0x52 /* DELAY_TIME_EN WOF_DEGLITCH_EN WOF_EDGE_INT EXT_SYNC_SET[3:0] */
00228 #define DELAY_TIME_EN BIT(7)
00229 #define WOF_DEGLITCH_EN BIT(6)
00230 #define WOF_EDGE_INT BIT(5)
00231 #define EXT_SYNC_SET_MASK 0x0F
00232
00233 #define TEMP_CONFIG 0x53 /* TEMP_DLPFCFG[2:0] */
00234 #define TEMP_DLPFCFG_SHIFT 5
00235 #define TEMP_DLPFCFG_MASK (0x7u < TEMP_DLPFCFG_SHIFT)
00236
00237 #define MOD_CTRL_USR 0x54 /* REG_LP_DMP_EN */
00238 #define REG_LP_DMP_EN BIT(7)
00239
00240 #define BANK2_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
00241
00242 /* ===== */
00243 /* BANK 3 */
00244 /* ===== */
00245
00246 /** - I2C master (aux) interface and slave windows */
00247 #define I2C_MST_ODR_CONFIG 0x00 /* I2C_MST_ODR_CONFIG[3:0] */
00248 #define MST_ODR_CFG_MASK 0x0F
00249
00250 #define I2C_MST_CTRL 0x01 /* MULT_MST_EN - I2C_MST_P_NSR I2C_MST_CLK[3:0] */
00251 #define MULT_MST_EN BIT(7)
00252 #define I2C_MST_P_NSR BIT(4)
00253 #define I2C_MST_CLK_MASK 0x0F
00254
00255 #define I2C_MST_DELAY_CTRL 0x02 /* DELAY_ES_SHADOW + I2C_SLVx_DELAY_EN bits */

```

```

00256 #define DELAY_ES_SHADOW BIT(7)
00257 #define I2C_SLV4_DELAY_EN BIT(4)
00258 #define I2C_SLV3_DELAY_EN BIT(3)
00259 #define I2C_SLV2_DELAY_EN BIT(2)
00260 #define I2C_SLV1_DELAY_EN BIT(1)
00261 #define I2C_SLV0_DELAY_EN BIT(0)
00262
00263 #define I2C_SLV0_ADDR 0x03 /* RNW + ID[6:0] */
00264 #define I2C_SLVx_RNW BIT(7)
00265 #define I2C_SLV0_REG 0x04
00266 #define I2C_SLV0_CTRL 0x05 /* EN BYTE_SW REG_DIS GRP LENG[3:0] */
00267 #define I2C_SLVx_EN BIT(7)
00268 #define I2C_SLVx_BYTE_SW BIT(6)
00269 #define I2C_SLVx_REG_DIS BIT(5)
00270 #define I2C_SLVx_GRP BIT(4)
00271 #define I2C_SLVx_LENG_MASK 0x0F
00272 #define I2C_SLV0_DO 0x06
00273
00274 #define I2C_SLV1_ADDR 0x07
00275 #define I2C_SLV1_REG 0x08
00276 #define I2C_SLV1_CTRL 0x09
00277 #define I2C_SLV1_DO 0x0A
00278
00279 #define I2C_SLV2_ADDR 0x0B
00280 #define I2C_SLV2_REG 0x0C
00281 #define I2C_SLV2_CTRL 0x0D
00282 #define I2C_SLV2_DO 0x0E
00283
00284 #define I2C_SLV3_ADDR 0x0F
00285 #define I2C_SLV3_REG 0x10
00286 #define I2C_SLV3_CTRL 0x11
00287 #define I2C_SLV3_DO 0x12
00288
00289 #define I2C_SLV4_ADDR 0x13
00290 #define I2C_SLV4_REG 0x14
00291 #define I2C_SLV4_CTRL 0x15 /* EN BYTE_SW REG_DIS DLY[4:0] */
00292 #define I2C_SLV4_DLY_MASK 0x1F
00293 #define I2C_SLV4_DO 0x16
00294 #define I2C_SLV4_DI 0x17
00295
00296 #define BANK3_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
00297
00298 /* ===== */
00299 /* REG_BANK_SEL (all banks) */
00300 /* ===== */
00301
00302 #define REG_BANK_SEL_USER_BANK_SHIFT 4
00303 #define REG_BANK_SEL_USER_BANK_MASK (0x3u << REG_BANK_SEL_USER_BANK_SHIFT)
00304 #define USER_BANK_0 BANK0
00305 #define USER_BANK_1 BANK1
00306 #define USER_BANK_2 BANK2
00307 #define USER_BANK_3 BANK3
00308
00309 /* AK09916 magnetometer registers (connected internally) */
00310 #define AK09916_I2C_ADDR 0x0C
00311 #define AK_WIA2 0x01
00312 #define AK_ST1 0x10
00313 #define AK_HXL 0x11
00314 #define AK_ST2 0x18
00315 #define AK_CNTL2 0x31
00316 #define AK_CNTL3 0x32
00317 #define AK_WIA2_VAL 0x09 /* AK09916C WIA2 expected */
00318
00319 #define INTERNAL_20MHZ 0x00 /* 0: Internal 20 MHz RC */
00320 #define AUTO_SEL 0x01 /* 1-5: Auto/PLL preferred (use 1 as default) */
00321 #define CLK_STOP 0x07 /* 7: Stop clock / timing gen reset */
00322
00323
00324 /* ----- Accelerometer Range ----- */
00325 /** - ±2g / ±4g / ±8g / ±16g (maps to FS_SEL 0..3) */
00326 #define g2 0
00327 #define g4 1
00328 #define g8 2
00329 #define g16 3
00330
00331 /* ----- Accelerometer Filter Path (FCHOICE) ----- */
00332 /**
00333  * Path select for accel:
00334  * - ACCEL_FCHOICE_BYPASS : DLPF bypassed (very wide BW, fastest)
00335  * - ACCEL_FCHOICE_DLPF : DLPF enabled (use ACCEL_DLPFCFG + SMP_LRT_DIV)
00336  */
00337 #define ACCEL_FCHOICE_BYPASS 0
00338 #define ACCEL_FCHOICE_DLPF 1
00339
00340 /* ----- Accelerometer DLPF Config (CFG) ----- */
00341 /**
00342  * ACCEL_DLPFCFG (0..7) -- choose cutoff/noise BW set (when DLPF is ON).

```

```

00343  * Tip: start with ACCEL_DLPFCFG_3 for a good noise/latency tradeoff.
00344  */
00345 #define ACCEL_DLPFCFG_0      0
00346 #define ACCEL_DLPFCFG_1      1
00347 #define ACCEL_DLPFCFG_2      2
00348 #define ACCEL_DLPFCFG_3      3
00349 #define ACCEL_DLPFCFG_4      4
00350 #define ACCEL_DLPFCFG_5      5
00351 #define ACCEL_DLPFCFG_6      6
00352 #define ACCEL_DLPFCFG_7      7
00353
00354 #define ACCEL_DEC3_AVG_4      0u /* averages 1 or 4 (depends on FCHOICE) */
00355 #define ACCEL_DEC3_AVG_8      1u
00356 #define ACCEL_DEC3_AVG_16     2u
00357 #define ACCEL_DEC3_AVG_32     3u
00358
00359 /* ---- Accel DLPF ODR helper (Hz) : base 1125 Hz, DIV 0..4095 ---- */
00360 #define ACCEL_SMPLRT_DIV_MIN   0u
00361 #define ACCEL_SMPLRT_DIV_MAX   4095u
00362 #define ACCEL_DLPF_BASE_HZ     1125.0f
00363 #define ACCEL_DLPF_ODR_HZ(div) (ACCEL_DLPF_BASE_HZ / (1.0f + (float)(div)))
00364
00365 /* ---- Accel BYPASS quick reference (FCHOICE=0) ----
00366  * 3dB 1209 Hz, NBW 1248 Hz, output rate 4500 Hz
00367  * (No divider/ODR control in bypass path.)
00368  */
00369 #define ACCEL_BYPASS_3DB_BW_HZ 1209.0f
00370 #define ACCEL_BYPASS_NBW_HZ    1248.0f
00371 #define ACCEL_BYPASS_RATE_HZ   4500.0f
00372
00373 /* (Optional) Accel DLPF nominal bandwidth references (FCHOICE=1)
00374  * Keep here for quick docs; exact values depend on datasheet rev.
00375  */
00376 #define ACCEL_DLPF0_3DB_BW_HZ  246.0f
00377 #define ACCEL_DLPF0_NBW_HZ     265.0f
00378 #define ACCEL_DLPF1_3DB_BW_HZ  246.0f
00379 #define ACCEL_DLPF1_NBW_HZ     265.0f
00380 #define ACCEL_DLPF2_3DB_BW_HZ  111.4f
00381 #define ACCEL_DLPF2_NBW_HZ     136.0f
00382 #define ACCEL_DLPF3_3DB_BW_HZ  50.4f
00383 #define ACCEL_DLPF3_NBW_HZ     68.8f
00384 #define ACCEL_DLPF4_3DB_BW_HZ  23.9f
00385 #define ACCEL_DLPF4_NBW_HZ     34.4f
00386 #define ACCEL_DLPF5_3DB_BW_HZ  11.5f
00387 #define ACCEL_DLPF5_NBW_HZ     17.0f
00388 #define ACCEL_DLPF6_3DB_BW_HZ  5.7f
00389 #define ACCEL_DLPF6_NBW_HZ     8.3f
00390 #define ACCEL_DLPF7_3DB_BW_HZ  473.0f
00391 #define ACCEL_DLPF7_NBW_HZ     499.0f
00392
00393
00394 /* ----- Gyroscope Range ----- */
00395 /** - ±250/±500/±1000/±2000 dps (maps to FS_SEL 0..3) */
00396 #define dps250 0
00397 #define dps500 1
00398 #define dps1000 2
00399 #define dps2000 3
00400
00401 /* ----- Gyroscope Filter Path (FCHOICE) ----- */
00402 /**
00403  * Path select for gyro:
00404  * - GYRO_FCHOICE_BYPASS : DLPF bypassed (widest BW, fastest)
00405  * - GYRO_FCHOICE_DLPF   : DLPF enabled (use GYRO_DLPFCFG + SMPLRT_DIV)
00406  */
00407 #define GYRO_FCHOICE_BYPASS 0
00408 #define GYRO_FCHOICE_DLPF 1
00409
00410 /* ----- Gyroscope DLPF Config (CFG) ----- */
00411 /**
00412  * GYRO_DLPFCFG (0..7) -- choose cutoff/noise BW set (when DLPF is ON).
00413  * Tip: start with GYRO_DLPFCFG_3 or _2 for balanced noise/latency.
00414  */
00415 #define GYRO_DLPFCFG_0      0
00416 #define GYRO_DLPFCFG_1      1
00417 #define GYRO_DLPFCFG_2      2
00418 #define GYRO_DLPFCFG_3      3
00419 #define GYRO_DLPFCFG_4      4
00420 #define GYRO_DLPFCFG_5      5
00421 #define GYRO_DLPFCFG_6      6
00422 #define GYRO_DLPFCFG_7      7
00423
00424 /* ---- Gyro DLPF ODR helper (Hz) : base 1100 Hz, guard ~4.3 Hz ---- */
00425 #define GYRO_SMPLRT_MIN_HZ    4.3f
00426 #define GYRO_DLPF_BASE_HZ     1100.0f
00427 #define GYRO_DLPF_ODR_HZ(rate_request) (rate_request) /* symbolic; your driver computes DIV =
round(1100/rate)-1 */
00428

```

```
00429 /* (Optional) Gyro BW "buckets" to tag configs in UI/logs (symbolic)
00430 * Use these if you want a quick human-readable label for NBW tiers.
00431 * Exact Hz depend on datasheet table; map per your implementation.
00432 */
00433 #define GYRO_BW_ULTRA_WIDE 0 /* bypass path */
00434 #define GYRO_BW_WIDE 1 /* e.g., CFG 0/1 */
00435 #define GYRO_BW_MEDIUM 2 /* e.g., CFG 2/3 */
00436 #define GYRO_BW_NARROW 3 /* e.g., CFG 4/5/6/7 */
00437
00438 /* ----- Convenience: presets (symbolic) ----- */
00439 /** - Friendly ODR presets you can map to dividers */
00440 #define Hz2 0
00441 #define Hz6 1
00442 #define Hz8 2
00443 #define Hz10 3
00444 #define Hz15 4
00445 #define Hz20 5
00446 #define Hz25 6
00447 #define Hz30 7
00448
00449 /* ----- Power / Fusion Modes ----- */
00450 /**
00451 * Generic power or fusion quality modes (BNO-style).
00452 * Map to sensor-specific sequences inside your driver.
00453 */
00454 #define LowPower 0
00455 #define Regular 1
00456 #define EnhancedRegular 2
00457 #define HighAccuracy 3
00458
00459 #endif /* ICM20948_REGS_H */
```

